Supplementary Information for Classification-Based Inference of Regulatory Networks (CIRN) Algorithm

Bayesian Logistic Regression for Directional and Signed Regulatory Reconstruction from Time-Series Data

Giovannie M. Entero[1,3], Neil Jerome A. Egarguin[1], Mark Jayson V. Cortez[1], Maica Krizna A. Gavina[1], Patricia Ann J. Sanchez[2], and Jomar F. Rabajante[1,3]

[1]*Institute of Mathematical Sciences, University of the Philippines Los Baños*
[2]*School of Environmental Science and Management, University of the Philippines Los Baños*
[3]*Graduate School, University of the Philippines Los Baños*

ABSTRACT

This supplementary information provides the detailed theoretical, statistical, diagnostic, reporting, and computational description of the current Classification-Based Inference of Regulatory Networks (CIRN) algorithm used in the accompanying manuscript. CIRN reformulates regulatory network reconstruction from time-series data as a Bayesian classification problem in which the binary response records whether a target variable is locally increasing or decreasing after near-stationary derivative observations are removed. Rather than estimating trajectory magnitudes or fitting the complete functional form of a governing dynamical system, CIRN evaluates whether temporally lagged state, first-derivative, and second-derivative representations of admissible predictors credibly shift the posterior probability of target-state increase. This document expands the main Materials and Methods section by giving the mathematical motivation for directional-response encoding, the construction of derivative-based predictor libraries, lag alignment and response-leakage prevention, the target-wise Bayesian logistic regression model, the highest-density-interval edge rule, the interpretation of posterior edge weights, the distinction between representation-specific, all-predictors, pairwise, and multivariable inference modes, the adaptive-jitter pathway for one-class or nearly one-class responses, and the diagnostic checks required for transparent reporting. It also describes the output compendium, publication-ready tables, coefficient-level and model-level diagnostics, optional validation and sensitivity-analysis outputs, and the modular R implementation used to configure, run, inspect, and reproduce a CIRN analysis. The supplement is intended to make the inferential target, assumptions, limitations, and reporting logic of CIRN explicit: retained edges are uncertainty-aware, temporally ordered, directed, signed, and representation-aware regulatory hypotheses under the declared target set, predictor set, preprocessing settings, lag, inference mode, response construction, diagnostic context, and posterior edge rule. They should not be interpreted as proof of interventional causality, complete mechanistic equations, governing functional-form recovery, or unconditional biological mechanisms.

Contents

Abstract **1**

I Introduction **5**

II Theoretical Foundations **7**
A Regulatory network inference as directional-response classification 7
B State, velocity, and curvature as candidate regulatory representations 8
C Admissible target-predictor structure 8

III Assumptions and Interpretive Scope **9**
A Core assumptions 9
B Delimitations 10
C When CIRN evidence is strongest 10

IV Directional-Response Encoding and Derivative Construction **12**
A Input time series 12
B Analysis variables 12
C Regularized time grid and smoothing 12
D Numerical derivatives 13
E Directional-response encoding 13
F Outlier handling 14

V Framework for Inferring a CIRN Network **14**
A Temporal alignment 14
B Response construction before standardization 14
C Excluding response-defining predictors 15
D Complete algorithm 15

VI Bayesian CIRN Model and Posterior Edge Rule **17**
A Target-wise Bayesian logistic regression 17
B All-predictors model 18
C Prior and posterior computation 18
D CIRN edge definition 19
E Adjacency Matrix and Edge-List Representation 20
F Overall Network Representation and Aggregation 21

VII Inference Modes and Representation-Specific Evidence **23**
A Mode and predictor-set terminology 23
B Flexible CIRN as a multi-resolution framework 23
C Generic interpretation of predictor representations 25
D Sublevel multivariable CIRN 28
E All-predictors CIRN 28
F Pairwise CIRN 28
G Cross-mode consistency 29

VIII Adaptive Jitter and Monotonic Response Cases **30**
A Why adaptive jitter is used 30

B How adaptive jitter is computed . . . 30
C Response jitter versus predictor jitter . . . 32
D Recommended visualization . . . 32
E Recommended table notation . . . 34

IX Diagnostics and Data Sufficiency 34
A Purpose and indexing of diagnostic outputs . . . 34
B MCMC diagnostics . . . 35
C Collinearity diagnostics . . . 38
D Class balance and usable sample size . . . 39
E Adaptive-jitter diagnostics . . . 40
F Interpreting diagnostic mismatches . . . 40

X Robustness and Sensitivity Analyses 42
A Supported sensitivity scenarios . . . 42
B Sensitivity inference scope . . . 43
C Sensitivity outputs . . . 43
D How sensitivity outputs should be interpreted . . . 44
E Ground-truth validation . . . 44

XI CIRN Output Compendium 47
A Major output sheets . . . 47
B Core edge-table columns . . . 49

XII Computational Cost and Implementation 52
A Number of model fits . . . 52
B Runtime considerations . . . 54
C Reproducibility record . . . 56

XIII Manual for the CIRN Algorithm Computational Script 57
A Overview of the current CIRN implementation . . . 57
B Software requirements and package roles . . . 58
C Input file, output directory, and execution workflow . . . 61
D Main configuration fields . . . 63
E Running the script . . . 68
F Modular organization of the R implementation . . . 70
G Minimum reporting checklist . . . 74

XIV Interpretive Cautions 77

XV Contribution and Inferential Position of CIRN 81

XVI Practical End-to-End CIRN Workflow 83

XVII CIRN Studio: Interactive Shiny Application 86
A Illustrative Interface and Example Analysis . . . 87
B Supported Data Sources and Guided Workflow . . . 97
C Reproducibility and Interpretation Safeguards . . . 98
D Local Use, Deployment, and Computational Requirements . . . 99

XVIII Detailed Results and Supporting Analyses of the Main Article **99**
A Geometric Characterization of Representative Dynamics 99
B Classical Lotka–Volterra Dynamics 105
C Stochastic Predator–Prey Dynamics with Demographic Noise 110
D Temporal Reorganization of Regulatory Structure under Lagged Interactions . . 112
E Structural Dependency Recovery in Damped Harmonic Oscillator Dynamics . . 118
F Extended Applications Across Dynamical Systems 121
G System-Level Comparator Recovery Across Benchmark Systems 125

XIX Summary of the Supplementary Material **127**

I. INTRODUCTION

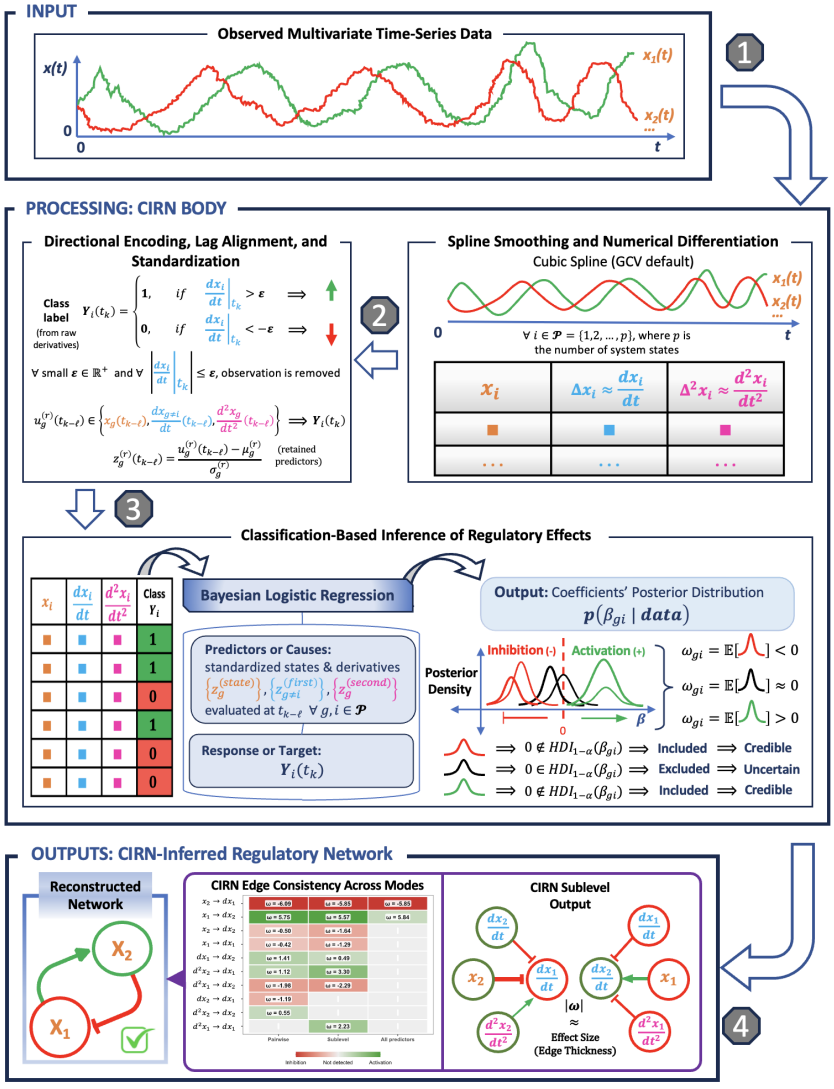
Regulatory network inference from multivariate time-series data is difficult because the desired output is richer than a list of correlated variables. In many biological, ecological, environmental, and synthetic dynamical systems, the analyst seeks relationships that are directional, signed, temporally ordered, and accompanied by a statement of uncertainty. Static dependence methods can identify variables that move together, but they usually do not determine whether a variable activates or inhibits a future target response. Equation-fitting and dynamical-system identification approaches can provide stronger mechanistic interpretation, but they often require dense sampling, explicit functional-form assumptions, candidate model libraries, or parameter identifiability conditions that may be difficult to satisfy in observational datasets.

CIRN addresses this intermediate inferential setting by changing the response being modeled. Instead of fitting the full magnitude of $x_i'(t)$, it models the sign of the local derivative of each target variable. The response $Y_i(t_k)$ is equal to one when the target is increasing and zero when the target is decreasing, after removing near-stationary observations whose derivative magnitude is within a small tolerance. Lagged state, velocity, and curvature representations of admissible predictors are then used in Bayesian logistic regression to estimate how those predictors shift the probability of target-state increase at a later time point. Posterior coefficient distributions define edge inclusion, sign, magnitude, and uncertainty, with retained edges determined by whether the 95% highest density interval excludes zero.

Supplementary Figure 1 provides a schematic overview of this workflow and is intended as a visual guide for the detailed sections that follow. The figure summarizes how CIRN moves from single or multivariate time-series observations to derivative-based predictor construction, directional-response encoding, temporally lagged Bayesian logistic regression, posterior HDI-based edge retention, and final directed and signed network reconstruction. Thus, the schematic should be read as the organizing map for the supplement: the subsequent sections expand each component of the pipeline, beginning with the theoretical basis of directional-response classification and ending with diagnostics, output interpretation, publication-ready reporting, and the modular R implementation.

This supplementary document follows the same methodological logic as the Materials and Methods section of the manuscript, but gives additional detail needed for reproducibility and careful interpretation. The order of operations is especially important: target and predictor sets are declared before preprocessing; derivatives are constructed on the selected analysis variables; the binary response is defined from the raw target first derivative before predictor standardization; response-defining quantities are excluded from the corresponding predictor matrix; candidate predictors are temporally lagged; predictors are standardized only after response construction, lag alignment, predictor filtering, and complete-row filtering; and Bayesian models are fitted separately by target, representation, and analysis mode. This ordering prevents response leakage and preserves the interpretation of a positive coefficient as increased posterior probability of target increase.

The supplement is also designed as a user-facing manual for interpreting CIRN output tables and figures. The algorithm produces retained-edge sheets, all-coefficient summaries including non-retained terms, model-level diagnostics, effective-sample-size summaries, collinearity audits, edge-consistency summaries, optional ground-truth validation, optional sensitivity outputs, publication-ready tables, and reproducibility records. These outputs are intentionally not expected to have the same number of rows: retained edges and coefficient summaries are coefficient-level outputs, diagnostics are indexed by fitted model configurations, and VIF or pairwise-correlation sheets describe the predictor design. Correct interpretation therefore requires matching outputs through target, predictor representation, analysis mode, lag, response source, pairwise predictor when applicable, and model identifiers rather than by row count alone.



Supplementary Figure 1: Algorithmic pipeline of the Classification-Based Inference of Regulatory Networks (CIRN) framework. The CIRN workflow proceeds through four stages indicated by the dark-gray numbered arrows. (1) Multivariate observational time-series data, $\mathbf{X}(t) = \{x_1(t), x_2(t), \ldots, x_p(t)\}$, serve as input, with the underlying directed and signed regulatory structure treated as unknown. (2) CIRN smooths the selected trajectories and constructs dynamic predictor representations through numerical differentiation, yielding state variables $x_g(t)$, first derivatives $dx_g(t)/dt$, and second derivatives $d^2x_g(t)/dt^2$. For each target variable $x_i(t)$, the sign of its unstandardized estimated first derivative defines the binary directional response $Y_i(t_k)$: time points with $dx_i(t_k)/dt > \varepsilon$ are labeled as increasing (1), time points with $dx_i(t_k)/dt < -\varepsilon$ are labeled as decreasing (0), and time points satisfying $|dx_i(t_k)/dt| \leq \varepsilon$ are excluded from the binary response data set rather than assigned to a stationary class. Admissible predictors are then lagged so that predictor representations at $t_{k-\ell}$ precede the target response at t_k, and retained predictor columns are standardized before model fitting. The target first derivative used to define $Y_i(t_k)$ is excluded from its own predictor matrix to prevent response leakage. (3) For each target, CIRN fits Bayesian logistic regression models to estimate how earlier state-, first-derivative-, and second-derivative-based predictor representations shift the modeled probability of target increase. Sublevel CIRN mode fits state, first-derivative, and second-derivative predictors in separate multivariable models, whereas all-predictors CIRN mode fits all eligible representations jointly; when enabled, pairwise CIRN mode evaluates one declared source variable at a time for diagnostic cross-mode comparison. (4) Posterior coefficient distributions, $p(\beta_{gi} \mid \mathcal{D})$, are summarized for edge inference. A predictor-representation-to-target-response edge is retained when the posterior 95% highest-density interval (HDI) for its coefficient excludes zero. The posterior mean coefficient, $\omega_{gi} = \mathbb{E}[\beta_{gi} \mid \mathcal{D}]$, determines the reported sign on the target-response scale: $\omega_{gi} > 0$ indicates activation and $\omega_{gi} < 0$ indicates inhibition. Edge width is proportional to $|\omega_{gi}|$. Green arrows denote activating effects, red arrows denote inhibitory effects, and node sizes are used only for visualization. The resulting outputs summarize directed, signed, uncertainty-aware, and representation-resolved temporal-response regulatory hypotheses; they do not, by themselves, establish direct biochemical mechanism or interventional causality.

II. THEORETICAL FOUNDATIONS

A. Regulatory network inference as directional-response classification

Let

$$\boldsymbol{x}(t) = \{x_1(t), x_2(t), \ldots, x_p(t)\}$$

denote a multivariate time series observed at discrete time points. Many biological, ecological, and complex systems can be represented abstractly as

$$\boldsymbol{x}'(t) = f(t, \boldsymbol{x}),$$

but inferring the complete functional form of f from observational data is often difficult. CIRN does not attempt to identify the full governing equation. Instead, it focuses on a lower-dimensional inferential target: whether the local direction of change of each target variable can be explained by temporally preceding state, velocity, and curvature information from admissible predictors.

This shift from magnitude modeling to sign modeling is the central simplification. Estimating $x_i'(t)$ as a continuous response requires the model to explain the size of the derivative, which can be sensitive to scale, noise, smoothing choices, and unmodeled forcing. Modeling $\operatorname{sgn}\{x_i'(t)\}$ instead asks a more modest but biologically meaningful question: which earlier predictor representations help distinguish target increase from target decrease? This makes the inferential target closer to regulatory switching, response direction, and qualitative dynamical behavior than to full parameter recovery.

Main CIRN Question. Given lagged state and derivative-based representations of candidate predictors, which predictors credibly shift the posterior probability that a target variable is increasing rather than decreasing?

Definition II.1 (Directional Response)**.** For a target variable x_i, the *directional response* at time t_k is the binary variable

$$Y_i(t_k) = \begin{cases} 1, & \text{if} \quad x_i'(t_k) > \varepsilon, \\ 0, & \text{if} \quad x_i'(t_k) < -\varepsilon, \\ \text{removed}, & \text{if} \quad |x_i'(t_k)| \le \varepsilon. \end{cases}$$

The label $Y_i(t_k) = 1$ denotes a local increase of the target, whereas $Y_i(t_k) = 0$ denotes a local decrease.

Remark II.1 (No Neutral Class)**.** Observations with near-zero target derivatives are removed before fitting. They are not modeled as a third class. This choice keeps the statistical model binary and prevents numerically ambiguous derivative signs from defining weak regulatory evidence.

The tolerance ε is therefore not a biological threshold by itself. It is a numerical and interpretive safeguard used to avoid assigning confident increasing or decreasing labels to derivative estimates that are effectively zero. A smaller ε retains more observations but may include ambiguous signs. A larger ε produces cleaner directional labels but may reduce the model-ready sample size. For final analyses, ε should be reported and, for noisy empirical data, may be examined through sensitivity analysis.

B. State, velocity, and curvature as candidate regulatory representations

CIRN is motivated by the assumption that the sign of a target derivative may depend not only on the current state of candidate regulators, but also on their rates of change and local curvature. For target x_i, this motivating relationship can be written as

$$x_i'(t) = f_i\big(t, x_g(t), x'_{g\neq i}(t), x''_g(t)\big), \qquad i, g \in \{1, 2, \ldots, p\}.$$

This expression is not fitted directly. It motivates the construction of a predictor library containing original state variables, first derivatives, and second derivatives.

Definition II.2 (CIRN Predictor Representation)**.** For each candidate predictor x_g, CIRN considers up to three unstandardized predictor representations:

$$u_g^{(\text{state})}(t) = x_g(t), \qquad u_g^{(\text{first})}(t) = x_g'(t), \qquad u_g^{(\text{second})}(t) = x_g''(t).$$

These correspond respectively to state, velocity, and curvature information.

Notation Convention. Throughout this supplement, $u_g^{(r)}$ denotes the unstandardized predictor representation after derivative construction and temporal lagging, whereas $z_g^{(r)}$ denotes the standardized predictor used in the Bayesian logistic regression model. The binary response $Y_i(t_k)$ is constructed before predictor standardization from the raw target first derivative.

Remark II.2 (Interpretation of Derivative Predictors)**.** First-derivative predictors describe whether and how fast a candidate regulator is changing. Second-derivative predictors describe changes in rate, such as acceleration, deceleration, or concavity. These terms are not automatically direct mechanisms. They are temporal representations that may carry regulatory information, especially in oscillatory, delayed, or phase-shifted systems.

C. Admissible target-predictor structure

Let $\mathcal{T}$ denote the declared target set and $\mathcal{P}$ denote the declared predictor set. If both sets are unspecified, CIRN uses all non-time numeric variables as both targets and predictors. If domain knowledge rules out certain directions, the analyst can specify asymmetric target and predictor sets before preprocessing and model fitting.

Definition II.3 (Admissible CIRN Hypothesis)**.** For $x_i \in \mathcal{T}$ and $x_g \in \mathcal{P}$, where $i, g \in \{1, 2, \ldots, p\}$, a CIRN hypothesis asks whether a lagged representation of x_g changes the posterior probability that x_i is increasing. Self-representation terms may be admissible when the analysis design includes them, but the target first derivative used to construct Y_i is excluded from its own predictor matrix. The hypothesis is conditional on the selected lag, preprocessing settings, predictor representation, and model-fitting mode.

Causal Scope. CIRN imposes temporal ordering through lagged predictors, but it does not prove interventional causality. A retained edge is a posterior-supported, temporally ordered regulatory hypothesis under the fitted model.

III. ASSUMPTIONS AND INTERPRETIVE SCOPE

A. Core assumptions

CIRN is intended for derivative-informed, time-ordered inference from multivariate trajectories. Its assumptions are methodological rather than purely biological: they define the conditions under which the fitted coefficients can be interpreted as useful temporal-response evidence.

1. **Observed trajectories contain informative temporal variation.** The measured time series should contain enough increasing and decreasing behavior for at least some target responses to form usable binary classes. If a target derivative is one-class after preprocessing, lagging, and filtering, logistic inference is not identifiable for that response unless the adaptive-jitter pathway is used and clearly disclosed.
2. **Temporal ordering is meaningful under the selected lag.** A retained edge assumes that predictors at $t_{k-\ell}$ are scientifically interpretable as preceding the target response at t_k. The lag ℓ is therefore part of the hypothesis being tested, not a nuisance detail. Changing the lag can change edge support, especially in oscillatory, delayed, or phase-shifted systems.
3. **The declared predictor set contains plausible regulatory information.** CIRN cannot recover an omitted regulator, latent driver, or unmeasured confounder. The target set $\mathcal{T}$ and predictor set $\mathcal{P}$ should be chosen to match the scientific question, and asymmetric candidate graphs should be used when reverse directions are not meaningful.
4. **Derivative estimates are sufficiently stable for classification.** The first derivative defines the response, while first- and second-derivative predictors may enter the model. Spline smoothing, sampling frequency, measurement noise, and irregular temporal spacing therefore affect both the response labels and the predictor representations.
5. **Response leakage is prevented by construction.** The target first derivative used to define $Y_i(t_k)$ must not enter the predictor matrix for the same target fit. If adaptive response jitter is used, jitter-generated response-defining quantities must also be excluded from candidate predictors.
6. **Model-ready sample size is adequate.** The relevant sample size is not only the number of rows in the original file, but the number of usable increasing and decreasing target-response observations after smoothing, derivative estimation, near-zero derivative removal, lagging, predictor filtering, and complete-row filtering. Let $m = \min(n_0, n_1)$, where n_0 and n_1 are the model-ready decreasing and increasing response counts, respectively. In the current implementation, $m < 5$ is treated as below minimum, $5 \leq m < 10$ as a small minority class, and $m/p < 5$ as low minority observations per active predictor. Fits with $m \geq 10$ and at least five minority-class observations per active predictor are considered basically adequate, although larger and more balanced responses are preferable for stable interpretation.
7. **The logistic model is an interpretable approximation.** The fitted model estimates additive effects on the log-odds of target increase. It does not claim that the biological or physical system is governed by a logistic equation. The regression scale is used to summarize directional-response evidence.
8. **Posterior summaries are conditional on the specified model.** HDIs, posterior means, odds ratios, $\hat{R}$, effective sample size, LOO summaries, and VIF diagnostics are interpreted under the chosen preprocessing, lag, representation mode, prior, and likelihood. They do not remove the need for domain review.

B. Delimitations

The current CIRN framework is designed for structural hypothesis generation from observational or simulated time-series data. It does not require interventions, randomized treatments, or known governing equations, but that accessibility also defines its inferential boundary.

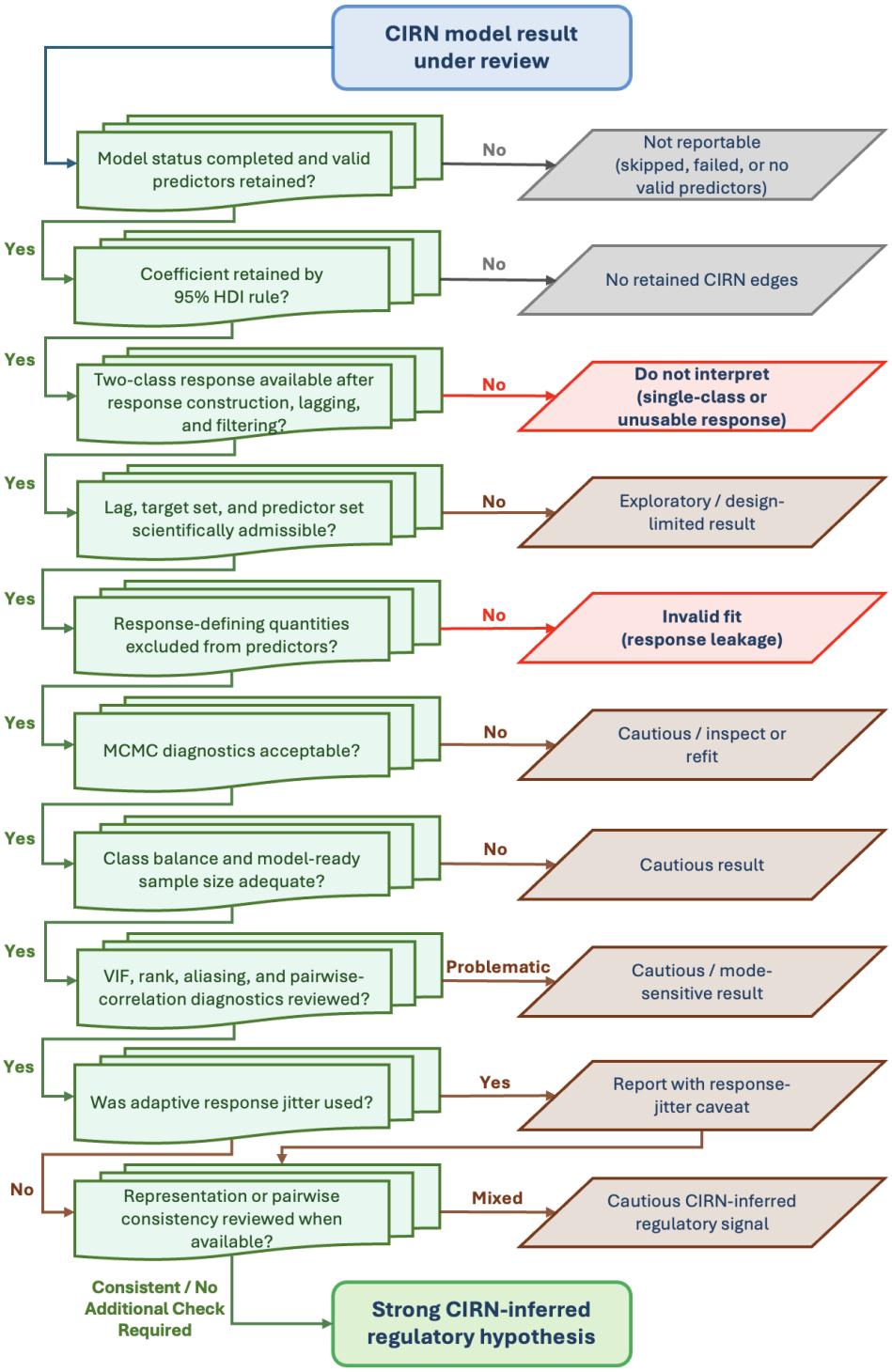
1. CIRN does not establish interventional causality. A retained edge means that a lagged predictor representation credibly shifts the posterior probability of target increase under the fitted model.
2. CIRN does not recover complete governing equations, kinetic parameters, biochemical rate laws, ecological functional responses, or environmental exposure-response mechanisms.
3. CIRN does not explicitly model latent variables, unobserved confounders, higher-order interactions, time-varying coefficients, spatial dependence, or multi-scale forcing unless these are represented through the supplied predictor set and analysis design.
4. CIRN is not a forecasting method. Predictive checks can be useful diagnostics, but edge inclusion is based on posterior coefficient uncertainty rather than forecasting accuracy.
5. CIRN does not automatically solve collinearity among state, velocity, and curvature predictors. Collinearity is reported through VIF, rank, aliasing, and pairwise-correlation diagnostics and must be considered when interpreting signs and mode-specific edges.
6. CIRN is not intended as an omnibus substitute for correlation analysis, Granger Causality, Convergent Cross Mapping (CCM), Partial Cross Mapping (PCM), Ordinary Differential Equation (ODE) fitting, or equation discovery. Rather, it occupies a complementary inferential niche: it provides a directed, signed, temporally ordered, uncertainty-aware evidential layer for posterior-supported target-response regulatory hypotheses, positioned between static dependence screening and full mechanistic reconstruction.

C. When CIRN evidence is strongest

CIRN evidence is most persuasive when the target response has adequate class balance, the lag choice is scientifically defensible, derivative estimates are stable, posterior diagnostics are satisfactory, collinearity is not severe or is carefully interpreted, and retained edges are consistent with domain knowledge or stable across planned robustness checks. Cross-mode support can strengthen interpretation, but it should not be treated mechanically. Pairwise support indicates marginal temporal-response evidence; sublevel multivariable support indicates representation-specific conditional evidence; all-predictors support indicates a more stringent joint conditional signal.

Recommended Reporting Language. A retained edge should be described as a *posterior-supported temporal-response regulatory signal* or a CIRN-*inferred directed and signed regulatory hypothesis*. Avoid language implying proven intervention, direct mechanism, or complete causal identification unless independent experimental evidence supports that interpretation.

Supplementary Figure 2 summarizes how the assumptions above should be checked before assigning interpretive strength to a fitted result. The workflow is intended to support transparent reporting: it separates non-retained or invalid fits from results that can be reported cautiously or as strong CIRN-inferred regulatory hypotheses.



Supplementary Figure 2: CIRN interpretation-readiness checklist. Decision workflow for assigning an interpretation status to a fitted CIRN result or retained edge. The blue header denotes the CIRN result under review, while the light-green stacked boxes represent sequential interpretation checkpoints along the main review path. Gray terminal boxes indicate results that are not reportable or not retained, including skipped, failed, predictor-depleted, or non-HDI-retained fits. Red terminal boxes indicate invalid or uninterpretable fits, such as unusable two-class responses or response leakage. Amber/brown terminal boxes indicate results that may be reported only with caution because of design limitations, diagnostic concerns, class-balance limitations, collinearity, adaptive response jitter, or mixed representation/pairwise support. The final green endpoint denotes a strong CIRN-inferred regulatory hypothesis after the relevant checks have been satisfied or judged not additionally required. The workflow is an interpretation and reporting aid; it does not modify the posterior 95% HDI rule used for edge selection.

IV. DIRECTIONAL-RESPONSE ENCODING AND DERIVATIVE CONSTRUCTION

A. Input time series

CIRN starts from a rectangular multivariate time-series table. The first column should contain the temporal index, denoted here by t, and all remaining columns should contain numeric state variables:

$$\{t, x_1, x_2, \ldots, x_p\}.$$

In the R implementation, this column is stored internally as `time_col`. If `time_col` is left unspecified, the script uses the first column of the input file as the time index. For clarity and reproducible visualization, users are encouraged to name this column with a short label such as `t` or `time`. State-variable names may be letters or short descriptive labels, but very long names should be avoided because they can make network plots and output tables difficult to read.

The time column may be numeric, date-based, or date-time based. Date and date-time values are converted to a numeric scale before smoothing and differentiation. In practice, numeric time indices such as $1, 2, 3, \ldots, n$ are often easiest to inspect and debug, especially in simulation studies. Before derivative construction, observations are ordered by time. If duplicate time values are present, numeric observations sharing the same time value are averaged before smoothing and differentiation. All state variables used as targets or candidate predictors must be numeric; non-numeric state columns are not admissible for derivative-based CIRN fitting.

B. Analysis variables

Let $\mathcal{T}$ denote the set of target variables for which directional responses are modeled, and let $\mathcal{P}$ denote the set of admissible candidate predictors. CIRN constructs derivatives for the selected analysis variables

$$\mathcal{A} = \mathcal{T} \cup \mathcal{P}.$$

Targets and predictors may be identical, partially overlapping, or different, depending on the scientific question. If the same variable appears as both a target and a candidate predictor, the implementation still prevents response leakage by excluding the target first derivative used to define the binary response from the corresponding predictor matrix. Thus, the target derivative can define $Y_i(t_k)$, but that same response-defining quantity is not allowed to act as a predictor for its own fitted response.

C. Regularized time grid and smoothing

CIRN evaluates all selected variables on a common regularized time grid over the observed time domain. If n_{obs} denotes the number of observed time points after ordering and duplicate-time aggregation, and q denotes the number of regularized grid intervals per observed interval, then

$$n_{\text{grid}} = (n_{\text{obs}} - 1)q + 1.$$

The manuscript analyses used $q = 1$ unless stated otherwise. With $q = 1$, the regularized grid contains one interval for each observed interval. For equally spaced data, this coincides with the original sampling grid; for irregularly spaced data, it produces an evenly spaced grid over the same observed time range. Larger values of q densify the derivative grid and should be used only when the analysis plan justifies additional interpolation.

Each selected variable is then smoothed by a cubic smoothing spline. When the smoothing parameter `spar` is not specified, it is selected by generalized cross-validation through the spline-fitting routine. The purpose of smoothing is numerical stabilization: first and second derivatives amplify local noise, so differentiating a raw noisy trajectory can produce unstable signs and curvature estimates. The spline is therefore a preprocessing device for derivative estimation, not a mechanistic model of the underlying biological, ecological, or environmental process.

D. Numerical derivatives

Let $\tilde{x}_g(t_k)$ be the spline-smoothed value of variable x_g at regularized time t_k, and let Δt be the spacing of the regularized grid. For interior grid points, CIRN approximates the first derivative using the centered finite difference

$$x'_g(t_k) \approx \frac{\tilde{x}_g(t_{k+1}) - \tilde{x}_g(t_{k-1})}{2\Delta t},$$

and approximates the second derivative using

$$x''_g(t_k) \approx \frac{\tilde{x}_g(t_{k+1}) - 2\tilde{x}_g(t_k) + \tilde{x}_g(t_{k-1})}{(\Delta t)^2}.$$

The first derivative represents local velocity or direction of change, whereas the second derivative represents local curvature or acceleration of change. Endpoints that cannot support centered finite differences are marked unavailable for derivative-based fitting. Consequently, rows with unavailable derivative or lagged-predictor values may be removed later when the model-ready data matrix is assembled.

E. Directional-response encoding

For each target variable x_i, CIRN converts the estimated first derivative into a binary response before predictor standardization. Given a small tolerance $\varepsilon > 0$, the response at time t_k is

$$Y_i(t_k) = \begin{cases} 1, & x'_i(t_k) > \varepsilon, \\ 0, & x'_i(t_k) < -\varepsilon. \end{cases}$$

Observations satisfying $|x'_i(t_k)| \le \varepsilon$ are removed from the response because they are near-stationary or numerically ambiguous. This step is central to CIRN: the model does not try to predict the magnitude of $x'_i(t_k)$. Instead, it estimates how lagged candidate predictors shift the posterior probability that the target is increasing rather than decreasing.

The predictor library is built from the state, first-derivative, and second-derivative representations of admissible candidate predictors:

$$\left\{ x_g(t_k),\, x'_g(t_k),\, x''_g(t_k) : g \in \mathcal{P} \right\}.$$

These representations are later temporally lagged and standardized before Bayesian logistic regression. The distinction between response construction and predictor construction is important: the raw target first derivative defines the class label, while candidate predictor representations supply the information used to explain that class label under the selected lag.

If the raw directional response contains only one class, or is nearly one-class according to the configured minimum class count, the implementation may attempt adaptive response jitter. In that pathway, very small perturbations are applied to the target trajectory before recomputing the target derivative and response labels. Jitter is used only to obtain a modelable two-class directional response and is explicitly recorded in the output sheets. Fits requiring adaptive response jitter should be interpreted and reported with the corresponding caveat.

F. Outlier handling

The implementation allows optional median absolute deviation screening before spline fitting. When enabled, detected outliers may be kept, winsorized, or removed according to the analysis configuration. In the manuscript analyses, outlier handling should be reported exactly as configured rather than treated as an implicit part of the method. This choice is recorded in the `settings` sheet and should be interpreted as a preprocessing decision, not as part of the core posterior edge-inclusion rule.

Supplementary Figure 1 already summarizes this trajectory-to-response pathway at the full-pipeline level, including spline smoothing, derivative construction, near-zero derivative exclusion, lag alignment, standardization, Bayesian logistic regression, and HDI-based edge retention.

V. FRAMEWORK FOR INFERRING A CIRN NETWORK

A. Temporal alignment

For a specified lag $\ell \geq 1$, predictors are evaluated at $t_{k-\ell}$, whereas the target response is defined at t_k:

$$u_g^{(r)}(t_{k-\ell}) \longrightarrow Y_i(t_k).$$

Here r indexes the predictor representation: state, first derivative, second derivative, or an all-predictors model containing several representations. Only predictors are lagged. The response is not lagged. This is intentional: the fitted model asks whether information available at an earlier time point helps explain whether the target is increasing or decreasing at a later time point. After this alignment and filtering step, the retained $u_g^{(r)}$ columns are standardized and denoted $z_g^{(r)}$ in the fitted model.

After lagging, rows introduced with unavailable predictor values are removed before fitting. This removal is not an additional biological filter; it is a data-alignment step required by the regression design matrix. The lag value ℓ should therefore be treated as part of the scientific hypothesis. In fast systems, $\ell = 1$ may represent the most immediate observed temporal ordering. In systems with delayed responses, phase offsets, or slower regulation, larger lags may be scientifically relevant, but they should be specified before interpretation and reported with the final edge table. Changing ℓ changes the question being asked and may change the retained network.

B. Response construction before standardization

The binary response is always constructed from the raw target first derivative before predictor standardization. Predictors are standardized only after lagging and filtering:

$$z_g^{(r)}(t_{k-\ell}) = \frac{u_g^{(r)}(t_{k-\ell}) - \mu_g^{(r)}}{\sigma_g^{(r)}}.$$

The means $\mu_g^{(r)}$ and standard deviations $\sigma_g^{(r)}$ are computed from the model-ready unstandardized predictor column $u_g^{(r)}$ after lagging and complete-row filtering. Standardization places state, velocity, and curvature predictors on comparable numerical scales, which improves Bayesian fitting and makes posterior coefficients easier to compare within a fitted model.

The ordering matters. If the target derivative were standardized before defining $Y_i(t_k)$, the response would represent whether the target derivative is above or below its own mean, not whether the target is truly increasing or decreasing. CIRN avoids this class-label error by constructing $Y_i(t_k)$

from the raw derivative sign first, then standardizing only retained predictors. This preserves the interpretation of $Y_i = 1$ as target increase and $Y_i = 0$ as target decrease.

C. Excluding response-defining predictors

For target x_i, the first derivative x_i' used to define Y_i is excluded from that target's predictor matrix. This exclusion prevents response leakage, meaning the model is not allowed to use the same quantity that defines the class label as an explanatory predictor for that class label. If adaptive response jitter is used, jitter-generated target columns and jitter-generated response derivatives are also excluded from candidate predictors.

The target state x_i and target second derivative x_i'' may remain eligible when x_i is explicitly declared as a predictor. This is not leakage by itself because these quantities do not directly define the binary response. Their interpretation, however, should be stated carefully. A retained $x_i(t_{k-\ell}) \to dx_i$ term reflects state-dependent self-response support, whereas a retained $x_i''(t_{k-\ell}) \to dx_i$ term reflects curvature-related self-response support. Neither should be described as a separate external regulator unless the scientific context supports that interpretation.

D. Complete algorithm

The implementation-level CIRN workflow is summarized in Supplementary Algorithm S1. This algorithm is the text counterpart of Supplementary Figure 1: the figure provides the visual map, whereas Supplementary Algorithm S1 specifies the ordered computational operations required to reproduce a CIRN analysis. Unlike the shorter algorithm in the main manuscript, the version below is written to be self-contained for manual use; it states the directional-response rule, response-leakage exclusion, model-fitting step, HDI edge rule, and diagnostic outputs directly.

Supplementary Algorithm S1. Implementation-level Classification-Based Inference of Regulatory Networks (CIRN)

Require: Single/Multivariate time-series table $\mathbf{X}(t)$; time column; target set $\mathcal{T}$; predictor set $\mathcal{P}$; lag $\ell \geq 1$; near-zero derivative tolerance $\varepsilon > 0$; representation mode $m \in \{\texttt{sublevel}, \texttt{all_predictors}, \texttt{both}\}$; optional pairwise fitting with pairwise representation mode $m_{\text{pair}} \in \{\texttt{sublevel}, \texttt{all_predictors}, \texttt{both}\}$; adaptive-jitter settings; optional ground-truth validation settings; and optional sensitivity settings with inference scope $s \in \{\texttt{use_config}, \texttt{sublevel}, \texttt{all_predictors}, \texttt{both}, \texttt{pairwise_only}, \texttt{everything}\}$.

Ensure: Retained directed and signed feature-level CIRN edges; posterior summaries; model diagnostics; class-balance and collinearity audits; fitted-model records; mode-consistency summaries; analysis settings; optional ground-truth benchmark results; and optional state-level and feature-level sensitivity-stability summaries.

Identify the time column and all non-time numeric state variables.
if $\mathcal{T}$ or $\mathcal{P}$ is unspecified **then**
    Use all non-time numeric variables for the unspecified set.
end if
Restrict the analysis variables to $\mathcal{A} = \mathcal{T} \cup \mathcal{P}$.
Retain observations with usable time values and numeric measurements for $\mathcal{A}$.
Order observations by time and average duplicate time points when present, and remove missing observations.
Construct a regularized time grid over the observed time range.
for each selected predictor variable $x_g \in \mathcal{A}$ **do**
    Apply the configured outlier-handling rule.
    Fit a cubic smoothing spline to $x_g(t)$.
    Estimate $x_g'(t_k)$ and $x_g''(t_k)$ on the regularized grid using centered finite differences.
end for
for each target $x_i \in \mathcal{T}$ **do**
    Construct the raw directional response from the target derivative:

$$Y_i(t_k) = \begin{cases} 1, & x_i'(t_k) > \varepsilon, \\ 0, & x_i'(t_k) < -\varepsilon, \end{cases} \quad \text{and remove rows with } |x_i'(t_k)| \leq \varepsilon.$$

    Record raw class counts $n_0 = \#\{Y_i = 0\}$, $n_1 = \#\{Y_i = 1\}$, and $m_i = \min(n_0, n_1)$.

if adaptive jitter is enabled and the raw response is one-class or sparse **then**
  Apply the smallest successful target-trajectory jitter level from the configured grid.
  Recompute the target derivative and active directional response.
  Record `response_mode`, `response_source`, `jitter_used`, `jitter_scale`, and class counts.
end if
if the active response has fewer than two classes **then**
  Skip the target fit and record diagnostic status `skipped_single_class`.
  Continue to the next target.
end if
Construct candidate predictor representations

$$u_g^{(r)}(t_k) \in \{x_g(t_k),\, x_g'(t_k),\, x_g''(t_k) : x_g \in \mathcal{P}\}.$$

Exclude response-defining quantities, including $x_i'(t_k)$ and any jitter-generated response columns.
Lag predictor columns by ℓ time steps so predictors at $t_{k-\ell}$ align with $Y_i(t_k)$.
Remove invalid, unavailable, non-finite, all-missing, or zero-variance predictors.
Retain complete model-ready rows and update class counts.
if fewer than two response classes remain after lagging and filtering **then**
  Skip the target fit and record diagnostic status `skipped_after_lagging`.
  Continue to the next target.
end if
Standardize retained predictor columns in the model-ready data.
if $m \in \{\texttt{sublevel}, \texttt{both}\}$ **then**
  for each representation $r \in \{\text{state}, \text{first}, \text{second}\}$ **do**
    Use the eligible standardized predictors $z_g^{(r)}(t_{k-\ell})$ for that representation.
    if no valid predictors remain **then**
      Record diagnostic status `skipped_no_predictors`.
    else
      Fit the Bayesian logistic model

$$\operatorname{logit}\Big(\pi_i^{(r)}(t_k)\Big) = \beta_{0i}^{(r)} + \sum_{g \in \mathcal{P}_i^{(r)}} \beta_{gi}^{(r)} z_g^{(r)}(t_{k-\ell}).$$

      Record posterior summaries, MCMC diagnostics, class-balance summaries, and collinearity diagnostics.
      Retain each coefficient whose 95% HDI excludes zero.
    end if
  end for
end if
if $m \in \{\texttt{all_predictors}, \texttt{both}\}$ **then**
  Fit the joint all-predictors Bayesian logistic model

$$\begin{aligned}\operatorname{logit}(\pi_i(t_k)) = \beta_{0i} &+ \sum_{g \in \mathcal{P}_i^{(\text{state})}} \beta_{gi}^{(\text{state})} z_g^{(\text{state})}(t_{k-\ell}) \\ &+ \sum_{g \in \mathcal{P}_i^{(\text{first})}} \beta_{gi}^{(\text{first})} z_g^{(\text{first})}(t_{k-\ell}) \\ &+ \sum_{g \in \mathcal{P}_i^{(\text{second})}} \beta_{gi}^{(\text{second})} z_g^{(\text{second})}(t_{k-\ell}).\end{aligned}$$

  Record posterior summaries, MCMC diagnostics, class-balance summaries, and collinearity diagnostics.
  Retain each coefficient whose 95% HDI excludes zero.
end if
end for
if pairwise fitting is enabled **then**
  Resolve the configured pairwise representation mode m_{pair}.
  Repeat the target-wise workflow one candidate predictor at a time under m_{pair}.
  Store pairwise retained edges, posterior summaries, diagnostics, and collinearity audits separately from the multivariable CIRN results.
end if
for each retained coefficient β **do**
  Set $\omega = \mathbb{E}[\beta \mid \text{data}]$, compute $\exp(\omega)$, and record the 95% HDI.
  Assign activation if $\omega > 0$ and inhibition if $\omega < 0$ on the target-response scale.
end for

63: **if** a ground-truth signed adjacency matrix is supplied and validation is enabled **then**
64: Preserve the feature-level provenance of every retained state, first-derivative, and second-derivative edge.
65: For state-level benchmarking only, map each representation-specific edge to its corresponding base regulator-to-target relation.
66: Compare the collapsed inferred network with the supplied ground truth and compute the configured signed and structural benchmark measures.
67: **end if**
68: **if** sensitivity analysis is enabled **then**
69: Resolve the sensitivity inference scope s.
70: Construct the configured perturbation scenarios and stochastic replicates.
71: Rerun the CIRN modes selected by s for every valid sensitivity scenario and record completed, skipped, and failed runs.
72: Summarize state-level edge stability after collapsing derivative representations to their corresponding base variables.
73: Summarize feature-level edge stability while preserving the specific predictor representation, such as X, dX, or d^2X.
74: **end if**
75: Return retained feature-level signed edges; posterior coefficient summaries and 95% HDIs; diagnostics; fitted-model records; pairwise and multivariable results; mode-consistency summaries; optional ground-truth benchmark results; optional state-level and feature-level sensitivity-stability summaries; analysis settings; figures; and reproducibility outputs.

The key reproducibility point is that edge interpretation depends on the whole chain of decisions: declared targets and predictors, derivative construction, response filtering, lag alignment, predictor standardization, Bayesian fitting, HDI edge retention, and diagnostic review. Supplementary Algorithm S1 should therefore be read as the implementation-level guide for reproducing the CIRN output compendium, not as a separate inference rule beyond the posterior HDI criterion.

VI. BAYESIAN CIRN MODEL AND POSTERIOR EDGE RULE

A. Target-wise Bayesian logistic regression

For target $x_i(t) \in \mathcal{T}$, predictor representation r, and usable observation times t_k, CIRN models

$$Y_i(t_k) \sim \text{Bernoulli}\Big(\pi_i^{(r)}(t_k)\Big),$$

where $\pi_i^{(r)}(t_k)$ is the modeled probability that $x_i(t)$ is increasing. For each representation r, a sublevel CIRN mode model is

$$\text{logit}\Big(\pi_i^{(r)}(t_k)\Big) = \beta_{0i}^{(r)} + \sum_{g \in \mathcal{P}_i^{(r)}} \beta_{gi}^{(r)} z_g^{(r)}(t_{k-\ell}),$$

where $\mathcal{P}_i^{(r)}$ contains the admissible predictors for target x_i under representation r. Pairwise CIRN mode uses the same model form but restricts the declared source variable according to the pairwise setting.

The probability $\pi_i^{(r)}(t_k)$ is the posterior-modeled probability that target $x_i(t)$ is increasing at time t_k, conditional on the lagged predictor representation values included in the model. The coefficient $\beta_{gi}^{(r)}$ is a log-odds effect: it describes how a one-standard-deviation increase in the standardized lagged predictor representation $z_g^{(r)}(t_{k-\ell})$ changes the log-odds that the target is increasing, holding the other predictors in the same model fixed. A positive coefficient increases the log-odds of target increase. A negative coefficient decreases the log-odds of target increase.

This is a classification model, not a trajectory-magnitude model. The response is the direction of local target change, not the value of the target and not the magnitude of the target derivative. Therefore, a retained coefficient should be interpreted as evidence that a lagged predictor

representation shifts the probability of target increase or decrease. It should not be interpreted as an estimated kinetic parameter, a rate constant, or a complete term in a governing differential equation.

B. All-predictors model

In the joint all-predictors formulation, state, first-derivative, and second-derivative predictors are fitted together:

$$\begin{aligned}\operatorname{logit}(\pi_i(t_k)) = \beta_{0i} &+ \sum_{g\in\mathcal{P}_i^{(\text{state})}} \beta_{gi}^{(\text{state})} z_g^{(\text{state})}(t_{k-\ell}) \\ &+ \sum_{g\in\mathcal{P}_i^{(\text{first})}} \beta_{gi}^{(\text{first})} z_g^{(\text{first})}(t_{k-\ell}) \\ &+ \sum_{g\in\mathcal{P}_i^{(\text{second})}} \beta_{gi}^{(\text{second})} z_g^{(\text{second})}(t_{k-\ell}).\end{aligned}$$

Here, $\mathcal{P}_i^{(\text{state})}$, $\mathcal{P}_i^{(\text{first})}$, and $\mathcal{P}_i^{(\text{second})}$ denote the admissible state, first-derivative, and second-derivative predictor sets for target $x_i(t)$, respectively. In particular, $\mathcal{P}_i^{(\text{first})}$ excludes the target first derivative used to define $Y_i(t_k)$, preventing response leakage. Each slope is a log-odds effect: after accounting for the other predictors in the same model, a positive coefficient corresponds to a higher modeled probability of target increase and a negative coefficient to a lower probability. The logistic model is an interpretable approximation for directional-response inference, not the biological or physical equation governing the system; its coefficients are not kinetic parameters, rate constants, or recovered dynamical terms, but can be a basis.

For this reason, all-predictors results should be interpreted together with VIF, rank, aliasing, pairwise-correlation, class-balance, and MCMC diagnostics. A term that disappears in the all-predictors model is not necessarily false; it may be redundant with another representation or unstable under joint conditioning. Conversely, a term retained in the all-predictors model should be described as conditional support under the fitted predictor library, not as proof that no other unmeasured process is involved.

C. Prior and posterior computation

Slope coefficients are assigned weakly informative Gaussian priors,

$$\beta_{gi}^{(r)} \sim \text{Normal}(0, 2),$$

where 2 is the prior standard deviation. This prior centers effects at zero while allowing moderate log-odds effects when supported by the data. Weakly informative priors are commonly used to stabilize Bayesian regression under collinearity, limited sample size, and partial separation [6, 10, 22].

Posterior computation is performed using `brms` with the `cmdstanr` backend and a Bernoulli likelihood with logit link [2, 5]. The implementation uses Hamiltonian Monte Carlo with the No-U-Turn Sampler in Stan [8, 23]. Unless otherwise stated, manuscript analyses used four chains, 3000 iterations per chain, 1000 warmup iterations, `adapt_delta = 0.95`, and a fixed random seed.

The posterior distribution of each coefficient is the central object used by CIRN. Rather than relying on a single fitted point estimate, CIRN summarizes the posterior mean, interval estimates,

and whether the coefficient's 95% highest density interval excludes zero. The posterior mean gives the reported signed effect weight. The HDI determines edge inclusion. Equal-tailed intervals and odds ratios are useful summaries, but they do not replace the HDI edge rule.

D. CIRN edge definition

Definition VI.1 (Posterior-Supported CIRN Edge)**.** Let $\mathcal{D}_i^{(r)} = \Big\{ t_k,\, z_g^{(r)}(t_{k-\ell}),\, Y_i(t_k) : k \in \mathcal{K}_i^{(r)},\, g \in \mathcal{P}_i^{(r)} \Big\}$ denote the model-ready CIRN data for target $x_i(t)$ and representation r, where $\mathcal{K}_i^{(r)}$ is the set of usable time indices after response construction, lag alignment, filtering, and complete-row removal. For coefficient $\beta_{gi}^{(r)}$, CIRN retains the edge corresponding to predictor representation r of x_g, fitted through the standardized covariate $z_g^{(r)}(t_{k-\ell})$, when

$$0 \notin \mathrm{HDI}_{0.95}\Big(\beta_{gi}^{(r)}\Big).$$

The reported edge is directed toward the target directional response, commonly displayed as dx_i, because $Y_i(t_k)$ is defined from the sign of $x_i'(t_k)$. Thus the posterior mean coefficient, denoted by

$$\omega_{gi}^{(r)} = \mathbb{E}\Big[\beta_{gi}^{(r)} \mid \mathcal{D}_i^{(r)}\Big],$$

gives the sign and weight of the inferred edge.

If $\omega_{gi}^{(r)} > 0$, the edge is interpreted as activation in the target-response scale: larger values of the lagged predictor representation support a higher posterior probability that the target is increasing. If $\omega_{gi}^{(r)} < 0$, the edge is interpreted as inhibition in the target-response scale: larger values of the lagged predictor representation support a lower posterior probability that the target is increasing. The signed edge indicator is therefore

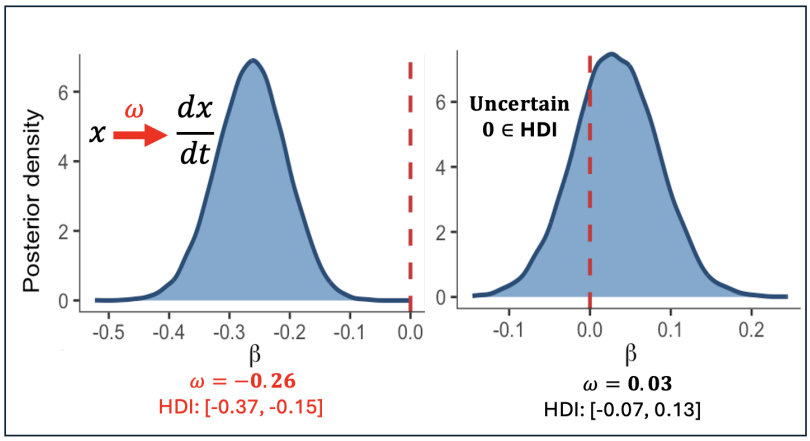
$$\mathrm{sgn}\Big(e_{gi}^{(r)}\Big) = \begin{cases} +1, & \text{if } \omega_{gi}^{(r)} > 0, \\ -1, & \text{if } \omega_{gi}^{(r)} < 0. \end{cases}$$

These signs are defined relative to $Y_i(t_k)$, not necessarily to the long-term abundance or concentration of the target. The absolute value $|\omega_{gi}^{(r)}|$ provides a graded effect magnitude on the log-odds scale, conditional on the fitted model, predictor set, representation, lag, and preprocessing settings. Odds ratios, computed as $\exp(\omega_{gi}^{(r)})$, are stored as auxiliary summaries but do not determine edge inclusion or sign. The implementation also assigns a relative-strength label within each target based on the magnitude of retained posterior mean effects. This label is used only for table reporting and visualization. It does not affect edge inclusion, edge direction, or regulatory sign.

Odds ratios are reported as $\exp\{\omega_{gi}^{(r)}\}$. An odds ratio greater than 1 corresponds to a positive posterior mean coefficient, and an odds ratio less than 1 corresponds to a negative posterior mean coefficient. Because the predictors are standardized, the odds ratio is interpreted per one-standard-deviation increase in the lagged predictor representation within the model-ready data. Odds ratios are useful for scale-aware explanation, but they are not used for edge inclusion.

Remark VI.1 (No p-value Edge Rule)**.** CIRN does not retain edges using frequentist p-values. Edge inclusion is determined by posterior HDI exclusion of zero. Predictive diagnostics and model-comparison summaries do not determine edge sign or edge inclusion.

Recommended Coefficient Language. Report ω as the posterior mean log-odds effect and report $\exp(\omega)$ as the corresponding odds ratio. A retained edge should be described as a CIRN-inferred regulatory hypothesis on the target-response scale. For example: "the lagged first derivative of X supported a higher posterior probability that Y was increasing." This is more precise than saying only that "X activates Y."



Supplementary Figure 3: Posterior edge-retention rule in CIRN. Example posterior coefficient distributions illustrating the CIRN HDI-based edge rule. A coefficient is retained as a CIRN edge when its 95% highest density interval (HDI) excludes zero; the posterior mean $\omega = \mathbb{E}[\beta \mid \text{data}]$ determines the reported signed effect weight. The left density shows a retained inhibitory coefficient because the entire HDI lies below zero. The right density shows a non-retained coefficient because its HDI contains zero, indicating insufficient posterior support for a signed effect.

E. Adjacency Matrix and Edge-List Representation

Definition VI.2. For a fixed predictor representation r, CIRN can represent retained effects as a signed and weighted adjacency matrix

$$\mathbf{W}^{(r)} = \left[\omega_{gi}^{(r)}\right],$$

where rows correspond to predictor terms or predictor variables under representation r, and columns correspond to target directional responses. Each entry is defined as

$$\omega_{gi}^{(r)} = \begin{cases} \mathbb{E}\left[\beta_{gi}^{(r)} \mid \text{data}\right], & \text{if } 0 \notin \text{HDI}_{0.95}\left(\beta_{gi}^{(r)}\right), \\ 0, & \text{otherwise.} \end{cases}$$

Nonzero entries correspond to posterior-supported CIRN edges. Positive entries indicate activation, negative entries indicate inhibition, and zero entries indicate that no edge was retained under the HDI criterion. Because each target is modeled separately and predictors are temporally lagged, $\mathbf{W}^{(r)}$ is generally asymmetric.

The primary implementation output is an edge list rather than a single collapsed adjacency matrix. Each retained edge records the predictor term, target response, posterior mean effect ω, odds ratio, regulatory sign, relative-strength label, 95% HDI bounds, equal-tailed credible interval bounds, response mode, response source, response-trigger metadata, class counts, jitter metadata when applicable, predictor representation, analysis mode, and fitting method. Pairwise-derived

edges are stored with a separate pairwise analysis label and declared pairwise predictor, whereas multivariable edges are stored separately from the optional pairwise output.

This edge-list representation preserves the provenance of each retained interaction. State-, first-derivative-, second-derivative-, all-predictors-, and optional pairwise-derived edges can therefore be inspected separately, compared across modes, or mapped back to state-level predictor–target relationships when performance evaluation against a known reference network requires a common state-level convention.

F. Overall Network Representation and Aggregation

After fitting all the Bayesian models, CIRN collects the retained predictor-to-target-response effects into a system-level network. This step is an edge-list aggregation, not model averaging: coefficients are not averaged across targets, temporal representations, or CIRN modes. Each retained edge preserves its source variable, predictor representation, target directional response, lag, posterior mean, 95% HDI, sign, response provenance, and fitting context.

For representation r and CIRN mode $m \in \{\texttt{pairwise}, \texttt{sublevel}, \texttt{all_predictors}\}$, define the retained edge set as

$$\mathcal{E}^{(r,m)} = \left\{ \left(z_g^{(r)}, Y_i, r, m \right) : 0 \notin \mathrm{HDI}_{0.95}\left(\beta_{gi}^{(r,m)} \right) \right\},$$

where $z_g^{(r)}$ is the lagged standardized predictor representation and Y_i is the binary directional response of target x_i. The corresponding representation-aware graph is

$$\mathcal{G}^{(m)} = \left(\mathcal{V}^{(m)}, \mathcal{E}^{(m)} \right),$$

where $\mathcal{V}^{(m)}$ is the representation-level node set for CIRN mode m, consisting of eligible source representations (state and derivative levels) and target-response variables.

Sublevel CIRN mode results are obtained by collecting the retained state-, first-derivative-, and second-derivative edges:

$$\mathcal{E}_{\text{sublevel}} = \bigcup_{r \in \{\text{state,first,second}\}} \mathcal{E}^{(r,\text{sublevel})}.$$

The all-predictors CIRN mode edge set is defined analogously from the joint all-predictors model:

$$\mathcal{E}_{\text{all}} = \bigcup_{r \in \{\text{state,first,second}\}} \mathcal{E}^{(r,\text{all})}.$$

The multivariable CIRN output is then

$$\mathcal{E}_{\text{multi}} = \mathcal{E}_{\text{sublevel}} \cup \mathcal{E}_{\text{all}}.$$

Representation and CIRN mode labels remain attached to every retained edge. Thus, the same base predictor–target pair may appear through state, rate-of-change, curvature, or joint all-predictors evidence; these are distinct temporal-response hypotheses, not duplicate coefficients to be averaged. Pairwise CIRN mode edges are stored separately and are combined with multivariable results only for visualization or cross-mode summaries.

Each retained CIRN edge carries the following information: predictor term, target response, direction, regulatory sign, posterior mean effect ω, posterior uncertainty, predictor representation,

analysis mode, response mode, class-count diagnostics, jitter metadata when applicable, and model-fitting method. Direction is determined by temporal lagging, regulatory sign is determined by the posterior mean coefficient, and posterior support is determined by the 95% HDI exclusion rule.

The final CIRN network should therefore be interpreted as a probabilistic, directed, signed, and representation-aware summary of target-wise classification models. It provides a system-level abstraction of regulatory structure while preserving the evidence trail for each retained interaction. This design allows users to distinguish edges that are stable across modes from edges that are representation-dependent, pairwise-only, conditionally retained, or sensitive to predictor competition.

VII. INFERENCE MODES AND REPRESENTATION-SPECIFIC EVIDENCE

A. Mode and predictor-set terminology

The CIRN results compendium use two different labels that should not be merged. The column `analysis_mode` describes the fitting context, whereas `predictor_set` describes the predictor representation. Keeping these labels separate is essential when summarizing results across sheets.

Column	Meaning
`analysis_mode`	Fitting mode: `multivariable` or `pairwise`.
`predictor_set`	Predictor representation: `original`, `first_derivative`, `second_derivative`, or `all_predictors`.

Thus, a row with `analysis_mode = multivariable` and `predictor_set = first_derivative` means that the edge came from a multivariable model using first-derivative predictors. A row with `analysis_mode = pairwise` and `predictor_set = all_predictors` means that the fit was one-predictor-at-a-time, but the candidate predictor was evaluated through the all-predictors representation set. These distinctions affect interpretation, especially when comparing `edges_combined`, `diagnostics`, `vif_group`, and `edge_consistency`.

B. Flexible CIRN as a multi-resolution framework

Different CIRN formulations address different scientific questions. These formulations are best interpreted as complementary layers of inference within a flexible and multi-resolution regulatory inference framework. In all cases, the fitted response is not the continuous derivative magnitude itself, but the binary directional response $Y_i(t_k)$ constructed from the sign of the target's raw estimated first derivative after near-stationary observations are removed. Thus, the shorthand examples below should be read as target-response models: dX denotes the directional response of X, while X, dX, and d^2X denote lagged predictor representations when they are admissible.

B.1 Individual or pairwise CIRN

The individual, or pairwise, CIRN formulation evaluates one predictor variable at a time for each target response. In the current implementation, this layer is obtained by setting `run_pairwise = TRUE`. Depending on `pairwise_representation_mode`, the one-predictor-at-a-time fit may evaluate state, first-derivative, second-derivative, or all eligible representations of that predictor.

Examples of pairwise CIRN questions include:

$$dX \sim X, \qquad dX \sim Y, \qquad dX \sim dY, \qquad dX \sim d^2X, \qquad dX \sim d^2Y.$$

This formulation is highly interpretable and useful for exploratory screening because it asks whether a single lagged predictor representation contains temporal-response information for the target. It is also useful for detecting marginal signals that may be weakened or redistributed in multivariable models. However, because predictors are evaluated independently, pairwise CIRN does not condition on other measured predictors. Its retained edges should therefore be interpreted as marginal posterior-supported temporal-response regulatory signals, not as isolated direct mechanisms.

B.2 Subgroup or representation-specific CIRN

The subgroup CIRN formulation models selected subsets of predictors simultaneously. In the current script, this can be implemented by declaring a restricted `predictors` set based on the scientific question, biological prior knowledge, pathway membership, dynamical similarity, derivative-based representation, or planned statistical screening. The representation-specific version of this idea is the `sublevel` mode, which fits state, first-derivative, and second-derivative predictor groups separately.

Examples include:

$$dX \sim X + Y, \qquad dX \sim dX + dY, \qquad dX \sim d^2X + d^2Y.$$

Subgroup or sublevel CIRN serves as a compromise between one-predictor-at-a-time screening and full joint fitting. It reduces dimensionality while partially accounting for other predictors within the same selected group. This formulation is particularly useful when the analyst wants to preserve interpretability, compare state-, velocity-, and curvature-supported evidence, or focus on a biologically meaningful module. In the current manuscript, the representation-specific multivariable outputs correspond to this layer of CIRN inference.

B.3 Full or all-predictors CIRN

The full CIRN formulation jointly models all eligible state, first-derivative, and second-derivative predictors for a target response. In the current implementation, this is the `all_predictors` mode:

$$dX \sim X + Y + dX + dY + d^2X + d^2Y,$$

after response-defining quantities have been excluded and after lagging, filtering, complete-row retention, and standardization. The signs of the fitted effects are not assigned in the formula; they are estimated from the posterior coefficient means.

This approach estimates conditional temporal-response support while allowing multiple representations to compete in the same fitted model. Compared with pairwise fitting, the all-predictors model can better distinguish predictors that retain support after conditioning on other measured state, velocity, and curvature terms. However, full models can be more sensitive to multicollinearity, limited model-ready sample size, derivative noise, overfitting, high computational cost, and convergence instability. For this reason, all-predictors results should be interpreted together with class-balance, MCMC, VIF, rank, aliasing, and pairwise-correlation diagnostics.

B.4 Interpretation across resolutions

The three formulations may be interpreted as distinct but complementary levels of CIRN inference:

$$\text{pairwise CIRN} \Rightarrow \text{marginal temporal-response evidence},$$

$$\text{sublevel CIRN} \Rightarrow \text{module- or representation-specific conditional evidence},$$

$$\text{all-predictors CIRN} \Rightarrow \text{joint conditional temporal-response evidence}.$$

Here, *marginal temporal-response evidence* means that a single lagged predictor representation is informative for the target directional response when evaluated on its own. *Module- or representation-specific conditional evidence* means that a predictor remains supported after being fitted together with other predictors in a selected subgroup, such as a biological module or a state-, first-derivative-,

or second-derivative representation set. *Joint conditional temporal-response evidence* means that a predictor remains supported in the all-predictors model, where eligible state, velocity, and curvature terms compete simultaneously for the same target response.

Rather than selecting only one formulation, a multi-resolution workflow may be used: pairwise CIRN for exploratory edge screening, subgroup or sublevel CIRN for representation-specific refinement, and all-predictors CIRN for joint conditional assessment. Edges that are retained with the same sign across multiple CIRN resolutions may indicate stronger and more stable directed regulatory hypotheses. In contrast, disagreements across resolutions should not be treated automatically as errors; they may indicate indirect effects, shared predictor information, collinearity, hidden variables, nonlinear interactions, lag sensitivity, or representation-dependent temporal structure. The current CIRN implementation supports these layers through `run_pairwise`, user-declared target and predictor sets in `cirn_config()`, `representation_mode`, `pairwise_representation_mode`, and the cross-mode `edge_consistency` output.

C. Generic interpretation of predictor representations

The predictor representation determines what kind of temporal information supports a retained edge. Because all predictors are lagged and standardized before fitting, the interpretation should always be stated relative to the target directional response at a later time point. A positive coefficient means that larger values of the lagged predictor representation increase the posterior probability that the target is increasing; a negative coefficient means that larger values of the lagged predictor representation decrease that probability.

Supplementary Table 1: Generic interpretation of CIRN predictor representations.

Predictor representation	**Temporal meaning**	**Generic interpretation of a retained edge**	**Reporting caution**
State predictor, $x_g(t_{k-\ell})$	Lagged level, abundance, concentration, or system state	The prior level of x_g contains information about whether target x_i subsequently increases or decreases. If $\omega > 0$, higher x_g supports greater probability of target increase; if $\omega < 0$, higher x_g supports lower probability of target increase.	Interpret as level-dependent temporal-response support, not proof that the state variable directly causes the target.
First-derivative predictor, $x'_g(t_{k-\ell})$	Lagged velocity, trend, or rate of change	The recent direction or speed of change of x_g contains information about the subsequent direction of the target. If $\omega > 0$, more positive velocity of x_g supports target increase; if $\omega < 0$, more positive velocity of x_g suppresses target increase.	Useful in oscillatory, delayed, or transition-like systems; derivative estimates are more noise-sensitive than state values.

Predictor representation	**Temporal meaning**	**Generic interpretation of a retained edge**	**Reporting caution**
Second-derivative predictor, $x_g''(t_{k-\ell})$	Lagged curvature, acceleration, deceleration, or change in trend	The curvature of x_g contains information about the upcoming target response. If $\omega > 0$, positive curvature or acceleration of x_g supports target increase; if $\omega < 0$, positive curvature or acceleration of x_g suppresses target increase.	Often indicates phase, turning-point, or curvature-mediated information; should be interpreted with smoothing and noise diagnostics.
All-predictors retained term	Conditional evidence among state, velocity, and curvature terms	The retained representation remains informative after competing with other eligible representations in the same target-wise model.	Stronger conditional evidence, but more sensitive to collinearity, sample size, and derivative noise.

How to Report. When reporting a retained derivative-based edge, describe the representation explicitly. For example, write "the lagged first derivative of X supported a higher posterior probability that Y increases" rather than only "X activates Y." This wording keeps the interpretation faithful to the CIRN target-response scale.

Supplementary Table 2 illustrates how the generic representation-level interpretations can be applied to the classical Lotka–Volterra predator–prey system. The examples are taken from the retained edges in the classical Lotka–Volterra `edges_combined` output. Here, X denotes the prey state and Y denotes the predator state; dX and dY denote the corresponding target directional responses. Values of ω are posterior mean coefficients, and HDIs are 95% highest density intervals.

Supplementary Table 2: Classical Lotka–Volterra example of CIRN predictor-representation interpretation.

Predictor representation	**Temporal meaning**	**Generic interpretation of a retained edge**	**Biological interpretation note**
State predictor, $Y(t_{k-\ell}) \to dX$	Lagged predator level	Higher lagged predator abundance supported a lower posterior probability that prey X was increasing. This edge was retained with $\omega = -5.85$ and HDI $[-8.27, -3.49]$ in the all-predictors multivariable model.	This recovers the expected predator-pressure signal in the Lotka–Volterra system: when predator abundance is high, prey growth is more likely to be suppressed in the subsequent directional response.

Predictor representation	**Temporal meaning**	**Generic interpretation of a retained edge**	**Biological interpretation note**
State predictor, $X(t_{k-\ell}) \to dY$	Lagged prey level	Higher lagged prey abundance supported a higher posterior probability that predator Y was increasing. This edge was retained with $\omega = 5.84$ and HDI $[3.60, 8.28]$ in the all-predictors multivariable model.	This matches the biological resource-consumer interpretation of the model: prey availability supports predator increase, and CIRN captures this as a strong positive temporal-response signal.
First-derivative predictor, $X'(t_{k-\ell}) \to dY$	Lagged prey velocity or recent prey growth trend	A more positive recent change in prey X supported a higher posterior probability that predator Y was increasing. This edge was retained with $\omega = 0.49$ and HDI $[0.03, 1.00]$ in the first-derivative multivariable model.	This indicates that predator increase is informed not only by prey abundance, but also by whether the prey population has recently been rising, consistent with the delayed response structure of predator–prey cycles.
Second-derivative predictor, $X''(t_{k-\ell}) \to dY$	Lagged prey curvature or change in prey growth trend	Positive curvature in the prey trajectory supported a lower posterior probability that predator Y was increasing. This edge was retained with $\omega = -2.29$ and HDI $[-3.53, -1.07]$ in the second-derivative multivariable model.	This curvature-mediated signal is biologically meaningful in an oscillatory system: changes in prey acceleration can indicate approach to a turning point before the predator response fully unfolds.
Second-derivative predictor, $Y''(t_{k-\ell}) \to dX$	Lagged predator curvature or change in predator growth trend	Positive curvature in the predator trajectory supported a higher posterior probability that prey X was increasing. This edge was retained with $\omega = 3.30$ and HDI $[1.69, 4.93]$ in the second-derivative multivariable model.	This suggests that CIRN can detect phase-structured information in the predator trajectory, where curvature in predator dynamics helps classify the upcoming prey directional response.

Predictor representation	Temporal meaning	Generic interpretation of a retained edge	Biological interpretation note
All-predictors retained term	Conditional support after state, velocity, and curvature terms compete	The biologically expected state-level edges $Y \rightarrow dX$ and $X \rightarrow dY$ remained retained in the all-predictors model, with consistent inhibitory and activating signs, respectively.	This strengthens the CIRN interpretation: the core predator–prey regulatory signals persist even when state, velocity, and curvature representations are evaluated jointly.

D. Sublevel multivariable CIRN

In sublevel mode, CIRN fits separate multivariable models for original-state, first-derivative, and second-derivative predictor sets. Within a sublevel, all eligible predictors are modeled jointly. This allows the analyst to determine whether support is state-mediated, velocity-mediated, or curvature-mediated.

A sublevel edge is conditional on the other predictors in the same representation. For example, an original-state edge $X \rightarrow dY$ retained in multivariable state mode means that the lagged state of X remains informative after adjustment for the other eligible lagged state variables in that target-wise model. It does not imply that first- or second-derivative representations were considered in that same fit. Those representations are evaluated in their own sublevel models or in the all-predictors model.

E. All-predictors CIRN

The all-predictors model fits all eligible representations jointly. It is a more stringent conditional analysis because the representations compete in the same model. Edges retained in this mode can provide strong evidence of conditional support, but the model may be affected by collinearity, derivative noise, or limited minority-class observations per predictor.

All-predictors results are useful when the scientific question is not only whether a regulator matters, but which representation of that regulator carries the strongest conditional target-response information. A retained state term suggests level-dependent support. A retained first-derivative term suggests velocity-dependent support. A retained second-derivative term suggests curvature- or turning-point-related support. If multiple representations of the same source variable are retained, the result should be reported as representation-specific evidence rather than collapsed into a single unqualified source-target statement.

F. Pairwise CIRN

Pairwise mode fits one predictor at a time against each target. It is useful for identifying marginal temporal-response information and for comparing whether an edge persists after adjustment in multivariable modes. Pairwise-only edges should be interpreted as screening or diagnostic evidence rather than as independent conditional evidence.

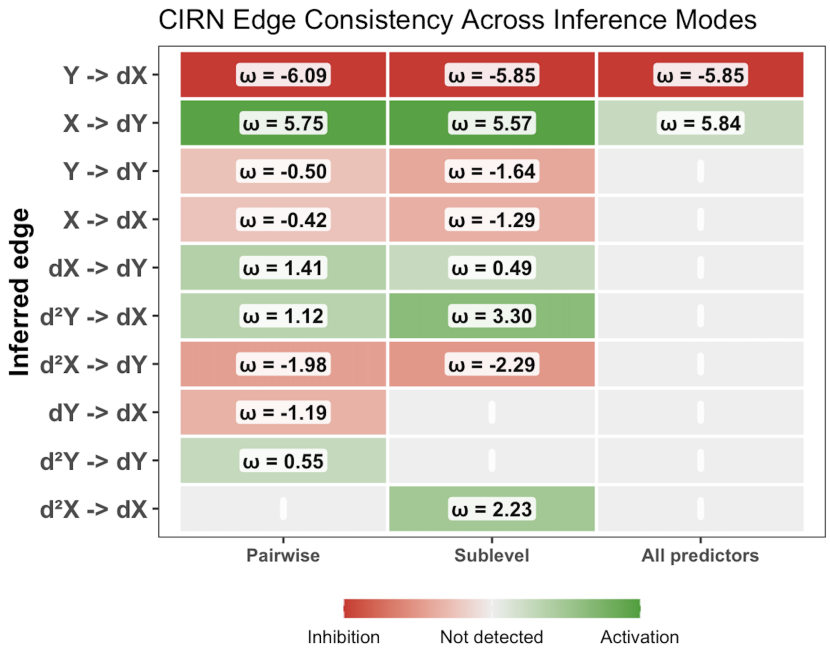
Pairwise fits can be especially helpful when multivariable models are affected by small sample size or collinearity. If a pairwise edge is retained but the corresponding multivariable edge is not,

the predictor may carry useful marginal information that is shared with other predictors. If both pairwise and multivariable evidence point in the same direction, the result is easier to explain. If signs conflict across modes, the edge should be reported cautiously and checked against collinearity, class balance, and representation-specific diagnostics.

G. Cross-mode consistency

The `edge_consistency` sheet summarizes whether an edge appears in pairwise, sublevel, and all-predictors outputs. Agreement across modes strengthens confidence. Disagreement across modes can be informative because it may reveal representation dependence, predictor competition, collinearity, phase shifts, or small-sample sensitivity.

Cross-mode consistency should be used as interpretive context, not as a second edge-selection rule. The primary edge rule remains the 95% HDI exclusion criterion within each fitted model. The consistency summary helps readers distinguish stable results from mode-sensitive results. A mode-sensitive edge may still be scientifically interesting, especially in nonlinear or oscillatory systems, but it should not be presented with the same strength as an edge that is directionally stable across relevant model contexts.



Supplementary Figure 4: Cross-mode edge-consistency visualization for the classical Lotka–Volterra system. Tile plot summarizing retained CIRN edges across pairwise, representation-specific multivariable sublevel, and joint all-predictors inference modes. Rows correspond to inferred predictor–target response pairs, and columns indicate the inference mode in which the edge was retained. Cell labels report the posterior mean edge weight, $\omega = \mathbb{E}[\beta \mid \text{data}]$. Red cells indicate inhibitory target-response effects ($\omega < 0$), green cells indicate activating target-response effects ($\omega > 0$), and gray cells indicate edges not retained under the 95% HDI rule. The sublevel column summarizes representation-specific multivariable support across state, first-derivative, and second-derivative predictor levels. The expected ecological state-level effects, $Y \to dX$ and $X \to dY$, are retained consistently across all three inference modes, whereas several derivative- and curvature-based edges appear only in pairwise or sublevel models. This pattern illustrates why pairwise, sublevel, and all-predictors outputs should be interpreted as complementary layers of CIRN evidence rather than interchangeable summaries.

Supplementary Figure 4 illustrates the role of pairwise CIRN using the classical Lotka–Volterra system. Pairwise fitting evaluates one candidate predictor representation at a time against a target directional response, and is therefore useful as a sensitive screening layer for marginal temporal-

response signals. In the Lotka–Volterra example, the two expected ecological state-level effects are already visible in pairwise mode: predator abundance Y inhibits subsequent prey increase dX ($\omega = -6.09$), whereas prey abundance X activates subsequent predator increase dY ($\omega = 5.75$). These effects remain retained in the sublevel and all-predictors modes with the same signs, providing strong cross-mode support for the canonical predator–prey regulatory structure.

The remaining pairwise-retained edges should be read as additional temporal information rather than as automatically equivalent biological interactions. Several derivative- and curvature-based terms, such as $dX \to dY$, $d^2Y \to dX$, and $d^2X \to dY$, are retained in pairwise and/or sublevel modes but not in the all-predictors model. This indicates that these terms carry marginal or representation-specific information about the oscillatory phase structure of the system, but their support is reduced once state, velocity, and curvature representations compete jointly. Thus, pairwise CIRN is valuable for revealing candidate temporal signals that may be biologically meaningful, especially in cyclic systems, but cross-mode comparison is needed to distinguish stable regulatory signatures from mode-sensitive or representation-dependent effects.

VIII. ADAPTIVE JITTER AND MONOTONIC RESPONSE CASES

A. Why adaptive jitter is used

A target trajectory can be monotonic, nearly monotonic, or dominated by one derivative sign. In such cases, the raw directional response may contain only one class, or may contain too few observations in the minority derivative class to support a stable Bayesian logistic regression. Adaptive minimal jitter is an optional response-construction safeguard for these cases.

The issue is not merely computational. Logistic regression requires variation in the response. If every model-ready observation is labeled as increasing, or every observation is labeled as decreasing, the model cannot estimate how predictors separate increasing from decreasing target states. A severely sparse minority class can also produce unstable posterior estimates, wide intervals, or separation-like behavior. Adaptive jitter is therefore invoked only when the raw derivative response is one-class or nearly one-class according to the configured minimum class-count threshold.

Importantly, setting `adaptive_jitter = TRUE` (recommended) does not force jittering for every target. CIRN first constructs the raw directional response from the unjittered target derivative and checks whether both derivative classes are sufficiently represented. If the raw response already contains at least the configured minimum number of observations in each class, the raw response is retained, `jitter_used = FALSE`, and no adaptive jitter is applied. Adaptive jitter is attempted only when the raw response is one-class or when the minority derivative class is smaller than `jitter_min_class_count`.

This decision is target-specific rather than dataset-wide or predictor-wide. Thus, within the same multivariable dataset, CIRN may use the raw directional response for targets whose derivatives already provide adequate increasing and decreasing classes, while using the adaptive response-jitter pathway for targets whose raw derivative response is one-class or too sparse. A monotonic predictor is not automatically jittered in the primary CIRN workflow; adaptive jitter concerns the target trajectory only when that trajectory is being used to define $Y_i(t_k)$.

B. How adaptive jitter is computed

Adaptive jitter is applied to the target trajectory used for response construction, not directly to the binary class labels. Let $x_i(t_k)$ denote the target trajectory available in the derivative-construction

table before adaptive response jitter is applied. For a candidate jitter attempt a, CIRN constructs

$$x_i^{*(a)}(t_k) = x_i(t_k) + \eta_{k,a}, \qquad \eta_{k,a} \sim \text{Normal}(0, \sigma_{J,a}^2),$$

where

$$\sigma_{J,a} = c_a B_i.$$

Here, c_a is a candidate jitter scale factor selected from the configured `jitter_scale_grid`, and B_i is the selected jitter basis. The term "grid" refers to an ordered list of candidate scale factors, not to the time grid used for derivative estimation. In the current implementation, the default search grid is

$$\mathcal{C} = \{10^{-8}, 3{\times}10^{-8}, 10^{-7}, 3\times10^{-7}, 10^{-6}, 3\times10^{-6}, 10^{-5}, 3\times10^{-5}, 10^{-4}, 3\times10^{-4}, 10^{-3}, 3\times10^{-3}, 10^{-2}\}$$

and c_a must be selected from this. CIRN sorts this list, removes duplicate values if present, and searches it from the smallest value to the largest. For each $c_a \in \mathcal{C}$, the actual jitter standard deviation is

$$\sigma_{J,a} = c_a B_i.$$

Thus, if `jitter_scale_basis = "state_sd"`, then $B_i = \text{sd}\{x_i(t_k)\}$, and a scale factor of 10^{-6} means that the jitter standard deviation is 10^{-6} times the target-state standard deviation. CIRN attempts the smallest perturbation first and stops at the first scale factor that produces a usable two-class response with at least `jitter_min_class_count` observations in each class. This is why the procedure is called adaptive minimal jitter.

The default basis is `state_sd`, which expresses the jitter scale as a fraction of the target-state standard deviation and is the conservative default. The available bases are:

`state_sd`, `state_range`, `derivative_sd`, `derivative_max_abs`, `absolute`.

Their meanings are:

- `state_sd`: $B_i = \text{sd}\{x_i(t_k)\}$. The scale factor is interpreted as a fraction of the target-state standard deviation.
- `state_range`: $B_i = \max_k x_i(t_k) - \min_k x_i(t_k)$. The scale factor is interpreted as a fraction of the target-state range.
- `derivative_sd`: $B_i = \text{sd}\{x_i'(t_k)\}$. The scale factor is interpreted as a fraction of the target-derivative standard deviation.
- `derivative_max_abs`: $B_i = \max_k |x_i'(t_k)|$. The scale factor is interpreted as a fraction of the largest absolute target derivative.
- `absolute`: $B_i = 1$. The grid values are used directly as absolute jitter standard deviations.

If the selected basis is non-finite or zero, the implementation falls back to a finite target-state magnitude when available, and otherwise uses $B_i = 1$. Thus, adaptive jitter is scale-aware by default rather than an arbitrary unscaled perturbation.

For each attempted scale, CIRN recomputes the target first derivative from the jittered trajectory. For interior time points,

$$x_i^{*\prime}(t_k) \approx \frac{x_i^*(t_{k+1}) - x_i^*(t_{k-1})}{t_{k+1} - t_{k-1}}.$$

The binary directional response is then reconstructed using the same ε-threshold rule as the raw response:

$$Y_i^*(t_k) = \begin{cases} 1, & \text{if } x_i^{*'}(t_k) > \varepsilon, \\ 0, & \text{if } x_i^{*'}(t_k) < -\varepsilon, \\ \text{removed}, & \text{if } |x_i^{*'}(t_k)| \leq \varepsilon. \end{cases}$$

The first jitter scale that produces at least the configured minimum number of observations in both derivative classes is retained. In the current implementation, the default `jitter_min_class_count` is 5. If the raw response already has two classes but is sparse, and jitter fails to improve the class count, the raw response may be retained with a diagnostic trigger. If the raw response is one-class and jitter fails, the corresponding target fit is skipped because a logistic model is not supported.

C. Response jitter versus predictor jitter

In the primary CIRN workflow, adaptive jitter is a response-construction pathway. It is used to recover a usable increasing-versus-decreasing target response when the raw target derivative is one-class or nearly one-class. CIRN does not randomly flip class labels, and it does not jitter predictors in the primary analysis.

The implementation also includes an optional predictor-jitter sensitivity mode. This mode is activated only when adaptive response jitter has been triggered and `jitter_predictors = TRUE`. Under this optional setting, declared predictor trajectories can also be jittered and their derivatives recomputed as a sensitivity analysis. This should not be used for the primary reported network unless explicitly justified and labeled as a predictor-jitter sensitivity run.

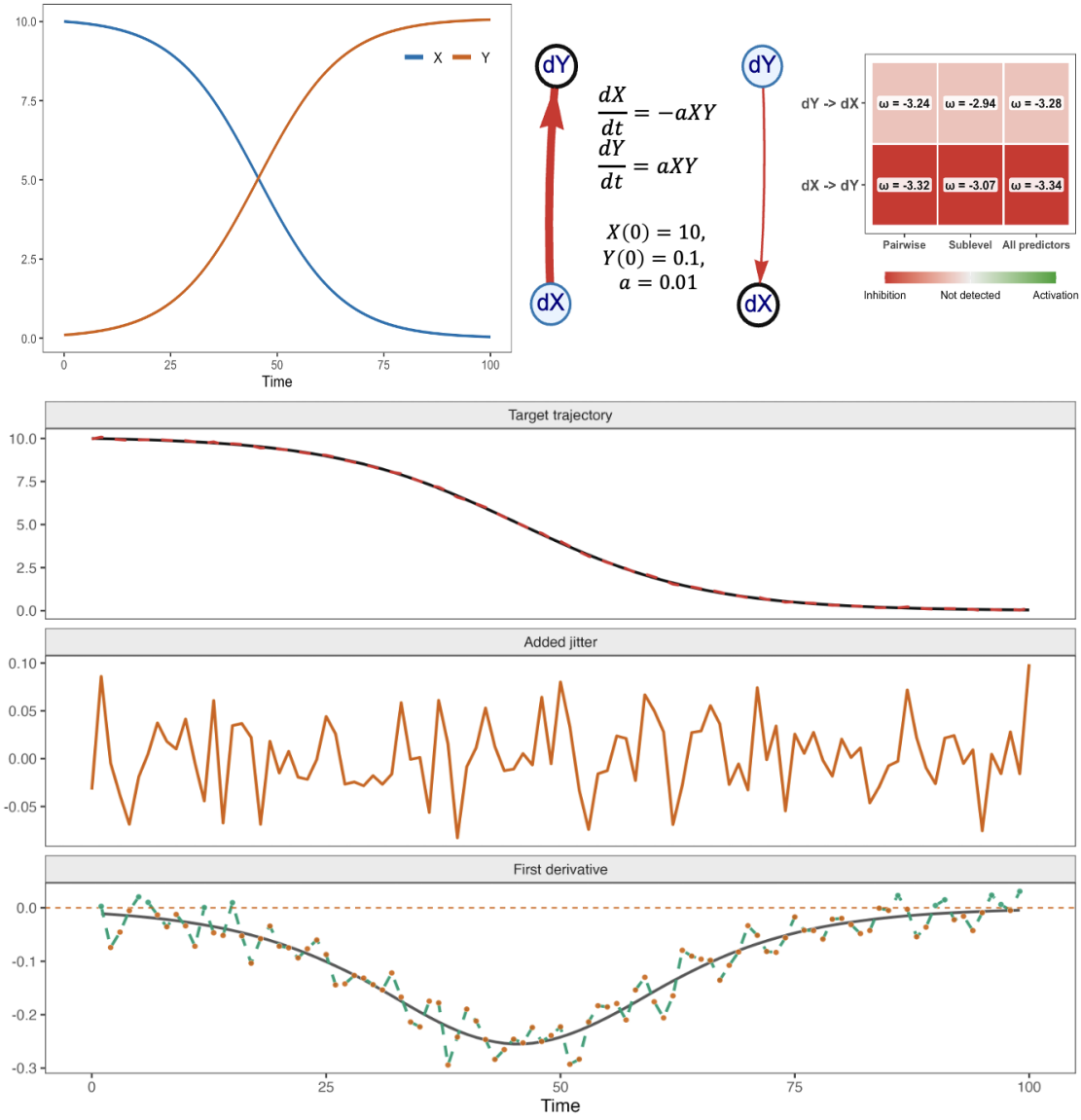
Reporting Jitter. Any model that uses adaptive jitter should be flagged. The appropriate interpretation is that the edge was supported under a minimally perturbed directional-response construction. The `jitter_used`, `response_mode`, `response_label`, `response_source`, `response_trigger`, `jitter_scale_factor`, `jitter_scale`, `jitter_basis`, `jitter_basis_value`, `jitter_attempts`, `class_0_count`, `class_1_count`, and `min_class_count` columns should be checked before reporting the edge.

D. Recommended visualization

When adaptive jitter is used, the supplement should include a target-specific diagnostic visualization rather than only a text description. The purpose of the visualization is to show what was perturbed, how small the perturbation was relative to the target trajectory, and how the usable binary response was obtained. A clear diagnostic should therefore show the original target behavior, the minimally jittered trajectory used for response construction, the added perturbation, and the resulting derivative classes. This is especially useful for monotonic or nearly monotonic targets, where the raw derivative may produce only one directional class. The figure should make clear that adaptive jitter does not randomly flip class labels; instead, CIRN minimally perturbs the target trajectory, recomputes the derivative, and then applies the same ε-threshold rule used for ordinary directional-response construction.

Supplementary Figure 5 illustrates the recommended diagnostic layout using a simple two-variable monotonic-transition system. The upper portion provides system-level context by showing the raw trajectories, governing equations, CIRN-inferred network, and cross-mode edge-consistency summary. The lower portion documents the target-specific adaptive-jitter response construction for X. In this example, the raw derivative of X produced only the decreasing class, so adaptive

minimal jitter was invoked to recover a usable two-class directional response while preserving the large-scale shape of the target trajectory.



Supplementary Figure 5: Adaptive response-jitter visualization for a simple monotonic-transition system. The figure illustrates how adaptive minimal jitter is documented when a target's raw derivative response is one-class or nearly one-class. The upper portion summarizes the example system: raw state trajectories show X decreasing and Y increasing monotonically; the governing equations, $dX/dt = -aXY$ and $dY/dt = aXY$, define opposite directional responses; the CIRN-inferred derivative-response network shows posterior-supported inhibitory edges; and the cross-mode consistency heatmap shows that the inferred effects are retained across pairwise, sublevel, and all-predictors modes. The lower portion shows the adaptive minimal-jitter diagnostic for target X. In the target-trajectory panel, the solid black curve is the processed target trajectory and the dashed red curve is the minimally jittered target trajectory used only for response construction. In the added-jitter panel, the orange curve shows the zero-mean perturbation added to the target trajectory. In the first-derivative panel, the solid gray curve is the raw derivative and the dashed green curve is the derivative recomputed from the minimally jittered target trajectory; the orange dashed horizontal threshold line marks the near-zero derivative cutoff used for response encoding; orange points indicate observations classified as decreasing, and green points indicate observations classified as increasing after the jittered derivative was thresholded. For target X, the raw derivative response contained only decreasing observations ($0 = 99, 1 = 0$), whereas the minimally jittered derivative produced a usable two-class response ($0 = 88, 1 = 11$). CIRN used a jitter scale factor of 0.01 with `state_sd` as the basis, giving a jitter standard deviation of 0.03951. This diagnostic shows that adaptive jitter minimally perturbs the target trajectory and then recomputes the derivative; it does not randomly flip class labels and does not imply predictor jitter in the primary CIRN model.

E. Recommended table notation

For compact manuscript and supplementary tables, the jitter field can be reported using a short code that identifies how the target directional response was constructed for the fitted model. This notation distinguishes ordinary raw-response CIRN fits from adaptive response-jitter fits and from optional predictor-jitter sensitivity analyses.

Supplementary Table 3: Recommended notation for reporting adaptive jitter status in CIRN edge tables.

Jitter code	Meaning	Recommended interpretation
No (raw)	The raw target derivative produced a usable two-class directional response; no adaptive jitter was applied.	Standard CIRN directional-response fit.
Yes (response)	Adaptive jitter was applied only to the target trajectory used to construct the binary response.	Jitter-qualified CIRN edge; report that the edge was supported under adaptive response construction.
Yes (response + predictor)	Adaptive response jitter was used, and optional predictor-jitter sensitivity mode was also enabled.	Sensitivity-qualified result; report separately from the primary CIRN network unless explicitly justified.

The jitter code is assigned at the fitted-model level and then carried into the retained-edge table. Therefore, a single multivariable dataset may contain both `No (raw)` and `Yes (response)` entries when some target variables have adequate raw increasing/decreasing classes and others require adaptive response jitter. A monotonic predictor alone does not determine the jitter code; adaptive jitter is triggered by the target response used to define $Y_i(t_k)$.

Supplementary Table 4 gives an edge-list example using the deterministic Lotka–Volterra predator–prey system. In this system, all retained rows are labeled `No (raw)` because the raw directional responses for both targets contained usable increasing and decreasing classes without adaptive jitter. Thus, the table illustrates how the jitter column is reported in a standard raw-response case; in systems with mixed monotonic and non-monotonic target trajectories, the same column may contain both `No (raw)` and `Yes (response)` entries.

When writing the manuscript or supplement, a jitter-supported edge should not be hidden. Recommended language is: "This edge was retained under the adaptive response-jitter pathway and should be interpreted as a jitter-qualified CIRN-inferred regulatory signal." If the edge table is space-limited, the jitter column can be coded compactly, but the caption or note should define the coding.

IX. DIAGNOSTICS AND DATA SUFFICIENCY

A. Purpose and indexing of diagnostic outputs

The diagnostic outputs in CIRN are intended to qualify how fitted models and retained edges should be interpreted. They do not modify the edge-retention rule. A coefficient is retained as a CIRN-inferred edge only when its 95% highest density interval excludes zero. The diagnostic sheets are then used to determine whether the retained edge should be reported as a strong regulatory hypothesis, a cautious or mode-sensitive signal, a jitter-qualified result, or a result requiring inspection or refitting.

Supplementary Table 4: Example CIRN edge-list notation for the deterministic Lotka–Volterra predator–prey system. The table illustrates how retained rows from `edges_combined` can be reported with cross-mode support and compact jitter notation.

Mode	Predictor set	CIRN edge	ω	OR	95% HDI	Strength	Support	Class (0/1)	Jitter
multivariable	original	$X \longrightarrow\!\!\!\!\mid\ dX$	-1.29	0.28	$[-2.14, -0.46]$	weak	P,S	41/57	No (raw)
pairwise	original	$X \longrightarrow\!\!\!\!\mid\ dX$	-0.42	0.66	$[-0.85, -0.01]$	credible	P,S	41/57	No (raw)
multivariable	original	$X \longrightarrow dY$	5.57	262.66	$[3.53, 7.72]$	strong	P,S,A	66/32	No (raw)
multivariable	all_predictors	$X \longrightarrow dY$	5.84	343.00	$[3.60, 8.28]$	strong	P,S,A	66/32	No (raw)
pairwise	original	$X \longrightarrow dY$	4.29	72.82	$[2.78, 6.02]$	strong	P,S,A	66/32	No (raw)
pairwise	all_predictors	$X \longrightarrow dY$	5.75	314.02	$[3.67, 7.86]$	strong	P,S,A	66/32	No (raw)
multivariable	original	$Y \longrightarrow\!\!\!\!\mid\ dX$	-5.85	2.87×10^{-3}	$[-8.11, -3.68]$	strong	P,S,A	41/57	No (raw)
multivariable	all_predictors	$Y \longrightarrow\!\!\!\!\mid\ dX$	-5.85	2.89×10^{-3}	$[-8.27, -3.49]$	strong	P,S,A	41/57	No (raw)
pairwise	original	$Y \longrightarrow\!\!\!\!\mid\ dX$	-4.80	8.20×10^{-3}	$[-6.60, -3.08]$	strong	P,S,A	41/57	No (raw)
pairwise	all_predictors	$Y \longrightarrow\!\!\!\!\mid\ dX$	-6.09	2.26×10^{-3}	$[-8.34, -4.02]$	strong	P,S,A	41/57	No (raw)
multivariable	original	$Y \longrightarrow\!\!\!\!\mid\ dY$	-1.64	0.19	$[-2.69, -0.62]$	weak	P,S	66/32	No (raw)
pairwise	original	$Y \longrightarrow\!\!\!\!\mid\ dY$	-0.50	0.61	$[-0.99, -0.05]$	weak	P,S	66/32	No (raw)
multivariable	first_derivative	$dX \longrightarrow dY$	0.49	1.63	$[0.03, 1.00]$	weak	P,S	66/32	No (raw)
pairwise	first_derivative	$dX \longrightarrow dY$	0.49	1.63	$[0.03, 1.00]$	weak	P,S	66/32	No (raw)
pairwise	all_predictors	$dX \longrightarrow dY$	1.41	4.11	$[0.39, 2.44]$	weak	P,S	66/32	No (raw)
pairwise	all_predictors	$dY \longrightarrow\!\!\!\!\mid\ dX$	-1.19	0.30	$[-2.59, -0.05]$	moderate	P	41/57	No (raw)
multivariable	second_derivative	$d^2X \longrightarrow dX$	2.23	9.31	$[1.11, 3.49]$	weak	S	41/57	No (raw)
multivariable	second_derivative	$d^2X \longrightarrow\!\!\!\!\mid\ dY$	-2.29	0.10	$[-3.53, -1.07]$	moderate	P,S	66/32	No (raw)
pairwise	second_derivative	$d^2X \longrightarrow\!\!\!\!\mid\ dY$	-1.98	0.14	$[-3.10, -0.99]$	moderate	P,S	66/32	No (raw)
multivariable	second_derivative	$d^2Y \longrightarrow dX$	3.30	27.20	$[1.69, 4.93]$	moderate	P,S	41/57	No (raw)
pairwise	second_derivative	$d^2Y \longrightarrow dX$	1.12	3.05	$[0.51, 1.74]$	weak	P,S	41/57	No (raw)
pairwise	second_derivative	$d^2Y \longrightarrow dY$	0.55	1.73	$[0.02, 1.14]$	strong	P	66/32	No (raw)

Notes: Mode is the raw `analysis_mode` value from `edges_combined`; Predictor set is the raw `predictor_set` value. Rows are ordered by predictor term: X, Y, dX, dY, d^2X, and d^2Y. Support gives cross-mode evidence for the same term–target pair (P=pairwise, S=sublevel or representation-specific multivariable, A=all predictors). ω is the posterior mean coefficient, OR is the odds ratio, and the 95% HDI is the highest-density interval used for edge retention. Strength is the relative effect category reported in `edges_combined`. Class (0/1) reports the number of decreasing/increasing target-direction labels used in. the fitted edge model. Jitter reports whether adaptive response jitter or optional predictor-jitter sensitivity was used for the fitted edge model. `No (raw)` means the raw directional response was used without jitter; `Yes (response)` means adaptive jitter was applied to the target trajectory for response construction; `Yes (response + predictor)` means optional predictor-jitter sensitivity was also enabled. In this Lotka–Volterra example, all retained rows are `No (raw)`

The diagnostic sheets are indexed at different levels. The `edges_combined` sheet is coefficient- or edge-level. The `diagnostics` and `effective_sample_size` sheets are fitted-model-level. The `vif_group` sheet is predictor-term-level within a fitted model, whereas `vif_pairs` is predictor-pair-level. The `edge_consistency` sheet is term–target-level after summarizing support across pairwise, sublevel, and all-predictors modes. For this reason, diagnostic rows should be matched to retained edges by target, predictor set, analysis mode, response source, and pairwise predictor when applicable, not by row number.

B. MCMC diagnostics

The `diagnostics` sheet records fitted-model status, response source, class counts, number of predictors, jitter metadata, maximum $\hat{R}$, minimum effective sample-size ratio, and optional LOO summaries. These are model-level summaries rather than edge-level summaries. A single fitted model can produce no retained edges, one retained edge, or several retained edges.

Supplementary Table 5: Interpretation of CIRN MCMC and predictive diagnostics.

Diagnostic	What the script reports	Recommended interpretation
Model status	`status`	`completed` indicates that the Bayesian logistic model was fitted. Skipped or failed statuses indicate that the model was not interpretable under the requested configuration and should not be treated as biological evidence for absence of regulation.
Convergence	`max_rhat`	The largest $\hat{R}$ among monitored parameters for the fitted model. Values ≤ 1.01 indicate that chains mixed well enough for interpretation. Values between 1.01 and 1.05 warrant inspection of trace plots and posterior summaries. Values > 1.05 suggest poor convergence and should generally prompt refitting or cautious interpretation [26].
Sampling efficiency	`min_neff`	The minimum effective sample-size ratio from `brms::neff_ratio`, not the raw number of posterior draws. Values near 1 indicate efficient sampling. Values below approximately 0.1 suggest poor mixing or strong autocorrelation. Low values reduce confidence in posterior means, odds ratios, and HDI bounds.
Predictive fit	`loo_elpd`	Optional leave-one-out expected log predictive density, available only when LOO is computed. Higher values indicate better out-of-sample predictive fit among compared model specifications. LOO is not used for edge inclusion, edge sign, or network construction.
Response metadata	`response_mode`, `response_source`, `response_trigger`	Identifies whether the response was constructed from the raw first derivative or through adaptive response jitter. This is essential for deciding whether an edge should be reported as a standard or jitter-qualified CIRN result.
Class information	`class_0_count`, `class_1_count`, `min_class_count`	Reports the number of decreasing and increasing target-response observations used in the fitted model. These counts should be interpreted together with the data-sufficiency diagnostics because severe imbalance can make posterior summaries less stable.
Jitter metadata	`jitter_used`, `jitter_scale`, `jitter_scale_factor`, `jitter_basis`	Documents whether adaptive response jitter was used and how large the perturbation was. If `jitter_used = TRUE`, retained edges should be reported as jitter-qualified CIRN-inferred regulatory signals.

The most important practical distinction is that `max_rhat` and `min_neff` evaluate posterior sampling quality, whereas `loo_elpd` evaluates predictive adequacy. The $\hat{R}$ diagnostic assesses whether the independent MCMC chains have converged to the same posterior distribution. Conceptually, $\hat{R}$ compares the variability of samples within each chain with the variability between chains. If all chains are exploring the same posterior region, the between-chain and within-chain variation should be similar, and $\hat{R}$ should be close to 1. Values appreciably above 1 indicate that different chains may still be exploring different regions of the posterior, which means that posterior summaries such as ω, odds ratios, and HDI bounds may not yet be stable.

The effective sample-size ratio complements $\hat{R}$ by measuring how much independent posterior information is contained in the MCMC output. In the current CIRN implementation, Bayesian logistic regression is fitted with `brms` using the Stan backend. Stan uses Hamiltonian Monte Carlo, specifically the No-U-Turn Sampler (NUTS), to sample from the posterior distribution. During warm-up, the sampler adapts its tuning parameters and learns the geometry of the posterior. After warm-up, the saved MCMC draws are treated as samples from the posterior distribution of the model parameters, including the CIRN regression coefficients β. The *total posterior draws* are the total number of saved post-warm-up samples across all chains. For example, if four chains each save $1,000$ post-warm-up draws, then the total number of posterior draws is $4,000$.

However, MCMC draws are not always independent. Consecutive draws can be autocorrelated, meaning that one draw may be very similar to the previous draw. When this happens, the posterior sample contains less independent information than its raw size suggests. The *effective posterior draws*, or effective sample size, estimate how many independent draws would provide approximately the same amount of posterior information as the autocorrelated MCMC draws. Thus, 4,000 saved posterior draws might behave like only 800 independent draws if chain autocorrelation is substantial.

In CIRN, the reported value is the effective sample-size ratio,

$$\mathrm{ESS}_{\mathrm{ratio}} = \frac{\text{effective posterior draws}}{\text{total posterior draws}}.$$

A value near 1 means that most saved posterior draws behave like independent information. A small value means that the chains are producing highly autocorrelated samples, so posterior summaries such as ω, odds ratios, and HDI bounds may be less stable. The `min_neff` column reports the smallest ESS ratio among monitored parameters, so it should be interpreted as a worst-case sampling-efficiency summary for the fitted model. When LOO is computed, the reported expected log predictive density can be written as

$$\mathrm{ELPD}_{\mathrm{LOO}} = \sum_{k=1}^{n} \log p(y_k \mid y_{-k}),$$

where y_{-k} denotes all observations except observation k. This quantity is useful for comparing fitted models on predictive grounds, but it is not an edge-selection statistic.

Class and jitter diagnostics should be read together with MCMC summaries. Class counts are defined as

$$n_0 = \#\{Y_i = 0\}, \qquad n_1 = \#\{Y_i = 1\},$$

where class 0 denotes a decreasing target response and class 1 denotes an increasing target response. If adaptive response jitter is used, the perturbation scale is

$$\sigma_J = cB_i,$$

where c is the selected `jitter_scale_factor` and B_i is the target-specific `jitter_basis`. None of these quantities determines whether an edge is retained. Edge retention is determined by the 95%

HDI rule; MCMC and predictive diagnostics determine how confidently the retained edge should be reported.

C. Collinearity diagnostics

The CIRN implementation reports variance inflation factors, model-rank information, aliasing indicators, and pairwise predictor correlations. These diagnostics evaluate whether the predictors in a fitted Bayesian logistic model contain redundant or nearly redundant information. They are especially important for all-predictors models, where state, first-derivative, and second-derivative representations are entered into the same model and can compete to explain the same target directional response.

Collinearity does not mean that a fitted model is automatically invalid. Rather, it means that the model may have difficulty assigning posterior evidence uniquely to one predictor term. For example, if $x_g(t_{k-\ell})$ and $x'_g(t_{k-\ell})$ carry very similar information over the observed time window, the posterior effect attributed to one term may depend on whether the other term is included. In that situation, the retained edge may still be informative, but it should be interpreted as conditional on the fitted predictor set rather than as an isolated effect.

The variance inflation factor (VIF) quantifies how strongly a predictor can be linearly explained by the other predictors in the same fitted model. For predictor u_j, the usual VIF can be written as

$$\mathrm{VIF}_j = \frac{1}{1 - R_j^2},$$

where R_j^2 is obtained by regressing u_j on the remaining predictors in the model. A VIF close to 1 indicates little linear redundancy. Larger VIF values indicate that the coefficient for u_j may be unstable because much of its variation is shared with other predictors. In the current implementation, VIF values below 5 are reported as acceptable, VIF values of at least 5 are flagged as high collinearity, VIF values of at least 10 are flagged as severe collinearity, and infinite VIF values indicate exact collinearity.

The `vif_group` sheet reports term-level collinearity diagnostics for each fitted model. The `term` column identifies the predictor being evaluated, `vif` gives the variance inflation factor when available, and `status` classifies the result as `ok`, `high_collinearity`, `severe_collinearity`, `exact_collinearity`, `single_predictor`, `rank_deficient_model`, or `aliased_coefficient`. The `single_predictor` status simply means that VIF is not defined because the model contains only one predictor. It is not a warning.

The `vif_pairs` sheet reports pairwise predictor correlations. These values answer a simpler question than VIF: whether two predictors are strongly correlated with each other before considering the full predictor set. In the current implementation, pairwise absolute correlations of at least 0.9 are flagged as high pairwise collinearity, and correlations near 1 are flagged as exact or near-exact collinearity. Pairwise correlation is useful for identifying specific redundant predictor pairs, whereas VIF summarizes how much a predictor is explained by all other predictors jointly.

Rank and aliasing diagnostics address a stronger problem than ordinary correlation. Let $\mathbf{U}$ denote the fitted model matrix containing the retained predictors. The model is full rank when the columns of $\mathbf{U}$ provide independent information. If the rank of $\mathbf{U}$ is smaller than the number of model columns, then the design matrix is rank-deficient:

$$\mathrm{rank}(\mathbf{U}) < \text{number of columns in } \mathbf{U}.$$

This means that at least one predictor column is an exact linear combination of other columns. An aliased term is a predictor whose coefficient cannot be estimated separately because its information

is already represented by another predictor or predictor combination. In the output, this is indicated by `rank_deficient = TRUE` or `aliased = TRUE`.

These diagnostics should be used as interpretation qualifiers, not as automatic deletion rules. CIRN does not remove predictors solely because VIF is high. Bayesian regularization and posterior uncertainty can tolerate moderate collinearity, but severe collinearity, rank deficiency, or aliasing can make coefficient signs, magnitudes, odds ratios, and HDI exclusion sensitive to model specification. Therefore, retained edges from highly collinear or rank-deficient models should be described as cautious, conditional, or mode-sensitive unless they are also supported across pairwise, sublevel, and all-predictors outputs.

Practical Interpretation. If an edge is retained in an all-predictors model but the corresponding model has high VIF, severe pairwise correlation, rank deficiency, or aliasing, the edge should not be read as a clean isolated predictor effect. It should be reported as a posterior-supported target-response signal conditional on a collinear predictor set. Cross-mode consistency, biological plausibility, class balance, and MCMC diagnostics should be reviewed before assigning strong interpretation.

D. Class balance and usable sample size

For each fitted model, let

$$n_0 = \#\{Y_i = 0\}, \qquad n_1 = \#\{Y_i = 1\}.$$

Class 0 denotes a decreasing target response, and class 1 denotes an increasing target response. The usable model-ready sample size is

$$n_{\text{usable}} = n_0 + n_1,$$

and the minority class count is

$$n_{\text{minority}} = \min(n_0, n_1).$$

These counts are computed after derivative construction, near-zero response removal, lagging, predictor validation, and complete-row filtering. They should therefore not be confused with the number of rows in the original time-series file.

The `effective_sample_size` sheet is a data-sufficiency audit for the classification problem, not an MCMC effective-sample-size summary. It reports `usable_n`, `minority_class_count`, `n_predictors`, `minority_per_predictor`, `class_balance`, and `sample_size_flag`. In the current implementation,

$$\texttt{class_balance} = \frac{n_{\text{minority}}}{n_{\text{usable}}}, \qquad \texttt{minority_per_predictor} = \frac{n_{\text{minority}}}{p_{\text{model}}},$$

where p_{model} is the number of predictors retained in the fitted model. A class-balance value closer to 0.5 indicates a more balanced increasing/decreasing response, whereas values near 0 indicate severe imbalance.

The script assigns `sample_size_flag` using simple rule-based thresholds. A flag `below_minimum` means that the minority class contains fewer than 5 observations. A flag `small_minority_class` means that the minority class contains 5–9 observations. A flag `low_minority_per_predictor` means that the minority class contains fewer than 5 observations per predictor. A flag `adequate_basic` means that these basic data-sufficiency thresholds were satisfied, although MCMC and collinearity diagnostics should still be reviewed.

Class imbalance affects interpretation because a model may have enough observations to fit the dominant directional response but limited information about the rarer directional state. This issue is most important in multivariable and all-predictors models, where several predictors compete for limited directional-response information. Retained edges from models flagged as `below_minimum`, `small_minority_class`, or `low_minority_per_predictor` should be reported cautiously.

Sufficiency Guideline. A large raw time series does not guarantee a well-supported CIRN fit. The relevant quantity is the model-ready number of increasing and decreasing target-response observations after derivative construction, response filtering, lagging, predictor validation, and complete-row filtering.

E. Adaptive-jitter diagnostics

The adaptive-jitter diagnostics record whether a fitted target response was constructed from the raw target derivative or from the adaptive response-jitter pathway described in the previous section. These diagnostics should be used as reporting qualifiers. They do not create an additional edge-selection rule, and they do not replace the 95% HDI criterion for retaining edges.

For interpretation, the most important distinction is whether `jitter_used` is `FALSE` or `TRUE`. If `jitter_used = FALSE`, the retained edge was fitted using the raw target directional response. If `jitter_used = TRUE`, the retained edge was fitted using an adaptive-jitter directional response and should be described as a jitter-qualified CIRN-inferred regulatory signal.

The adaptive-jitter diagnostic plot should be used to verify that the perturbation is small relative to the target trajectory and that the resulting two-class response is produced by recomputing the derivative, not by manually changing class labels. In the diagnostic figure, the processed target trajectory, minimally jittered trajectory, added jitter, raw derivative, jittered derivative, and resulting increasing/decreasing classes provide a visual audit trail for the reported `jitter_used`, `jitter_scale`, and class-count fields.

A jitter-qualified edge can still be scientifically useful, especially for monotonic or nearly monotonic target trajectories. However, it should not be reported in the same language as an ordinary raw-response fit. Recommended wording is: "This edge was retained under the adaptive response-jitter pathway and should be interpreted as a jitter-qualified CIRN-inferred regulatory signal."

F. Interpreting diagnostic mismatches

The number of rows in `edges_combined` will not generally match the number of rows in `diagnostics`, `effective_sample_size`, `vif_group`, or `vif_pairs`. This is expected. The edge table asks which coefficients survived the HDI rule. The diagnostics table asks whether each fitted model completed and sampled acceptably. The effective-sample-size table asks whether each fitted model had enough usable directional-response information. The VIF tables ask whether the predictor matrix was collinear or rank-deficient.

Thus, an edge table may contain more rows than a diagnostic sheet because one fitted model can produce several retained coefficients. Conversely, a diagnostic sheet may contain model rows that produce no retained edges. The correct interpretation is not based on one-to-one row matching, but on the model identifiers linking the sheets.

Supplementary Table 6: Interpretation of adaptive-jitter diagnostic fields.

Diagnostic field	How to interpret it
`jitter_used`	Indicates whether the fitted target used adaptive response jitter. `FALSE` means the raw directional response was used. `TRUE` means the target response was constructed from a minimally jittered target trajectory.
`response_mode`	Identifies the active response-construction mode. A raw directional response indicates standard CIRN fitting; an adaptive-jitter response indicates that the fitted model should be reported with a jitter qualification.
`response_source`	Identifies the derivative column used to define the binary response. For raw fits, this is the target first derivative. For jitter-supported fits, this is the recomputed first derivative from the minimally jittered target trajectory.
`response_trigger`	Records why the response pathway was used or considered. For example, it may indicate a raw directional response, a one-class raw derivative response, or a near-one-class raw derivative response. This field explains why jitter was or was not required.
`jitter_scale_factor`	Reports the successful scale factor selected from the configured jitter-scale grid. It is useful for documenting how far the algorithm had to search before obtaining a usable two-class response.
`jitter_scale`	Reports the actual jitter standard deviation used for the target trajectory. This is the most direct numerical record of the perturbation magnitude.
`jitter_basis` and `jitter_basis_value`	Identify the scale basis used to convert the selected scale factor into an actual jitter standard deviation. These fields help determine whether the perturbation was scaled relative to the target-state standard deviation, target-state range, derivative scale, or an absolute value.
`jitter_attempts`	Reports how many candidate jitter scales were tried before a usable response was obtained. A small value indicates that only a very small tested perturbation was needed; a large value indicates that the response was more difficult to regularize.
`class_0_count`, `class_1_count`, `min_class_count`	Report the decreasing, increasing, and minority response counts in the fitted model. For jitter-supported fits, these counts describe the model-ready response after adaptive jitter, lagging, predictor validation, and complete-row filtering.
`predictor_jitter_used`	Indicates whether optional predictor-jitter sensitivity mode was used. This should ordinarily be `FALSE` for the primary reported CIRN network. If `TRUE`, the result should be treated as a sensitivity-qualified output rather than an ordinary primary fit.

Model-Level Versus Edge-Level Outputs. Match diagnostic sheets to edge sheets through target, predictor set, analysis mode, lag, response source, and pairwise predictor when available. Do not assume that row numbers correspond across sheets.

Supplementary Figure 2 provides the recommended diagnostic workflow for deciding whether a retained edge should be reported as strong, cautious, exploratory, invalid, or non-reportable. The workflow should be applied after edge retention, because it is a reporting-readiness aid rather than an additional edge-selection rule.

X. ROBUSTNESS AND SENSITIVITY ANALYSES

A. Supported sensitivity scenarios

The final R script includes optional robustness and sensitivity-analysis helpers. These helpers keep the primary CIRN inference rule unchanged: each sensitivity run reruns the full CIRN pipeline under a controlled perturbation, and edges are still retained only by the 95% HDI rule. Sensitivity analysis is disabled by default through `run_sensitivity_analysis = FALSE` because it can be computationally expensive, especially when pairwise fitting and full Bayesian MCMC settings are enabled.

If no custom sensitivity plan is supplied, the script constructs a default plan containing a baseline run and several perturbation scenarios. The supported default scenarios are summarized in Supplementary Table 7.

Supplementary Table 7: Sensitivity scenarios supported by the current CIRN implementation.

Scenario	What the script changes	Main interpretation
`baseline`	Reruns CIRN using the original dataset and primary inference settings.	Provides the reference condition for sensitivity comparison.
`noise_sd_fraction`	Adds Gaussian noise to each numeric non-time state variable before preprocessing. The noise standard deviation is a user-specified fraction of that variable's empirical standard deviation.	Tests whether retained edges are stable to small perturbations of observed state values.
`lag_units`	Keeps the dataset unchanged but reruns CIRN with alternative temporal lags.	Tests whether edge support depends on the assumed response delay.
`downsample_interval`	Keeps every nth observation from the time series.	Tests sensitivity to reduced sampling frequency.
`target_sample_size`	Reduces the trajectory to approximately a specified number of evenly spaced time points.	Tests sensitivity to shorter or smaller observed trajectories.
`row_missing_fraction`	Randomly removes a specified fraction of rows before preprocessing.	Tests robustness to missingness or incomplete sampling.

In the default configuration, noise sensitivity is evaluated at `0.01`, `0.05`, and `0.10` fractions of each variable's standard deviation; lag sensitivity is evaluated at lags 1, 2, and 3; downsampling uses intervals 2 and 5; target sample sizes are 25, 50, 75, and 100; and row-removal fractions are 0.10 and 0.25. The stochastic scenarios, namely added noise and random row removal, are repeated according to `sensitivity_replicates`. In the current default configuration, `sensitivity_replicates = 3`. Deterministic scenarios such as lag changes, downsampling, and target sample-size reduction use one run per setting unless the user supplies a custom plan.

Advanced users may supply a custom `sensitivity_plan`. In that case, the plan must contain the columns `sensitivity_id`, `condition`, `scenario`, `scenario_value`, `value`, and `replicate`.

This allows the user to restrict the sensitivity analysis to a smaller set of scientifically relevant perturbations.

B. Sensitivity inference scope

The setting `sensitivity_inference_scope` controls which CIRN inference modes are rerun inside the sensitivity-analysis loop. This setting is separate from the sensitivity scenario itself. The scenario defines how the data or lag setting is perturbed, whereas the inference scope defines which CIRN model layer is refitted under each perturbation.

Supplementary Table 8: Sensitivity inference scopes supported by the current CIRN implementation.

Scope	Meaning
`use_config`	Uses the main `cirn_config` inference settings. The sensitivity analysis follows the declared `representation_mode`, `run_pairwise`, and `pairwise_representation_mode` used in the primary configuration.
`sublevel`	Reruns only the sublevel multivariable CIRN models, namely the state, first-derivative, and second-derivative representation-specific models.
`all_predictors`	Reruns only the all-predictors multivariable CIRN model, in which eligible state, first-derivative, and second-derivative predictors compete in one joint model for each target.
`both`	Reruns both sublevel and all-predictors multivariable CIRN models, without pairwise CIRN.
`pairwise_only`	Focuses the sensitivity summaries on pairwise retained edges. Pairwise CIRN evaluates one candidate predictor at a time for each target and should be interpreted as marginal temporal-response evidence rather than multivariable conditional evidence.
`everything`	Reruns sublevel, all-predictors, pairwise sublevel, and pairwise all-predictors CIRN. This is the broadest and usually most computationally expensive sensitivity scope.

For routine robustness checks, `use_config` is the safest default because it makes sensitivity analysis match the primary declared analysis. The narrower scopes, such as `sublevel` or `all_predictors`, are useful when the analyst wants to isolate a specific model layer. The `everything` scope is most appropriate for final robustness appendices or extensive dissertation reporting, because it reruns all supported inference layers and can substantially increase runtime.

C. Sensitivity outputs

When sensitivity analysis is enabled, the script stores several outputs in the result compendium. The `sensitivity_plan` sheet records the planned scenarios. The `sensitivity_runs` sheet records whether each run completed, failed during data preparation, or failed during inference, together with raw sample size, scenario sample size, and runtime. The `sensitivity_edges` sheet stores retained edges from each sensitivity run. The `sensitivity_diagnostics` and `sensitivity_effective_`

`sample_size` sheets store model diagnostics and data-sufficiency summaries for the sensitivity fits. The `sensitivity_edge_stability` sheet summarizes state-level signed source–target recovery across completed runs, whereas the `sensitivity_feature_edge_stability` sheet summarizes feature-level retained-edge recovery while preserving state, first-derivative, and second-derivative predictor representations. If sensitivity CSV outputs are saved, the corresponding feature-level stability file is `CIRN_sensitivity_feature_edge_stability.csv`. If a signed reference adjacency matrix is supplied and ground-truth validation is enabled, the script also produces `sensitivity_ground_truth_metrics`.

The state-level stability summary is representation-agnostic at the source-variable level. That is, derivative terms are mapped back to their base predictor variable when stability is summarized. For example, X, X', and X'' are treated as the same source variable X for the purpose of state-level edge-stability counting. State-level sensitivity stability answers whether a base regulator-to-target relation is stable after collapsing derivative representations.

The feature-level stability summary preserves the predictor representation. Feature-level sensitivity stability answers whether a specific predictor representation, such as X, dX, or d^2X, is stable across perturbation scenarios. This does not remove representation information from the original `sensitivity_edges` table; instead, it provides a second stability summary for cases where state, velocity, and curvature terms should not be collapsed into the same source variable.

For each scenario and scenario value, the script computes

$$\text{detection rate} = \frac{\text{number of completed runs in which the signed edge was detected}}{\text{number of completed runs for that scenario value}}.$$

A detection rate close to 1 indicates that the signed edge was repeatedly recovered under that perturbation. A detection rate near 0 indicates that the edge was not stable under that sensitivity condition.

D. How sensitivity outputs should be interpreted

Sensitivity outputs are not part of the primary edge-retention rule. They provide empirical evidence about whether retained edges persist under controlled changes in noise level, lag choice, sampling frequency, trajectory length, and missingness. Stable edges across planned sensitivity conditions can be prioritized for discussion because they are less dependent on the exact analysis configuration. Unstable edges should be interpreted as condition-sensitive hypotheses rather than automatically dismissed as false.

A sensitivity-stable edge is not automatically causal, and a sensitivity-unstable edge is not automatically wrong. Stability means that the edge is less dependent on the perturbations that were tested. Instability means that the edge may depend on sampling density, lag choice, noise, missingness, model-ready class balance, or collinearity under the altered dataset. In manuscript reporting, sensitivity results are best used as qualifiers, such as "retained and stable across planned sensitivity scenarios," "retained but lag-sensitive," or "retained only under the primary configuration." This language is more informative than labeling edges simply as correct or incorrect.

E. Ground-truth validation

Ground-truth validation is optional and should be used only when a defensible signed reference network is available, such as in simulated or benchmark systems. The reference network must be supplied as a square signed adjacency matrix with matching row and column names. In the current

implementation, entries are interpreted as

$$A_{gi} = \begin{cases} +1, & \text{if source } g \text{ activates target } i, \\ -1, & \text{if source } g \text{ inhibits target } i, \\ 0, & \text{if no reference edge } g \text{ to } i \text{ is present.} \end{cases}$$

Rows correspond to source variables and columns correspond to target variables.

Before validation, CIRN converts the retained edge list into a state-level signed predicted adjacency matrix $\widehat{A}$. Derivative-based predictor terms are mapped back to their base state variable only for this evaluation step. Thus, X, X', and X'' are treated as source X when comparing against the reference graph. This mapping does not remove representation information from the original edge list; it only allows a representation-aware CIRN output to be compared with a conventional state-level reference network.

If multiple retained CIRN terms map to the same source–target pair, the current script keeps the sign of the term with the largest absolute posterior mean effect, $|\omega|$, for the state-level evaluation matrix. The inferred sign is assigned as

$$\widehat{A}_{gi} = \begin{cases} +1, & \text{if the selected retained effect has } \omega > 0, \\ -1, & \text{if the selected retained effect has } \omega < 0, \\ 0, & \text{if no retained edge } g \text{ to } i \text{ is present.} \end{cases}$$

The confusion-matrix components are then computed from A and $\widehat{A}$. A true positive requires both correct edge recovery and correct regulatory sign:

$$TP = \#\{(g,i) : \widehat{A}_{gi} \neq 0,\ A_{gi} \neq 0,\ \operatorname{sign}(\widehat{A}_{gi}) = \operatorname{sign}(A_{gi})\}.$$

A wrong-sign recovery occurs when the source–target edge is present in both matrices but the activation/inhibition sign is incorrect:

$$SE = \#\{(g,i) : \widehat{A}_{gi} \neq 0,\ A_{gi} \neq 0,\ \operatorname{sign}(\widehat{A}_{gi}) \neq \operatorname{sign}(A_{gi})\}.$$

In the script, these wrong-sign recoveries are counted as both false positives and false negatives because the interaction was detected but assigned the wrong regulatory sign:

$$FP = \#\{(g,i) : \widehat{A}_{gi} \neq 0,\ A_{gi} = 0\} + SE,$$

$$FN = \#\{(g,i) : \widehat{A}_{gi} = 0,\ A_{gi} \neq 0\} + SE,$$

and

$$TN = \#\{(g,i) : \widehat{A}_{gi} = 0,\ A_{gi} = 0\}.$$

From these quantities, the script reports

$$\text{Precision} = \frac{TP}{TP+FP}, \qquad \text{Recall} = \frac{TP}{TP+FN}, \qquad \text{Specificity} = \frac{TN}{TN+FP}$$

$$\text{Balanced Accuracy} = \frac{\text{Recall} + \text{Specificity}}{2}, \qquad F_1 = \frac{2 \times \text{Precision}\,\text{Recall}}{\text{Precision} + \text{Recall}},$$

and the Matthews correlation coefficient,

$$\text{MCC} = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP+FP)(TP+FN)(TN+FP)(TN+FN)}}.$$

Precision measures how many inferred signed edges are correct. A precision value near 1 means that most edges reported by CIRN are present in the reference network with the correct sign. A low precision value means that many inferred edges are false positives or wrong-sign recoveries. Recall measures how many true signed edges are recovered. A recall value near 1 means that CIRN recovered most reference interactions; a low recall value means that many true interactions were missed. Specificity measures how well non-edges are preserved. A specificity value near 1 means that CIRN correctly avoided adding edges where the reference network has no interaction.

Balanced accuracy averages recall and specificity, which is useful when true edges are much rarer than non-edges. A balanced accuracy near 1 indicates good recovery of both true interactions and true non-interactions. A value near 0.5 indicates performance close to chance for a binary edge-recovery problem. The F_1 score balances precision and recall; it is high only when inferred edges are mostly correct and most true edges are recovered. MCC summarizes all four confusion-matrix components. MCC ranges from -1 to 1: 1 indicates perfect agreement with the reference network, 0 indicates performance close to random guessing, and -1 indicates systematic disagreement. MCC is especially useful for sparse regulatory networks because it accounts for true positives, false positives, false negatives, and true negatives simultaneously.

The script also reports structural Hamming distance:

$$\text{SHD} = \sum_{g,i} \mathbf{1}\left[(\widehat{A}_{gi} \neq 0) \neq (A_{gi} \neq 0)\right],$$

which counts disagreement in edge presence or absence while ignoring sign. In plain terms, SHD counts how many edge additions or deletions would be needed to make the inferred network match the reference network. A value of 0 means that the inferred and reference networks have the same edge skeleton. Larger values indicate more structural disagreement.

Signed structural Hamming distance is

$$\text{signed SHD} = \sum_{g,i} \mathbf{1}\left[\widehat{A}_{gi} \neq A_{gi}\right],$$

which also penalizes incorrect activation/inhibition signs. A signed SHD of 0 means that the inferred network matches the reference network exactly, including edge direction and regulatory sign. If SHD is low but signed SHD is higher, CIRN may be recovering the correct source–target pairs but assigning some signs incorrectly. Lower SHD and lower signed SHD indicate closer agreement with the reference network.

As a practical guide, values closer to 1 are better for precision, recall, specificity, balanced accuracy, F_1, and MCC, except that MCC can also be negative when the inferred network systematically disagrees with the reference. Values closer to 0 are better for SHD, signed SHD, false positives, false negatives, and wrong-sign recoveries.

These validation metrics should not be interpreted as the primary evidence for empirical datasets whose true regulatory graph is incomplete or unknown. For empirical biological or environmental systems, forcing a ground-truth comparison can be misleading. For simulated and benchmark systems, however, these metrics provide a useful state-level recovery audit and should be interpreted alongside the representation-specific CIRN edge table, because a correct state-level recovery may have been supported by state, velocity, curvature, or all-predictors evidence.

Caveat: Time-Varying Network Structure or Network Rewiring. Ground-truth validation assumes that the supplied reference network is fixed over the time window being analyzed. In many biological, ecological, and environmental systems, the effective regulatory structure may change through time because of regime shifts, developmental stage, environmental forcing, feedback adaptation, or network rewiring. In such cases, a single static reference adjacency matrix may only approximate the dominant or averaged network architecture. A low recovery score may therefore reflect true time-varying regulation, context-specific interactions, or transient rewiring rather than failure of the inference method alone. When time-varying structure is scientifically plausible, ground-truth validation should be interpreted as recovery of the supplied reference architecture, not as exhaustive validation of all regulatory interactions active during the observation window.

XI. CIRN OUTPUT COMPENDIUM

A. Major output sheets

The Excel compendium preserves the structure of the analysis rather than compressing all results into one table. Sheets differ in row count because they represent different units of analysis: some are run-level, some are model-level, some are edge-level, and some are predictor- or predictor-pair-level. The safest reading order is to begin with `settings`, inspect retained edges, and then check the corresponding diagnostics, data-sufficiency summaries, and collinearity outputs.

Supplementary Table 9: Principal CIRN output sheets.

Sheet	Unit	Purpose
`cirn_summary`	Run-level	Run-level summary, including runtime when available.
`settings`	Run-level	Main analysis configuration, including targets, predictors, lag, smoothing, jitter, prior, MCMC, pairwise, LOO, and related inference settings.
`edges`	Edge-level	Retained multivariable CIRN edges selected by the 95% HDI rule.
`pairwise_edges`	Edge-level	Retained one-predictor-at-a-time diagnostic edges.
`edges_combined`	Edge-level	Combined retained edge table containing multivariable and pairwise edges; interpret using `analysis_mode` and `predictor_set`.
`diagnostics`	Model-level	MCMC, response, class-count, jitter, and optional LOO diagnostics for multivariable fits.

Sheet	Unit	Purpose
pairwise_diagnostics	Model-level	Same diagnostic summaries for pairwise fits.
effective_sample_size	Model-level	Data-sufficiency audit reporting usable sample size, class balance, minority class count, predictors per model, and sample-size flags.
edge_consistency	Term–target-level	Cross-mode support summary showing whether a term–target pair appears in pairwise, sublevel, and all-predictors outputs, including sign conflicts.
vif_group	Predictor-term-level	VIF, rank, and aliasing diagnostics for multivariable models.
vif_pairs	Predictor-pair-level	Pairwise correlation diagnostics among multivariable predictors.
pairwise_vif_group	Predictor-term-level	VIF diagnostics for pairwise fits when applicable.
pairwise_vif_pairs	Predictor-pair-level	Pairwise correlation diagnostics produced for pairwise fits when applicable.
evaluation_metrics	Network-level	Optional ground-truth recovery metrics for systems with a known signed reference adjacency matrix.
sensitivity_plan	Scenario-level	Planned robustness scenarios, including baseline, noise, lag, downsampling, target sample size, and missingness settings.
sensitivity_runs	Run-level	Completion status, sample size, runtime, and messages for each sensitivity run.
sensitivity_edges	Edge-level	Retained edges from each completed sensitivity run.
sensitivity_diagnostics	Model-level	Model diagnostics from sensitivity runs.

Sheet	Unit	Purpose
`sensitivity_effective_sample_size`	Model-level	Data-sufficiency summaries from sensitivity runs.
`sensitivity_edge_stability`	State-level edge summary	Detection-rate summary showing how often signed source–target relationships are recovered across sensitivity runs.
`sensitivity_ground_truth_metrics`	Network-level	Optional ground-truth recovery metrics computed within sensitivity runs when a reference adjacency matrix is supplied.

Optional sheets may be absent or empty when the corresponding analysis is disabled. For manuscript tables, `edges_combined` is usually the primary source because it contains retained multivariable and pairwise edges in a single table. However, it should not be used alone. At minimum, retained edges should be interpreted together with `diagnostics`, `effective_sample_size`, and the relevant VIF or pairwise-correlation sheets. For benchmark systems, `evaluation_metrics` should be reported separately because it summarizes recovery against a known reference structure rather than posterior edge evidence.

B. Core edge-table columns

The `edges`, `pairwise_edges`, and `edges_combined` sheets contain retained coefficients only. In the current implementation, a coefficient is retained when its 95% HDI excludes zero. Therefore, the edge tables should be read as posterior-supported edge lists rather than as complete coefficient tables containing both retained and non-retained terms.

Supplementary Table 10: Key edge-table columns for reporting and interpretation.

Column	Type	Meaning
`analysis_mode`	Model source	Indicates whether the row came from a `multivariable` fit or a `pairwise` fit.
`predictor_set`	Representation	Indicates whether the retained term came from `original`, `first_derivative`, `second_derivative`, or `all_predictors`.
`term`	Predictor	Retained predictor term. Terms ending in `_d1` are first derivatives; terms ending in `_d2` are second derivatives. Terms without these suffixes are state-level predictors.

Column	Type	Meaning
`target`	Response	Target directional response, usually written as `dX`, `dY`, and so on. This represents the binary increase/decrease response of the target variable.
`omega`	Effect	Posterior mean coefficient, $\omega = \mathbb{E}[\beta \mid \text{data}]$. Positive values increase the posterior probability of target increase; negative values decrease it.
`odds_ratio`	Effect	$\exp(\omega)$. Because predictors are standardized before fitting, this is the multiplicative change in the odds of target increase for a one-standard-deviation increase in the lagged predictor representation.
`sign`	Effect	Numeric sign of the posterior mean coefficient: $+1$ for positive effects and -1 for negative effects.
`regulation_type`	Effect	`activation` if $\omega > 0$, and `inhibition` if $\omega < 0$, interpreted on the target directional-response scale.
`rel_strength`	Reporting	Relative effect-size category among retained effects for the same target. This is for table reporting only and does not affect edge inclusion.
`hdi_lower95`, `hdi_upper95`	Inclusion rule	Bounds of the 95% highest density interval. A row appears in the retained edge table only when this interval excludes zero.
`eti_lower95`, `eti_upper95`	Posterior summary	Equal-tailed interval bounds. These are auxiliary posterior summaries and are not the edge-retention rule.
`response_mode`	Response provenance	Indicates whether the response was constructed from the raw directional derivative or from the adaptive-jitter directional response.
`response_label`	Response provenance	Human-readable label for the response source, such as raw first-derivative direction or jittered first-derivative direction.

Column	Type	Meaning
response_source	Response provenance	Derivative column used to construct the active binary response.
response_trigger	Response provenance	Records why the response pathway was used, such as raw directional response, one-class raw derivative, or near-one-class raw derivative.
dominant_direction	Response provenance	Sign of the dominant raw derivative direction when available. This helps identify monotonic or nearly monotonic target behavior.
class_0_count, class_1_count	Data sufficiency	Number of decreasing and increasing target-response observations in the model-ready data.
min_class_count	Data sufficiency	Smaller of the two response-class counts. This is important for judging class imbalance.
jitter_used	Jitter provenance	Indicates whether adaptive response jitter was used for the fitted target model.
jitter_scale_factor, jitter_scale, jitter_basis	Jitter provenance	Record the selected jitter scale factor, actual jitter standard deviation, and basis used to scale the jitter.
jitter_basis_value, jitter_attempts	Jitter provenance	Record the numerical basis value and the number of candidate jitter scales attempted before success.
predictor_jitter_used	Sensitivity provenance	Indicates whether optional predictor-jitter sensitivity mode was used. This should usually be FALSE for primary results.
predictor_jitter_scale_factor, predictor_jitter_variables	Sensitivity provenance	Record predictor-jitter settings when optional predictor-jitter sensitivity mode is enabled.
pairwise_target, pairwise_predictor	Pairwise bookkeeping	Identify the target and predictor used in a pairwise diagnostic fit. These are mainly relevant when analysis_mode = pairwise.

Column	Type	Meaning
`effect_interpretation`	Reporting	Text label clarifying whether a positive coefficient increases the probability of target increase under the raw or adaptive-jitter response construction.
`method`	Model source	Fitting method, reported as Bayesian logistic regression through `brms`.

When preparing a journal table, the most informative columns are usually `analysis_mode`, `predictor_set`, `term`, `target`, ω, odds ratio, 95% HDI, `regulation_type`, class counts, and jitter status. Large supplementary tables can retain response provenance and jitter metadata, whereas main-text tables should avoid mixing edge-level, model-level, and predictor-level diagnostics in a way that suggests one-to-one row matching where none exists.

For a manuscript-style implementation of this reporting logic, see Supplementary Tables 16 and 17. Supplementary Table 16 reorganizes the retained rows from `edges_combined` into a reader-facing expanded CIRN edge list for the deterministic Lotka–Volterra predator–prey system, with cross-mode support summarized from `edge_consistency`. Supplementary Table 17 reports model-level diagnostics for the same CIRN analysis; its rows correspond to fitted model configurations rather than retained edges, thereby avoiding row-count confusion between the edge-list output and diagnostic summaries.

Because these sheets have different units of analysis, they should be linked by shared identifiers such as `target`, `predictor_set`, `analysis_mode`, `response_source`, and `pairwise_predictor` when applicable, rather than by row number.

Important Reporting Note. The edge table does not contain every fitted coefficient. It contains retained coefficients after the 95% HDI rule has been applied. Non-retained coefficients should be inspected from fitted model objects or posterior summaries if a complete coefficient audit is needed.

XII. COMPUTATIONAL COST AND IMPLEMENTATION

A. Number of model fits

The computational cost of CIRN is driven mainly by the number of Bayesian logistic regression models that must be fitted. In the current implementation, a *model fit* means one target-wise Bayesian logistic regression for one predictor representation or representation set. Let n_T be the number of target variables and n_P be the number of candidate predictor variables.

The number of representation-specific model fits per target depends on the selected representation mode:

$$r(m) = \begin{cases} 3, & m = \texttt{sublevel}, \\ 1, & m = \texttt{all_predictors}, \\ 4, & m = \texttt{both}. \end{cases}$$

The `sublevel` mode fits three separate models for each target: one using state predictors, one using first-derivative predictors, and one using second-derivative predictors. The `all_predictors` mode

fits one joint model for each target using all eligible state, first-derivative, and second-derivative predictors together. The `both` mode fits all four model types.

For the primary multivariable layer, the number of scheduled model configurations is

$$M_{\text{multi}} = n_T \, r(m_{\text{multi}}),$$

where m_{multi} is the selected `representation_mode`.

If pairwise mode is disabled, the total scheduled model count is simply

$$M_{\text{total}} = M_{\text{multi}}.$$

If pairwise mode is enabled, the script additionally fits one target–predictor analysis at a time. If m_{pair} is the selected `pairwise_representation_mode`, the pairwise layer adds

$$M_{\text{pair}} = n_T n_P \, r(m_{\text{pair}})$$

scheduled model configurations, and the total scheduled model count becomes

$$M_{\text{total}} = M_{\text{multi}} + M_{\text{pair}}.$$

In the script, this total is stored as `total_model_fits` in the `settings` output.

This count is best interpreted as the number of scheduled model configurations, not necessarily the number of successful posterior fits. Some configurations may be skipped before or during model fitting if the target response is one-class, if lagging and complete-row filtering remove one response class, if no valid predictors remain, or if the Bayesian model fails numerically. These cases are recorded through statuses such as `skipped_single_class`, `skipped_after_lagging`, `skipped_no_predictors`, or `failed_model_fit`. The `completed_model_fits` field records progress through the scheduled configurations and should be interpreted together with the diagnostic-status fields.

The MCMC burden per completed Bayesian fit is determined by the number of chains and iterations. With `model_chains` (chains), `model_iter` (total iterations per chain), and `model_warmup` (warm-up iterations per chain), the number of saved post-warm-up posterior draws per fitted model is

$$D_{\text{saved}} = \texttt{model_chains}\,(\texttt{model_iter} - \texttt{model_warmup})\,.$$

With the current script settings $\texttt{model_chains} = 4$, $\texttt{model_iter} = 3000$, and $\texttt{model_warmup} = 1000$, this gives

$$D_{\text{saved}} = 4(3000 - 1000) = 8000$$

saved posterior draws per completed Bayesian model. The approximate posterior-sampling workload scales with

$$M_{\text{total}} \times \texttt{model_chains} \times \texttt{model_iter},$$

before considering additional costs from LOO computation, posterior summarization, diagnostics, plotting, validation, or sensitivity runs.

Sensitivity analysis multiplies the cost because it reruns the CIRN pipeline for every row of the sensitivity plan. Under the default sensitivity helper, the plan includes a baseline run, replicated Gaussian-noise scenarios, alternative lag settings, downsampling settings, target sample-size settings, and replicated row-missingness scenarios. The cost also depends on `sensitivity_inference_scope`, because this setting determines which CIRN model layers are refitted inside each sensitivity run. For example, `everything` is much more expensive than `sublevel`, because it reruns sublevel, all-predictors, pairwise sublevel, and pairwise all-predictors analyses rather than only the sublevel multivariable layer. Each sensitivity run has its own scheduled model count and runtime record, so sensitivity analysis should be enabled only after the primary configuration has been checked.

B. Runtime considerations

The current implementation fits each CIRN model as a Bayesian logistic regression using `brms` with the `cmdstanr` backend. For a fitted target x_i and an active predictor set, the response is binary:

$$Y_i(t_k) = \begin{cases} 1, & \text{if the target is increasing at } t_k, \\ 0, & \text{if the target is decreasing at } t_k. \end{cases}$$

The model therefore uses a Bernoulli likelihood:

$$Y_i(t_k) \sim \text{Bernoulli}\{\pi_i(t_k)\},$$

where

$$\pi_i(t_k) = P\{Y_i(t_k) = 1 \mid \mathbf{z}(t_{k-\ell})\}$$

is the posterior-modeled probability that target x_i is increasing at time t_k, conditional on the standardized lagged predictors. Because $\pi_i(t_k)$ is a probability, it must lie between 0 and 1. The logit link maps this probability to the real line:

$$\text{logit}\{\pi_i(t_k)\} = \log\left(\frac{\pi_i(t_k)}{1-\pi_i(t_k)}\right) = \beta_0 + \sum_{j=1}^{p_m} \beta_j z_j(t_{k-\ell}),$$

where $z_j(t_{k-\ell})$ is the jth standardized lagged predictor in the fitted model and p_m is the number of retained predictors in that model configuration. A positive coefficient increases the log-odds, and therefore the probability, that the target is increasing; a negative coefficient decreases that probability.

In the R script, this model is implemented through `brms::brm()` as:

```
brms::brm(
  formula = form,
  data    = df_derivs,
  family  = brms::bernoulli(link = "logit"),
  prior   = coef_prior,
  iter    = model_iter,
  warmup  = model_warmup,
  chains  = model_chains,
  cores   = model_cores,
  control = list(adapt_delta = adapt_delta),
  seed    = seed,
  refresh = 0,
  backend = "cmdstanr"
)
```

Here, `form` is the target-specific logistic-regression formula, `df_derivs` is the model-ready data after response construction, lagging, predictor filtering, complete-row filtering, and predictor standardization, and `coef_prior` is the Normal prior placed on the regression coefficients through `class = "b"`. The `brms` package translates the model into Stan code, and `cmdstanr` runs the Stan sampler. Stan samples from the posterior distribution using Hamiltonian Monte Carlo, specifically the No-U-Turn Sampler (NUTS).

The `model_cores` setting controls the number of CPU cores requested for a single fitted Bayesian model, mainly to allow multiple MCMC chains to run in parallel. For example, if `model_chains = 4` and `model_cores = 2`, the fitted model has four chains, but at most two cores are requested for chain-level parallel execution at a time.

The script schedules target-wise and pairwise model fits sequentially through the R workflow. It does not launch every pairwise model as a separate simultaneous parallel job. Instead, each scheduled model configuration invokes `brms`/Stan, and within that model fit, `model_cores` controls chain-level parallelism. This distinction matters because a run with many pairwise models can still be expensive even if `model_cores` is small: the cost accumulates across many repeated Bayesian fits.

Runtime increases with the number of targets, predictors, representation modes, pairwise fits, MCMC iterations, chains, optional LOO computation, and optional sensitivity runs. Pairwise mode can be especially expensive because it scales with $n_T n_P$ in addition to the representation count. Sensitivity analysis can be even more expensive because it reruns the full CIRN pipeline across multiple perturbation scenarios. For this reason, the script warns users when `run_pairwise = TRUE` and `model_cores` is greater than 2. The configuration comments recommend modest core counts, often 1 or 2, for long pairwise runs.

The script also limits several low-level thread pools before Stan-related model fitting begins:

$$\begin{aligned}\texttt{OMP_NUM_THREADS} &= \texttt{STAN_NUM_THREADS} = \texttt{TBB_NUM_THREADS} \\ &= \texttt{VECLIB_MAXIMUM_THREADS} = \texttt{MKL_NUM_THREADS} \\ &= 1.\end{aligned}$$

Briefly, `OMP_NUM_THREADS` limits OpenMP-based parallel threads, `STAN_NUM_THREADS` limits Stan's internal threading, `TBB_NUM_THREADS` limits Intel Threading Building Blocks threads used by Stan and related numerical routines, `VECLIB_MAXIMUM_THREADS` limits Apple's Accelerate/vecLib linear-algebra threading on macOS, and `MKL_NUM_THREADS` limits Intel Math Kernel Library threading when MKL-backed numerical routines are available. Setting each value to 1 makes these low-level libraries use one internal thread each, reducing nested parallelism during repeated `brms`/CmdStan fits.

Here, 1 means one internal thread per native numerical thread pool. It does not mean one model fit, one MCMC chain, or one posterior draw. The purpose is to prevent nested parallelism. Nested parallelism can occur when `brms`/CmdStan runs multiple chains while lower-level numerical libraries also try to create additional internal threads. This can overload the machine, especially when `run_pairwise = TRUE` schedules many Bayesian models or when sensitivity analysis repeats the full pipeline many times. The main user-facing controls for Bayesian parallelism remain `model_chains` and `model_cores`.

The primary runtime reported by the script is stored as `CIRN inference runtime (seconds)` in the `cirn_summary` output. This runtime is measured around the main call to `infer_network()`. It covers the primary CIRN inference call, including Bayesian fitting and diagnostics computed inside `infer_network()`, such as LOO when `compute_loo = TRUE`. It does not include downstream plotting, optional ground-truth validation, output-table construction, workbook writing, or sensitivity analysis performed after the primary inference call. Sensitivity runs have their own `runtime_seconds` entries in the `sensitivity_runs` output.

Long runtime does not necessarily indicate an error. It may reflect a large number of scheduled models, many MCMC chains and iterations, optional LOO computation, or difficult posterior geometry caused by class imbalance, collinearity, near-separation, or weakly identified coefficients. If runtime becomes prohibitive, a practical workflow is to first run a smaller diagnostic analysis using fewer targets, fewer predictors, pairwise mode disabled, shorter chains, LOO disabled, and sensitivity analysis disabled. Final reported analyses should then be rerun with the intended configuration and complete diagnostic review.

C. Reproducibility record

The implementation records the core inference settings and software environment needed to reproduce a CIRN analysis. The main `settings` sheet is produced from the settings returned by `infer_network()`. It records the analysis configuration used inside the primary inference call, including the time column, resolved target set, resolved predictor set, representation mode, lag, pairwise setting, derivative-grid setting, smoothing and outlier settings, response tolerance, adaptive-jitter settings, Bayesian MCMC settings, prior settings, LOO settings, random seed, progress settings, and the scheduled and completed model-fit counts.

This record is important because several configuration fields directly change the scientific question being asked. For example, changing `targets`, `predictors`, or `lag_units` changes the modeled regulatory hypothesis. Changing `representation_mode` changes which predictor representations are evaluated. Changing `response_eps`, smoothing, outlier handling, or adaptive-jitter settings changes how the directional response and derivative predictors are constructed. Changing `model_iter`, `model_warmup`, `model_chains`, `prior_sd`, or `adapt_delta` changes how the Bayesian posterior is estimated.

The script also writes the R session information to

`sessionInfo_CIRN.txt`,

which records the R version, platform, attached packages, and package versions available during the run. This file is saved in the same output directory as the input data and generated results. Because package versions, `brms`, `cmdstanr`, and the local Stan toolchain can affect Bayesian fitting, this session record is an important part of reproducible reporting. For strict reproducibility, the exact R script and the local CmdStan installation/version should also be preserved with the analysis files.

Some run-level choices are controlled in the user-edited script but are not necessarily part of the core `settings` sheet returned by `infer_network()`. These include the input-file path, output directory, figure-export settings, optional ground-truth validation switch, optional sensitivity-analysis switch, and the full user-edited `cirn_config` block. Therefore, the final analysis folder should preserve not only the Excel compendium and `sessionInfo_CIRN.txt`, but also the exact input data file and exact R script used to generate the results.

For analyses with optional robustness or validation components, additional records should be retained. Sensitivity analysis settings and outcomes are represented by `sensitivity_plan`, `sensitivity_runs`, `sensitivity_edges`, `sensitivity_diagnostics`, `sensitivity_effective_sample_size`, `sensitivity_edge_stability`, and `sensitivity_feature_edge_stability` when `run_sensitivity_analysis = TRUE`. The ground-truth validation results are recorded in `evaluation_metrics` when `run_ground_truth_validation = TRUE` and a signed reference adjacency matrix is supplied.

For publication, the minimum reproducibility record should include the input dataset or data-generation procedure, the exact R script used, the time column, declared target and predictor sets, lag value, representation mode, pairwise setting, smoothing and outlier settings, response tolerance, adaptive-jitter settings, prior settings, MCMC settings, random seed, optional LOO setting, optional validation setting, optional sensitivity setting, output compendium, and `sessionInfo_CIRN.txt`. Together, these files define the computational conditions under which the reported CIRN network, diagnostics, and sensitivity or validation summaries were produced.

Practical Recommendation. For each final reported analysis, keep one analysis folder containing the input data, exact `CIRN_Algorithm.R` script, output compendium, generated figures, optional sensitivity CSV files, and `sessionInfo_CIRN.txt`. This folder-level record is the safest way to make the reported CIRN results reproducible.

XIII. MANUAL FOR THE CIRN ALGORITHM COMPUTATIONAL SCRIPT

A. Overview of the current CIRN implementation

The current CIRN implementation is a script-based R workflow for reconstructing directed and signed regulatory hypotheses from single or multivariate time-series data. It implements the same methodological logic described in the main article: observed trajectories are smoothed, first and second temporal derivatives are estimated, each target response is encoded according to the sign of its local derivative, and lagged state-, velocity-, and curvature-based predictor representations are used to model whether the target variable is increasing or decreasing at a later time point.

For a target variable x_i, the script constructs the binary directional response

$$Y_i(t_k) = \begin{cases} 1, & x_i'(t_k) > \varepsilon, \\ 0, & x_i'(t_k) < -\varepsilon, \end{cases}$$

while observations satisfying $|x_i'(t_k)| \le \varepsilon$ are removed as near-stationary or numerically ambiguous. Candidate predictors are then temporally lagged so that information at $t_{k-\ell}$ is used to model the target directional response at t_k. After lag alignment and complete-row filtering, retained predictors are standardized and fitted in Bayesian logistic regression models.

In its general form, each fitted CIRN model estimates

$$\Pr\{Y_i(t_k) = 1 \mid \mathbf{z}(t_{k-\ell})\} = \pi_i(t_k), \qquad \operatorname{logit}\{\pi_i(t_k)\} = \beta_0 + \sum_j \beta_j z_j(t_{k-\ell}),$$

where $Y_i(t_k) = 1$ denotes an increasing target response, $Y_i(t_k) = 0$ denotes a decreasing target response, and $z_j(t_{k-\ell})$ denotes a standardized lagged predictor representation. Depending on the selected representation mode, the predictor set may contain state variables, first derivatives, second derivatives, or all eligible representations in a joint all-predictors model.

Posterior coefficient summaries are used to define the CIRN edge output. For each coefficient β_j, the script reports the posterior mean

$$\omega_j = \mathbb{E}[\beta_j \mid \text{data}],$$

the odds ratio $\exp(\omega_j)$, the 95% highest density interval, and the corresponding regulatory sign. A coefficient is retained as a CIRN edge only when its 95% HDI excludes zero. Positive posterior means are reported as activating effects in the target-response scale, whereas negative posterior means are reported as inhibitory effects in the target-response scale.

The implementation is intentionally distributed as a single executable R script so that users can run a complete analysis without installing a separate R package. Internally, however, the script is organized as a modular pipeline. The user edits a central `cirn_config` object, and the run block executes derivative construction, response construction, predictor preparation, Bayesian fitting, posterior edge extraction, diagnostics, visualization, validation, sensitivity analysis, and output export in a fixed order.

The central inference function is `infer_network()`. It calls the target-level routine `infer_for_target()`, which constructs the target-specific response and model-ready predictors, and then calls the Bayesian fitting routine `run_bayes_logit()`. This hierarchy reflects the scientific structure of CIRN: the network is assembled from target-wise directional-response models, and each retained edge is supported by a posterior coefficient summary under a specified lag, predictor set, representation mode, and diagnostic context.

The script writes both analysis-level and publication-oriented outputs. These include retained edge tables, all fitted coefficients including non-retained terms, model diagnostics, effective-sample-size summaries, collinearity diagnostics, edge-consistency summaries, optional ground-truth validation metrics, optional sensitivity-analysis outputs, static and interactive figures, the multi-sheet `CIRN_Results_Compendium.xlsx`, the two-sheet `CIRN_Publication_Tables.xlsx`, a LaTeX publication edge table, and `sessionInfo_CIRN.txt` for reproducibility.

A CIRN edge should be interpreted as a directed, signed, temporally ordered, posterior-supported regulatory hypothesis in the target-response scale. It does not by itself prove interventional causality, identify a direct biochemical or ecological mechanism, or recover the full governing equation of the system. Its interpretation depends on the declared target and predictor sets, lag choice, derivative quality, response balance, collinearity diagnostics, MCMC diagnostics, and any jitter or sensitivity settings used in the run.

B. Software requirements and package roles

The current CIRN script requires R together with packages for data handling, Bayesian modeling, posterior summarization, network visualization, diagnostic plotting, validation, and output export. The core inferential engine is Bayesian logistic regression fitted with `brms` using the `cmdstanr` backend. Thus, a working CmdStan installation and a compatible C++ toolchain are required before running the full Bayesian inference pipeline.

In the script, the CmdStan backend is enforced through

```
options(brms.backend = "cmdstanr").
```

The fitted models are run through `brms::brm()` with a Bernoulli likelihood and logit link. The package `brms` translates the model formula into Stan code, `cmdstanr` calls CmdStan, and Stan performs posterior sampling using Hamiltonian Monte Carlo (HMC) through the No-U-Turn Sampler (NUTS).

Supplementary Table 11: Software dependencies used by the current CIRN implementation.

Category	Package or tool	Required?	Role in the CIRN script
Base R	`stats`	Yes	Smoothing splines, aggregation, correlations, linear models, random-number generation, and probability calculations.
Base R	`utils`	Yes	Command-line arguments, CSV writing, progress bars, and general utility functions.

Category	Package or tool	Required?	Role in the CIRN script
Base R	`grDevices`	Yes	Graphics devices used for saving selected diagnostic plots.
Base R	`grid`	Yes	Graphics infrastructure used by plotting components.
Data handling	`dplyr`	Yes	Filtering, mutation, grouping, joining, and summary-table construction.
Data handling	`tidyr`	Yes	Data reshaping and construction of target–predictor grids.
Data handling	`purrr`	Yes	Functional iteration across targets, predictors, model fits, diagnostics, and outputs.
Data handling	`tibble`	Yes	Tidy table construction for edges, diagnostics, validation summaries, and compendium outputs.
Data handling	`broom`	Yes	Tidy handling of model-related summaries where needed.
Visualization	`ggplot2`	Yes	Static plots for trajectories, derivatives, jitter diagnostics, posterior summaries, edge consistency, and diagnostics.
Visualization	`scales`	Yes	Color, size, and numerical scaling for publication plots.
Network visualization	`igraph`	Yes	Network object construction and graph layout support.
Network visualization	`visNetwork`	Yes	Interactive browser-based network visualization.
HTML output	`htmltools`	Yes	HTML tag support for interactive network outputs.
HTML output	`htmlwidgets`	Yes	Saving interactive visualization widgets.

Category	Package or tool	Required?	Role in the CIRN script
Bayesian modeling	`brms`	Yes	Bayesian logistic regression interface used for CIRN target-response models.
Bayesian modeling	`cmdstanr`	Yes	CmdStan backend used by `brms` for Stan-based posterior sampling.
Bayesian modeling	`posterior`	Yes	Extraction and manipulation of posterior draws from fitted Bayesian models.
Posterior summaries	`bayestestR`	Yes	Highest density interval and posterior-summary support.
MCMC diagnostics	`bayesplot`	Yes	Posterior and MCMC diagnostic visualization.
Collinearity diagnostics	`car`	Yes	Variance inflation factor diagnostics.
Validation	`pROC`	Optional	AUC computation during optional ground-truth validation.
Excel export	`openxlsx`	Optional but recommended	Multi-sheet Excel export, including `CIRN_Results_Compendium.xlsx` and `CIRN_Publication_Tables.xlsx`.
External backend	`CmdStan`	Yes for Bayesian fitting	Compiles and samples the Stan models generated by `brms`.
System toolchain	C++ compiler / Xcode Command Line Tools on macOS	Yes for Bayesian fitting	Required for Stan model compilation.
System setting	`SDKROOT` on macOS	Yes on configured macOS runs	Checked by the script so Stan compilation fails early and clearly if the SDK path is unavailable.

The following R commands install the main non-base package dependencies:

```
install.packages(c(
  "dplyr", "tidyr", "purrr", "broom", "tibble",
```

```
  "ggplot2", "igraph", "visNetwork", "scales",
  "htmltools", "htmlwidgets", "brms", "posterior",
  "bayestestR", "bayesplot", "car", "pROC", "openxlsx"
))

install.packages(
  "cmdstanr",
  repos = c("https://mc-stan.org/r-packages/", getOption("repos"))
)

cmdstanr::install_cmdstan()
```

The packages `stats`, `utils`, `grDevices`, and `grid` are distributed with base R and normally do not require separate installation. The package `openxlsx` is optional but strongly recommended. If it is unavailable, the script can still save many outputs as CSV files, but the tabbed Excel workbooks used for review and manuscript preparation will not be created. The package `pROC` is used only for optional validation summaries; ordinary CIRN edge inference does not require AUC computation.

On macOS, Stan requires a working C++ toolchain. Users should install the Xcode Command Line Tools before fitting CIRN models. The current script also checks whether `SDKROOT` (a system environment variable that tells R/Stan where the macOS Software Development Kit is located) is visible to R because missing SDK information can cause Stan compilation failures. If `SDKROOT` is not set, the script stops early with an informative message rather than failing later during model compilation.

Practical Requirement. A successful full CIRN run requires both R package availability and a working Stan/CmdStan compilation environment. Package installation alone is not sufficient if CmdStan cannot compile and sample Bayesian models.

C. Input file, output directory, and execution workflow

The current CIRN script is designed to be run as a self-contained analysis workflow. At startup, the script first resolves the analysis directory, denoted internally as `analysis_dir`. When the location of the R script can be detected, `analysis_dir` is set to the folder containing the script. If the script location cannot be resolved, `analysis_dir` defaults to the current R working directory. This design allows relative input paths to be interpreted consistently without requiring the user to manually change the working directory.

The input dataset is read from a CSV file. In the current implementation, the input file can be supplied in three ways, in the following order of priority:

1. Set `input_file` explicitly inside the script. This is the recommended option for final analyses because it leaves a clear record of the dataset used. Example, find the line with `input_file <- "...(your csv file path)..."` the same as below

   ```
   input_file <- "/Users/Giovannie Entero/Desktop/My Dissertation/Dissertation
     Focus/Main Version/0_Predator_Prey_Simulation/Classical_Lotka_Volterra/
                                 Predator_Prey.csv"
   ```

2. Set `input_file = NULL` and pass the CSV file as the first command-line argument, for example:

   ```
   Rscript CIRN_Algorithm.R path/to/data.csv.
   ```

3. Set `input_file = NULL` and place exactly one CSV file in `analysis_dir`. If exactly one CSV file is found, the script uses it automatically.

If `input_file` is an absolute path, the script uses that path directly. If it is a relative path, the script resolves it relative to `analysis_dir`. If no input file is provided and no CSV file is found, the script stops. If no input file is provided and multiple CSV files are found, the script also stops and asks the user to specify the file explicitly. This safeguard reduces the risk of accidentally analyzing the wrong dataset.

After the input file is resolved, the script defines the output directory as

$$\texttt{output_dir} = \texttt{dirname(input_path)}.$$

Thus, generated outputs are saved beside the input dataset rather than in an unrelated working directory. This includes plots, CSV tables, Excel workbooks, publication-ready tables, and the reproducibility file `sessionInfo_CIRN.txt`. Keeping outputs beside the input dataset helps preserve a clear one-to-one relationship between the data file and the corresponding CIRN result files.

The input CSV should contain one time column and numeric state variables. The script reads the file using

$$\texttt{my_data <- read.csv(input_path)}.$$

If `time_col = NULL`, the first column of the CSV is used as the time column. Users should set `time_col` explicitly when the time variable is not the first column. The time column may be numeric, date-based, or date-time based, but derivative estimation ultimately requires a numeric time scale. For reproducibility and interpretability, numeric time indices such as $1, 2, 3, \ldots$ or measured elapsed time are recommended whenever appropriate.

Raw values are preserved when the CSV is imported. The script does not standardize the dataset at import. This is important because the directional response must be constructed from the raw estimated target derivative, not from a standardized derivative. Predictor standardization occurs later, after derivative construction, response encoding, temporal lag alignment, predictor filtering, and complete-row filtering. This ordering preserves the interpretation of $Y_i(t_k) = 1$ as an increasing target response and $Y_i(t_k) = 0$ as a decreasing target response.

The recommended execution workflow is as follows:

1. Prepare a clean CSV file with one time column and numeric state variables.
2. Confirm that the intended time column is first (you may name it as `t`), or set `time_col` explicitly.
3. Set `input_file` explicitly for the final analysis.
4. Edit the `cirn_config` block, especially `targets`, `predictors`, `lag_units`, `representation_mode`, `run_pairwise`, smoothing settings, response tolerance, jitter settings, prior settings, and MCMC settings.
5. Run the script from a fresh R session.
6. Inspect console messages, retained edge tables, model diagnostics, effective-sample-size summaries, VIF/correlation diagnostics, exported figures (some figures are not directly exported, so you may need to save it by yourself), publication-ready tables, and `sessionInfo_CIRN.txt`.

For publication-quality analyses, configuration changes should be made deliberately and recorded. Settings such as the target set, predictor set, lag, smoothing option, response tolerance, adaptive-jitter option, representation mode, and MCMC configuration are part of the analysis design. They should not be treated as cosmetic script options because they can affect the fitted models, retained edges, diagnostics, and interpretation of the resulting CIRN network.

Practical Recommendation. For a final reported analysis, set `input_file`, `time_col`, `targets`, and `predictors` explicitly. This makes the analysis easier to reproduce and prevents accidental changes caused by file order, working-directory changes, or extra CSV files in the analysis folder.

D. Main configuration fields

The `cirn_config` block (find this in the CIRN algorithm R script) is the main user-editable specification for a CIRN run. It should be treated as part of the statistical analysis design, not merely as a set of software options. Several fields determine the scientific question being asked, including which variables may act as targets, which variables may act as predictors, what temporal lag is assumed, which predictor representations are fitted, and whether pairwise or sensitivity analyses are performed.

Internally, the script separates primary inference settings from auxiliary settings. The primary inference settings are passed to `infer_network()`, whereas non-inference settings such as ground-truth validation, sensitivity-analysis controls, and some output controls are handled downstream by the run block. This design keeps the core CIRN inference path focused while still allowing optional validation, robustness assessment, and publication-ready reporting.

Supplementary Table 12: Main configuration fields in the current CIRN implementation.

Category	Setting	Meaning and interpretation
Variable specification	`targets`	Variables whose directional responses $Y_i(t_k)$ are modeled. If `NULL`, all non-time numeric variables are used as targets. For constrained biological or environmental systems, this field should be set explicitly using domain knowledge.
Variable specification	`predictors`	Variables allowed to act as candidate regulators. If `NULL`, all non-time numeric variables are used as predictors. Targets and predictors may differ, allowing expert-constrained candidate graphs.
Inference mode	`representation_mode`	Primary multivariable CIRN mode. `sublevel` fits state, first-derivative, and second-derivative models separately; `all_predictors` fits one joint model containing all eligible representations; `both` fits all four model types.
Inference mode	`pairwise_representation_mode`	Representation mode used for pairwise fits when `run_pairwise = TRUE`. If `NULL`, the script copies `representation_mode`.
Temporal alignment	`lag_units`	Number of time steps by which predictors precede the target response. For example, $\ell = 1$ means predictors at t_{k-1} are used to model $Y_i(t_k)$.

Category	Setting	Meaning and interpretation
Pairwise diagnostics	`run_pairwise`	If `TRUE`, the script runs one-predictor-at-a-time diagnostic CIRN models in addition to the primary multivariable models. Pairwise outputs are stored separately and should not be confused with the primary multivariable network.
Derivative construction	`points_per_interval`	Resolution of the regularized derivative grid. A value of 1 uses one grid interval per observed interval; larger values densify the grid before derivative estimation.
Derivative construction	`spar`	Spline smoothing parameter. If `NULL`, `smooth.spline()` chooses the smoothing level by generalized cross-validation (GCV). Fixed values should be used only when smoothing is part of a planned sensitivity analysis.
Outlier handling	`outlier_method`	Outlier-screening method used before spline smoothing. Current options are `none` and `MAD`.
Outlier handling	`outlier_thresh`	Threshold for MAD-based outlier screening when `outlier_method = "MAD"`.
Outlier handling	`outlier_action`	Action applied to detected outliers when MAD screening is enabled. Options are `keep`, `winsorize`, and `remove`.
Response construction	`response_eps`	Near-zero derivative tolerance used to define the binary target response. Values with $\lvert x_i'(t_k)\rvert \leq \varepsilon$ are removed as near-stationary or numerically ambiguous.
Adaptive jitter	`adaptive_jitter`	If `TRUE`, the script may apply adaptive minimal jitter to the target trajectory when the raw derivative response is one-class or too sparse. If the raw response is adequate, the raw response is retained even when this option is enabled.
Adaptive jitter	`jitter_predictors`	Optional sensitivity mode. If `TRUE`, and adaptive response jitter is triggered, the script can also jitter declared predictor state trajectories and recompute their first and second derivatives before fitting. This should remain `FALSE` for primary reported networks unless explicitly reported as a sensitivity analysis.

Category	Setting	Meaning and interpretation
Adaptive jitter	`jitter_min_class_count`	Minimum number of observations required in each response class after adaptive jitter. The current script uses this threshold to decide whether a two-class response is usable.
Adaptive jitter	`jitter_scale_grid`	Ordered grid of positive scale factors searched from smallest to largest during adaptive jitter. The first scale that yields a usable two-class response is retained.
Adaptive jitter	`jitter_scale_basis`	Basis used to convert the selected jitter scale factor into an actual jitter standard deviation. Options are `state_sd`, `state_range`, `derivative_sd`, `derivative_max_abs`, and `absolute`.
Bayesian sampling	`model_iter`	Total MCMC iterations per chain. This includes warm-up and post-warm-up sampling iterations.
Bayesian sampling	`model_warmup`	Number of warm-up iterations per chain. Warm-up draws are used for sampler adaptation and are not treated as posterior draws for inference.
Bayesian sampling	`model_chains`	Number of MCMC chains used for each Bayesian logistic model.
Bayesian sampling	`model_cores`	Number of CPU cores requested for each fitted Bayesian model, mainly for chain-level parallel execution. For long pairwise runs, modest values such as 1 or 2 are recommended.
Bayesian prior	`prior_mean`	Mean of the Normal prior assigned to regression coefficients.
Bayesian prior	`prior_sd`	Standard deviation of the Normal prior assigned to regression coefficients. This is the standard deviation, not the variance.
Bayesian sampling	`adapt_delta`	Stan NUTS tuning parameter. Larger values, such as 0.99, may help when divergent transitions occur, although they can increase runtime.
Predictive diagnostics	`compute_loo`	If `TRUE`, the script computes optional leave-one-out predictive summaries. LOO is a predictive diagnostic and does not determine edge inclusion.

Category	Setting	Meaning and interpretation
Predictive diagnostics	`loo_moment_match`, `loo_reloo`, `loo_k_threshold`	Advanced LOO settings used when diagnosing problematic Pareto-k behavior. These should usually remain at default values unless LOO diagnostics are being examined carefully.
Ground-truth validation	`run_ground_truth_validation`	If `TRUE`, the script compares inferred state-level edge recovery against a signed reference adjacency matrix. This should be enabled only for simulated or benchmark systems with defensible ground truth.
Ground-truth validation	`true_adj`	Signed reference adjacency matrix used when validation is enabled. Rows correspond to predictors or regulators, columns correspond to targets or responses, and entries are $+1$, -1, or 0.
Sensitivity analysis	`run_sensitivity_analysis`	If `TRUE`, the script reruns CIRN under robustness scenarios. This can be computationally expensive because the full pipeline is repeated.
Sensitivity analysis	`sensitivity_plan`	Optional custom sensitivity plan. If `NULL`, the script can generate a default plan from the sensitivity settings below.
Sensitivity analysis	`sensitivity_inference_scope`	Controls which CIRN inference modes are rerun during sensitivity analysis. This setting determines whether sensitivity runs use the main configuration, only sublevel models, only all-predictors models, both multivariable modes, pairwise-focused summaries, or all supported inference layers.
Sensitivity analysis	`sensitivity_replicates`	Number of repeated runs for stochastic sensitivity scenarios, such as added noise or random row removal.
Sensitivity analysis	`sensitivity_noise_sd_fractions`	Gaussian-noise levels expressed as fractions of each variable's empirical standard deviation.
Sensitivity analysis	`sensitivity_lag_units`	Alternative lag values used in lag-sensitivity runs.
Sensitivity analysis	`sensitivity_downsample_intervals`	Downsampling intervals. For example, a value of 2 keeps every second observation.

Category	Setting	Meaning and interpretation
Sensitivity analysis	`sensitivity_target_sample_sizes`	Target numbers of time points used for sample-size sensitivity runs.
Sensitivity analysis	`sensitivity_missing_fractions`	Fractions of rows randomly removed during missingness or data-sparsity sensitivity runs.
Sensitivity analysis	`sensitivity_save_outputs`	Whether to save sensitivity-analysis outputs as files.
Sensitivity analysis	`sensitivity_show_progress`, `sensitivity_progress_bar`	Progress-display settings for repeated sensitivity runs.
Reproducibility	`seed`	Random seed used for reproducible adaptive jitter, sensitivity scenarios, and Bayesian fitting.
Progress and debugging	`show_progress`, `progress_bar`	User-facing progress messages and progress-bar display during model fitting.
Progress and debugging	`progress_callback`	Optional custom progress-reporting function. This is usually `NULL` for ordinary analyses.
Progress and debugging	`debug`, `assign_debug_to_global`	Troubleshooting settings. These should usually be `FALSE` for final scripted analyses.

Several configuration fields should be interpreted as inferential choices. The fields `targets`, `predictors`, `lag_units`, `response_eps`, and `representation_mode` define the target-response question being asked. The fields `model_iter`, `model_warmup`, `model_chains`, `prior_sd`, and `adapt_delta` define how the Bayesian inference model is estimated. The fields `run_pairwise`, `compute_loo`, `run_ground_truth_validation`, and `run_sensitivity_analysis` control auxiliary analyses and should be reported separately from the primary retained-edge table.

The input file, time column, and output directory are configured outside the `cirn_config` block. Specifically, `input_file` determines the CSV dataset, `time_col` identifies the time variable, and `output_dir` is set automatically to the directory containing the resolved input file. These fields are part of the run setup rather than the target-level CIRN model specification, but they should still be recorded for reproducibility.

Configuration Principle. For a final analysis, the `cirn_config` block should be treated like a preregistered model specification. Changing targets, predictors, lag, smoothing, response tolerance, representation mode, jitter settings, or MCMC settings changes the analysis and may change the retained CIRN network.

E. Running the script

The current CIRN implementation can be run either interactively from R or RStudio, or non-interactively from the command line. For final analyses, a fresh R session is recommended so that package state, random seeds, previous objects, and compiled model artifacts from earlier exploratory runs do not obscure the reproducibility record. The script is designed to execute the full analysis from the input CSV and the `cirn_config` block without requiring manual calls to internal functions.

A typical interactive workflow is:

1. open the R script in a fresh R or RStudio session;
2. set `input_file` to the intended CSV file;
3. set `time_col` explicitly if the time column is not the first column;
4. edit the `cirn_config` block;
5. run the full script from top to bottom;
6. inspect the console messages, exported figures, Excel compendium, publication-ready workbook, and `sessionInfo_CIRN.txt`.

A typical command-line workflow is:

```
Rscript CIRN_Algorithm.R path/to/data.csv.
```

This command-line input is used only when `input_file` is set to `NULL` inside the script. If `input_file` is explicitly set in the script, that explicit file path takes priority. This priority rule is intentional because final analyses should record the exact input dataset rather than depend on command-line state or folder contents.

When the script is executed, it proceeds through the following major stages:

1. resolve the script directory and input file;
2. set the output directory beside the input dataset;
3. load required packages and configure the `cmdstanr` backend for `brms`;
4. check the macOS Stan compilation environment when applicable (for macOS users);
5. read the CSV file and identify the time column;
6. define optional ground-truth input if validation is enabled;
7. read the `cirn_config` block and complete derived settings, such as copying `representation_mode` into `pairwise_representation_mode` when the latter is `NULL`;
8. generate pre-CIRN visual diagnostics for raw trajectories and derivative construction;
9. call `infer_network()` through `do.call()` using the inference-relevant configuration settings;
10. extract retained edges, all fitted coefficients, diagnostics, effective-sample-size summaries, VIF/correlation diagnostics, pairwise outputs, and edge-consistency summaries;
11. run optional ground-truth validation and optional sensitivity analysis when enabled;
12. generate network, edge-consistency, posterior, and MCMC diagnostic figures when fitted models are available;
13. write publication-ready tables, the all-coefficients report, the multi-sheet result compendium, CSV exports, and session information.

The primary inference call is timed using `system.time()` around `infer_network()`. The reported `CIRN inference runtime` therefore measures the main inference call, including derivative construction, response construction, Bayesian fitting, and diagnostics computed inside `infer_network()`. It does not include all downstream plotting, workbook writing, optional validation, or sensitivity-analysis export. Sensitivity runs, when enabled, have their own runtime summaries.

During execution, the script reports model-fitting progress when `show_progress = TRUE`. The progress messages identify the analysis mode, target response, predictor representation, fit status, number of predictors, number of retained edges, and estimated remaining time when available. These messages are useful for distinguishing ordinary long runtime from a stalled or failed run. Long runtime is expected when pairwise mode, all-predictors mode, LOO, or sensitivity analysis is enabled, because these settings increase the number of Bayesian models fitted.

A completed run may still produce no retained edges for some targets or predictor representations. This is not automatically an error. A fitted model can be completed but yield no coefficients whose 95% HDI excludes zero. Conversely, some model configurations may be skipped before fitting if the response remains one-class, if lagging and filtering remove usable rows, if no valid predictors remain, or if model fitting fails numerically. These cases are recorded in the diagnostics sheets and should be reviewed before interpreting the absence of retained edges as absence of regulatory signal.

The main files produced by a completed run may include:

- `CIRN_Results_Compendium.xlsx`, the main multi-sheet result workbook;
- `CIRN_Results_Compendium_CSV/`, the folder containing CSV versions of non-empty result tables;
- `CIRN_Publication_Tables.xlsx`, the two-sheet publication-ready workbook containing edge and diagnostics summaries;
- `CIRN_Publication_Edge_Table.tex`, the LaTeX edge-table output;
- `CIRN_All_Coefficients_Retained_and_Not_Retained.csv`, the coefficient-level report containing retained and non-retained fitted terms;
- `CIRN_Edge_Consistency_Across_Modes.png` and `CIRN_Edge_Consistency_Across_Modes.pdf`, when cross-mode retained edges are available;
- `CIRN_sensitivity_feature_edge_stability.csv`, when sensitivity analysis is enabled and sensitivity CSV outputs are saved;
- network, derivative, posterior, and MCMC diagnostic figures generated by the plotting sections;
- `sessionInfo_CIRN.txt`, the R session record.

After the run completes, the retained edge table should not be inspected in isolation. At minimum, the analyst should review `edges_combined`, `all_coefficients_table`, `diagnostics`, `effective_sample_size`, `vif_group`, `vif_pairs`, `edge_consistency`, and the publication-ready tables. This review is especially important when adaptive jitter, pairwise mode, all-predictors mode, or sensitivity analysis was enabled.

Do Not Interrupt Final Bayesian Runs Prematurely. In pairwise, all-predictors, or sensitivity-analysis settings, the script may fit many Bayesian logistic models. A long run is not necessarily a failed run. Before stopping a final analysis, check whether the console is still reporting target-wise or pairwise model progress.

F. Modular organization of the R implementation

Although the current CIRN implementation is distributed as a single executable R script, it is organized as a modular computational pipeline. The script is not a sequence of unrelated commands. Instead, each major operation is handled by a function or function group, and the final run block calls these components in a fixed order using the user-defined configuration.

The main analytical hierarchy is:

`infer_network()` $\longrightarrow$ `infer_for_target()` $\longrightarrow$ `run_bayes_logit()` (nested).

Here, `infer_network()` coordinates the full analysis across targets, `infer_for_target()` constructs each target-specific directional-response problem, and the nested `run_bayes_logit()` routine fits the Bayesian logistic models and applies the posterior HDI edge-retention rule.

Supplementary Table 13: Modular structure of the current CIRN R implementation.

Module	Representative functions or script components	Role in the workflow
Run setup and reproducibility	`resolve_script_dir()`, `set_cirn_thread_limits()`, `make_target_specific_seed()`, `settings_to_table()`	Resolves the analysis directory, limits low-level numerical thread pools, stabilizes stochastic target-specific operations, and records the run settings.
Input and configuration	`input_file`, `time_col`, `cirn_config`, `cirn_non_inference_config_keys()`	Defines the dataset, time column, target set, predictor set, inference mode, lag, preprocessing settings, Bayesian settings, and optional validation or sensitivity controls.
Derivative construction	`compute_derivatives()`, `compute_grid_derivatives_from_state()`	Sorts and regularizes time, averages duplicate time points when present, applies spline smoothing, and computes state, first-derivative, and second-derivative representations.
Directional-response construction	`classify_directional_response()`, `response_class_counts()`, `adaptive_jitter_directional_response()`	Converts raw target first derivatives into decreasing/increasing classes, removes near-zero derivative cases, records class counts, and applies adaptive minimal response jitter only when needed.

Module	Representative functions or script components	Role in the workflow
Predictor preparation	`standardize_selected_numeric_columns()`, target-level lagging inside `infer_for_target()`	Constructs eligible state, first-derivative, and second-derivative predictors; removes response-defining quantities; temporally lags predictors; filters invalid columns; and standardizes retained predictors after lag alignment.
Collinearity diagnostics	`compute_collinearity_diagnostics()`, `classify_vif()`, `classify_abs_correlation()`	Reports VIF, model-rank information, aliasing indicators, and pairwise predictor correlations for fitted model designs.
Target-level inference	`infer_for_target()` with nested `run_bayes_logit()`	Builds target-specific model-ready data, excludes response-defining quantities, fits representation-specific and all-predictors Bayesian logistic models, computes posterior summaries, and retains coefficients whose 95% HDI excludes zero.
Network-level aggregation	`infer_network()`	Applies target-level inference across declared targets, optionally runs pairwise CIRN fits, aggregates retained edges, stores all fitted coefficients, collects diagnostics, and returns the main structured CIRN result object.
Effective-sample-size audit	`summarize_cirn_effective_sample_size()`	Summarizes model-ready class counts, usable sample size, minority-class counts, predictors per fitted model, minority observations per predictor, class balance, and sample-size flags.
Ground-truth validation	`evaluate_representation_agnostic()`, `metrics_to_long_table()`	Computes signed state-level recovery metrics when a defensible signed reference adjacency matrix is available.

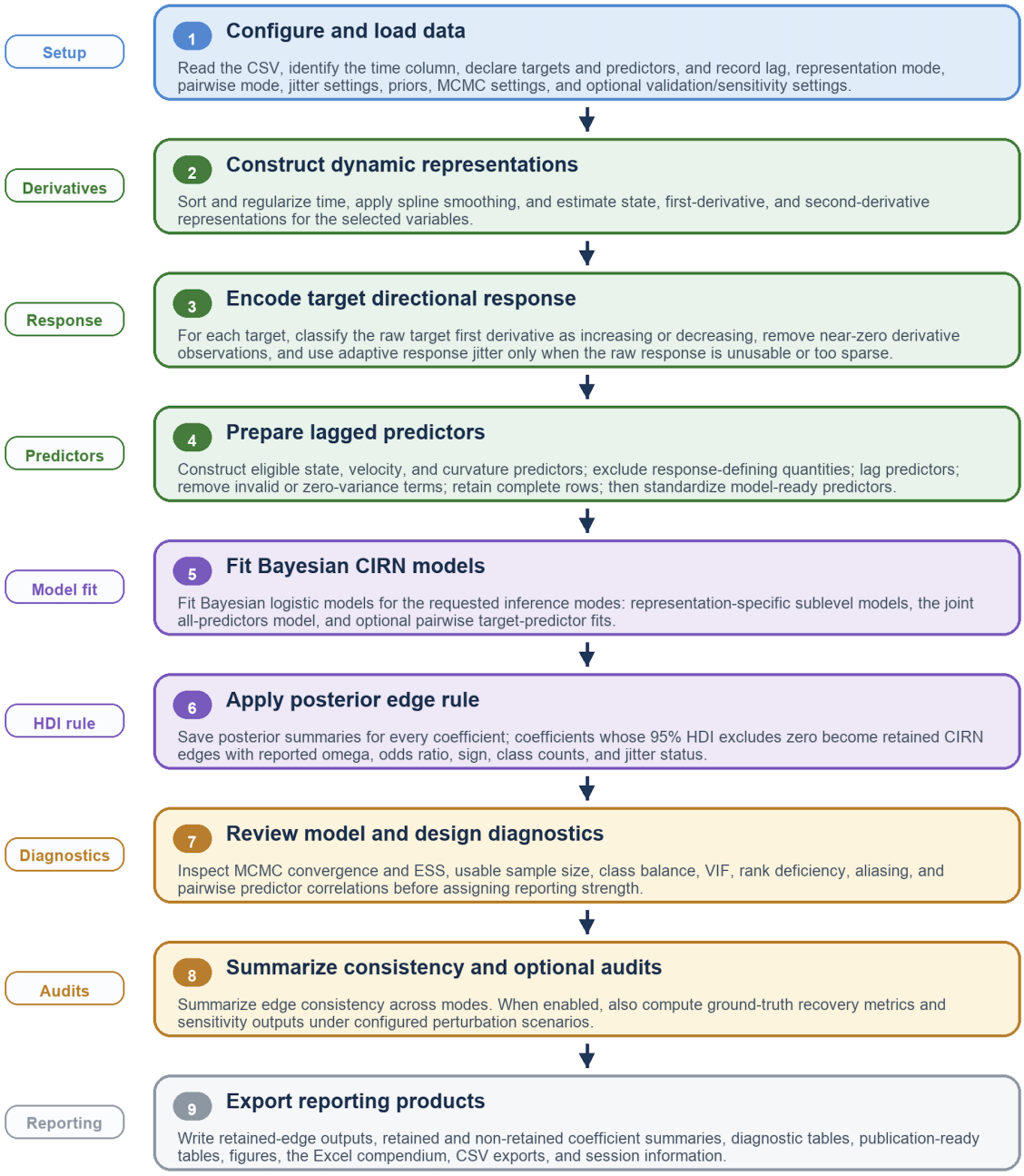
Module	Representative functions or script components	Role in the workflow
Sensitivity analysis	`make_cirn_sensitivity_plan()`, `prepare_cirn_sensitivity_dataset()`, `resolve_cirn_sensitivity_scope()`, `run_cirn_sensitivity_analysis()`, `summarize_sensitivity_edge_stability()`, `summarize_cirn_sensitivity_feature_edge_stability()`	Reruns CIRN under controlled perturbations, including added noise, alternative lags, downsampling, reduced sample sizes, and row missingness; resolves which inference modes are rerun; and summarizes both state-level and feature-level edge stability.
Visualization	`plot_raw_data_all_vars()`, `plot_derivative_example()`, `plot_adaptive_jitter_diagnostic()`, `plot_network_main()`, `plot_cirn_panels()`, `plot_edge_consistency()`, `plot_all_posterior_effects()`, `plot_diagnostics_summary()`, `plot_all_traces_with_burnin()`	Produces raw-data, derivative, adaptive-jitter, network, representation-panel, edge-consistency, posterior-density, diagnostic-summary, and MCMC trace visualizations.
Publication-ready reporting	`summarize_edge_consistency()`, `make_publication_edge_table()`, `make_publication_diagnostics_table()`, `make_all_coefficients_report_table()`, `write_publication_edge_table_latex()`, `write_publication_tables_workbook()`	Creates the reader-facing edge table, model-level diagnostics table, all-coefficients retained/not-retained report, LaTeX edge table, and two-sheet publication workbook.
Compendium and export	`write_cirn_result_compendium()`, `sanitize_output_name()`, `sanitize_excel_sheet_name()`, final run-block export commands	Saves CSV outputs, the multi-sheet Excel result compendium, publication-ready workbook, all-coefficients CSV, figures, and `sessionInfo_CIRN.txt`.

The modular organization is important for interpretation. Derivative construction determines the state, velocity, and curvature representations available to the model. Directional-response construction determines the binary target outcome. Predictor preparation determines which lagged terms enter each fitted model. Bayesian fitting determines posterior coefficient summaries. Edge aggregation determines the reported network. Diagnostics, validation, sensitivity analysis, and publication-ready reporting then provide the evidence needed to judge whether retained edges should be reported strongly, cautiously, or only as exploratory results.

This structure also explains why different output sheets have different row counts. The retained edge tables are coefficient-level outputs. The diagnostics sheets are fitted-model-level outputs. The VIF and pairwise-correlation sheets are predictor-design outputs. The effective-sample-size sheet is a model-readiness audit. The edge-consistency sheet summarizes cross-mode evidence for term–target pairs. Validation and sensitivity sheets are optional analysis-level summaries. These outputs are connected conceptually, but they are not all indexed by the same unit of observation.

The table lists major modules and representative functions rather than every helper func-

tion defined in the script. The implementation also contains lower-level helper routines for seed restoration, empty-result construction, progress reporting, HDI calculation, predictor validation, node-label formatting, plot theming, LaTeX escaping, Excel-sheet preparation, and output-name sanitization. These helpers support the major modules listed above but are not intended as separate user-facing entry points. Below is the workflow of the CIRN R implementation.



Supplementary Figure 6: Workflow of the current CIRN R implementation. The script executes CIRN as an ordered computational pipeline. The analysis begins by loading the input time series and user-specified configuration, then constructs smoothed state, first-derivative, and second-derivative representations. Target-wise directional responses are encoded from the raw target derivative, with adaptive response jitter used only when the raw response is unusable or too sparse. Eligible predictors are lagged, filtered, completed, and standardized before Bayesian logistic CIRN models are fitted under the selected representation and pairwise settings. Posterior coefficient summaries are evaluated using the 95% HDI rule, followed by diagnostic review, optional consistency or robustness audits, and export of reporting products.

Supplementary Figure 6 summarizes the execution order of the current CIRN implementation. The two boxes below complement the workflow by separating the main files produced by the script from the indexing rule needed to interpret them correctly. This distinction is important because the output compendium contains coefficient-level, fitted-model-level, predictor-design-level, and optional analysis-level summaries, which are related but not expected to have identical row counts.

Main Outputs to Report or Inspect

Edge/Coefficient Outputs

- `edges_combined`
- `all_coefficients_combined`
- `CIRN_Publication_Edge_Table.tex`
- `CIRN_All_Coefficients_Retained_and_Not_Retained.csv`

Consistency and Optional Audits

- `edge_consistency`
- `CIRN_Edge_Consistency_Across_Modes.png/.pdf`
- `evaluation_metrics`, if validation is enabled
- Sensitivity result tables, if sensitivity analysis is enabled

Model/Design Diagnostics

- `diagnostics` and `pairwise_diagnostics`
- `effective_sample_size`
- `vif_group` / `vif_pairs`
- `pairwise_vif_group` / `pairwise_vif_pairs`

Publication and Reproducibility Files

- `CIRN_Publication_Tables.xlsx`
- `CIRN_Results_Compendium.xlsx`
- `CIRN_Results_Compendium_CSV/`
- Network, posterior, jitter, and MCMC figures
- `sessionInfo_CIRN.txt`

Indexing Reminder

Edge and coefficient tables are coefficient-level outputs; diagnostics and effective-sample-size tables are fitted-model-level outputs; VIF and correlation tables describe the predictor design. Row counts therefore do not need to match.

Why The Modular Structure Matters. Each output table corresponds to a specific level of the pipeline. Retained edges are coefficient-level outputs, diagnostics are fitted-model-level outputs, VIF and pairwise-correlation tables are predictor-design outputs, and validation or sensitivity summaries are optional analysis-level outputs. These tables should not be forced into one-to-one row matching unless they share the same indexing level.

G. Minimum reporting checklist

A reported CIRN analysis should be reproducible from four linked records: the input time-series dataset, the user-edited `cirn_config` block, the saved output compendium, and the recorded R/session environment. The checklist below is intended as a practical reporting guide. Not every item must appear in the main text, but each item should be recoverable from the manuscript, supplementary information, archived script, output tables, or result files.

The checklist also helps prevent a common source of confusion: CIRN outputs are not all indexed by the same unit. Retained-edge and coefficient tables are coefficient-level outputs; diagnostic and effective-sample-size tables are fitted-model-level outputs; VIF and correlation tables describe the predictor design; and validation or sensitivity outputs are optional analysis-level summaries. These outputs should be interpreted together, but their row counts are not expected to match.

Supplementary Table 14: Minimum reporting checklist for a CIRN analysis.

Reporting item	Minimum information to report or archive	Primary source
System and dataset	Identify the biological, ecological, environmental, simulated, or benchmark system; input dataset or data-generation procedure; time variable; observed state variables; and sampling scale or time units when meaningful.	Input CSV; Methods; `settings`.
Target and predictor sets	Report the declared target set $\mathcal{T}$, declared predictor set $\mathcal{P}$, and any scientific exclusions from the candidate predictor space.	`cirn_config`; `settings`.
Temporal design	Report the lag ℓ, the derivative-grid setting, spline smoothing setting, and outlier-screening/action settings.	`cirn_config`; `settings`.
Derivative construction	State that state, first-derivative, and second-derivative representations were constructed after smoothing and regularized-grid derivative estimation. Report deviations from the default derivative settings.	Methods; `settings`.
Directional-response rule	Define class 0 and class 1. Class 0 denotes decreasing target response and class 1 denotes increasing target response after applying the derivative tolerance ε. State that near-zero derivative observations are removed.	Methods; `diagnostics`; edge tables.
Adaptive response jitter	State whether adaptive jitter was enabled and whether it was actually used. If used, report response source, response trigger, jitter scale factor, jitter scale, jitter basis, and jitter-qualified interpretation.	`diagnostics`; edge tables; jitter figures.
Predictor representation	Preserve whether each retained term came from state, first-derivative, second-derivative, or all-predictors fitting. Derivative-supported edges should be described by their representation, not collapsed into state-only language.	`edges_combined`; `all_coefficients_combined`.
Inference mode	Report `representation_mode`, whether `run_pairwise` was enabled, and the `pairwise_representation_mode` if pairwise fitting was used.	`cirn_config`; `settings`; edge tables.

Reporting item	**Minimum information to report or archive**	**Primary source**
Bayesian model settings	Report the Bernoulli-logit model, prior mean, prior standard deviation, MCMC iterations, warm-up, chains, cores, `adapt_delta`, random seed, and backend/software environment when relevant.	`settings`; `sessionInfo_CIRN.txt`.
Posterior edge rule	State that coefficients are retained when the 95% HDI excludes zero. Report ω, odds ratio $\exp(\omega)$, 95% HDI, and activation/inhibition sign convention.	`edges_combined`; `all_coefficients_combined`.
Retained-edge table	For manuscript tables, report the CIRN edge, analysis mode, predictor set, ω, odds ratio, 95% HDI, regulation type, class counts, support, and jitter status.	`publication_edge_table`; `CIRN_Publication_Tables.xlsx`.
Non-retained coefficients	Preserve non-retained coefficient summaries when transparency or reviewer audit is needed. These values should not be interpreted as retained regulatory hypotheses.	`all_coefficients_combined`; `CIRN_All_Coefficients_Retained_and_Not_Retained.csv`.
Model diagnostics	Report model status, maximum $\hat{R}$, minimum ESS ratio, usable sample size, class balance, number of predictors, and LOO summaries if computed.	`diagnostics`; `pairwise_diagnostics`; `effective_sample_size`.
Collinearity diagnostics	Report VIF status, rank deficiency, aliasing, and high pairwise predictor correlations, especially for all-predictors models.	`vif_group`; `vif_pairs`; pairwise VIF sheets.
Cross-mode consistency	State whether the same term–target pair is supported in pairwise, sublevel, and/or all-predictors outputs. Note any mode-specific support or sign conflicts.	`edge_consistency`; edge-consistency figure.
Ground-truth validation	If a signed reference network is available, report recovery metrics. If no defensible reference graph exists, state that ground-truth validation was not performed.	`evaluation_metrics`; `settings`.

Reporting item	Minimum information to report or archive	Primary source
Sensitivity analysis	State whether sensitivity analysis was enabled. If enabled, report the scenarios, replicates, `sensitivity_inference_scope`, state-level edge stability, feature-level edge stability, and whether `pairwise_only` or `everything` mode was used; if disabled, say so rather than implying robustness was tested.	`sensitivity_plan`; `sensitivity_runs`; `sensitivity_edge_stability`; `sensitivity_feature_edge_stability`.
Publication-ready outputs	Archive the publication edge table, publication diagnostic table, all-coefficients report, figures, and result compendium used to prepare the manuscript.	`CIRN_Publication_Tables.xlsx`; `CIRN_Results_Compendium.xlsx`.
Reproducibility record	Preserve the exact R script, input data, configuration block, random seed, output directory, result compendium, CSV export folder, figure files, and session information.	Script file; `CIRN_Results_Compendium_CSV/`; `sessionInfo_CIRN.txt`.

For the main article, the retained-edge table and model-diagnostic table should remain separate. The retained-edge table summarizes posterior-supported coefficients, whereas the diagnostic table summarizes fitted model configurations. A single fitted model can produce no retained edges, one retained edge, or several retained edges; therefore, merging edge-level and model-level diagnostics row by row can create misleading one-to-one interpretations.

Minimum Reporting Principle. A retained CIRN edge should never be reported without its modeling context. At minimum, the reader should know the predictor representation, target directional response, inference mode, posterior mean coefficient, 95% HDI, regulation sign, response class counts, jitter status, and whether diagnostics support routine or cautious interpretation.

XIV. INTERPRETIVE CAUTIONS

The CIRN output should be interpreted as a structured, uncertainty-aware summary of temporally ordered regulatory hypotheses. Each retained edge arises from a fitted Bayesian logistic model in which lagged state, first-derivative, or second-derivative predictor representations are used to explain the binary directional response of a target variable at a later time point. Thus, a retained edge indicates that the corresponding lagged predictor representation has posterior support for shifting the probability that the target is increasing, after applying the specified response rule, lag alignment, predictor filtering, and model configuration.

This interpretation is deliberately more specific than an ordinary undirected dependence statement, because the predictor precedes the target response in the fitted design and the posterior coefficient gives a signed effect on the target-response scale. At the same time, it is deliberately more cautious than a mechanistic or interventional claim. A retained CIRN edge does not, by itself,

prove that an experimental intervention on the source variable would change the target, identify a direct molecular or ecological mechanism, estimate a physical interaction parameter, or recover the full governing equation of the system. Instead, it should be read as a posterior-supported, directed, signed, uncertainty-aware, and representation-specific temporal-response signal that can guide interpretation, comparison across modes, and follow-up hypothesis testing. Indeed, CIRN can aid hypothesis generation, especially for unexplored, underexplored, and even well-studied systems.

The cautions below define and guide how CIRN results should be read and reported. They are especially important when moving from the full output compendium to a compact manuscript table, where it is easy to lose essential context such as predictor representation, inference mode, response construction, jitter status, class balance, collinearity diagnostics, and MCMC reliability.

Supplementary Table 15: Interpretive cautions for CIRN outputs.

Caution	**Meaning in CIRN**	**Recommended reporting implication**
Edges are regulatory hypotheses, not proof of mechanism or interventional causality.	A retained edge means that a lagged predictor term credibly shifts the posterior probability of target increase under the fitted Bayesian logistic model. The edge is directional in time and signed in the target-response scale, but it is not direct experimental proof of mechanism.	Report retained edges as CIRN-inferred regulatory hypotheses or posterior-supported temporal-response signals. Avoid language implying proven intervention, direct binding, or complete causal identification unless supported by independent evidence.
The sign is on the target-response scale.	If $\omega > 0$, larger values of the lagged predictor increase $\Pr\{Y_i(t_k) = 1\}$, the probability that the target is increasing. If $\omega < 0$, larger values of the lagged predictor decrease that probability. This is not always identical to saying that the source increases or decreases the target's absolute abundance.	Define activation and inhibition explicitly: activation means higher posterior probability of target increase; inhibition means lower posterior probability of target increase.
Derivative quality matters.	First- and second-derivative estimates depend on smoothing, sampling frequency, noise level, and trajectory resolution. Derivative-based predictors and derivative-defined responses can become unstable when the time series is sparse, noisy, or poorly smoothed.	Report smoothing, derivative-grid settings, response tolerance, and derivative-sensitive diagnostics. Interpret first- and second-derivative edges cautiously when sampling is sparse or noisy.

Caution	**Meaning in CIRN**	**Recommended reporting implication**
Lag choice matters.	CIRN aligns predictors at $t_{k-\ell}$ with the target directional response at t_k. Changing ℓ changes the temporal question being asked and can change which edges are retained, especially in delayed, oscillatory, or phase-shifted systems.	Justify the selected lag scientifically. When lag uncertainty is important, report lag-sensitivity results rather than treating one lag as universally definitive.
Near-stationary observations are excluded.	Observations with target derivative magnitude within the tolerance ε are removed before binary response modeling because they are numerically ambiguous as increasing or decreasing cases.	Report ε and the resulting class counts. Do not compare raw row counts directly with model-ready response counts.
Class balance affects interpretability.	Logistic models need usable increasing and decreasing response classes. A model with a very small minority class may fit but can yield unstable posterior summaries, especially when many predictors are included.	Interpret edges cautiously when the minority class is small, when usable sample size is limited, or when minority observations per predictor are low.
Collinearity can change signs and magnitudes.	State, velocity, and curvature predictors may be strongly correlated. In all-predictors models, multiple related representations compete in the same design matrix. High VIF, rank deficiency, aliasing, or near-perfect pairwise correlations can make coefficient signs and HDIs sensitive to model specification.	Review VIF, rank, aliasing, and pairwise-correlation diagnostics before emphasizing signs or effect sizes. Label such results as conditional or mode-sensitive when needed.
Pairwise and multivariable modes answer different questions.	Pairwise mode fits one predictor at a time and is useful for detecting marginal temporal-response information. Multivariable sublevel models estimate support after conditioning on other eligible predictors within the same representation. The all-predictors model estimates support after state, velocity, and curvature terms compete jointly.	Do not treat disagreement across modes as an automatic error. Use cross-mode consistency to distinguish robust, conditional, and mode-specific support.

Caution	**Meaning in CIRN**	**Recommended reporting implication**
All-predictors mode is stringent, not automatically superior.	The all-predictors model can provide strong conditional evidence, but it is more sensitive to sample size, collinearity, derivative noise, and weak identifiability because many related terms compete simultaneously.	A retained all-predictors edge is useful evidence, but a non-retained all-predictors term does not automatically invalidate representation-specific or pairwise support.
Adaptive response jitter must be disclosed.	Adaptive jitter is used only when the raw target derivative response is one-class, near one-class, or too sparse. The script jitters the target trajectory, recomputes the derivative, and reapplies the same response rule; it does not randomly flip class labels.	Label jitter-supported edges clearly. Use wording such as "jitter-qualified CIRN-inferred regulatory signal" when `jitter_used = TRUE`.
Optional predictor jitter is a sensitivity mode.	Predictor jitter is distinct from adaptive response jitter. In the current script, predictor jitter is optional and should not be treated as part of the primary network unless explicitly enabled and justified.	Keep primary raw-predictor results separate from predictor-jitter sensitivity results.
Output sheets have different units of analysis.	Retained-edge rows, all-coefficient rows, model-diagnostic rows, VIF rows, pairwise-correlation rows, validation rows, and sensitivity rows are indexed differently.	Do not force row-by-row matching across sheets. Join or compare outputs conceptually by target, predictor set, analysis mode, and term only when those identifiers are shared.
Representation provenance should be preserved.	A state-supported edge, first-derivative-supported edge, second-derivative-supported edge, and all-predictors retained term carry different temporal information.	Report the predictor representation explicitly. Avoid collapsing state, velocity, and curvature support into a single source-target claim without noting which representation supported the edge.
Absence of a retained edge is not proof of absence.	A term may fail the HDI rule because the effect is weak, the sample is small, the classes are imbalanced, predictors are collinear, the lag is misspecified, or the representation is not informative under the chosen model.	Phrase non-retention as lack of posterior support under the fitted configuration, not as definitive evidence that no regulation exists.

Caution	Meaning in CIRN	Recommended reporting implication
Empirical systems may change through time.	CIRN assumes that the fitted time window can be summarized by the modeled temporal-response relationships. In real biological or environmental systems, regulatory architecture may rewire across conditions, developmental stages, interventions, or time periods.	Interpret empirical networks as window- and context-specific unless stability across time windows or conditions has been tested.
Ground-truth metrics are context-dependent.	Validation metrics are meaningful when a defensible signed reference network exists, such as in simulations or curated benchmark systems. They can be misleading when the empirical reference graph is incomplete or uncertain.	Use validation metrics for benchmark systems. For empirical systems, emphasize uncertainty-aware hypothesis generation and diagnostic transparency.

Recommended Interpretation Language. A retained CIRN edge should be described as a directed, signed, posterior-supported temporal-response regulatory hypothesis. For example, write: "the lagged first derivative of X supported a higher posterior probability that Y increases," rather than only "X activates Y," unless we explicitly define what activate or inhibit means. This keeps the interpretation faithful to the predictor representation, temporal lag, and target-response scale used by CIRN.

XV. CONTRIBUTION AND INFERENTIAL POSITION OF CIRN

What CIRN Contributes and Where it Sits

CIRN is best understood as a temporal-response inference framework for reconstructing directed, signed, and uncertainty-aware regulatory hypotheses from single or multivariate time-series data. It occupies a distinct inferential position between static dependence methods and full equation-discovery approaches. Unlike correlation-based summaries, CIRN models a temporally ordered target response and reports whether lagged predictor representations support target increase or target decrease. Unlike full ODE or equation-fitting methods, CIRN does not require the analyst to specify the complete functional form of the governing dynamics. Instead, it uses state, velocity, and curvature representations to capture complementary levels of temporal information and evaluates them through Bayesian posterior uncertainty.

The strength of CIRN lies in this middle ground: it is information-rich and interpretable enough to produce readable regulatory hypotheses, flexible enough to be applied across deterministic, stochastic, oscillatory, simulated biological, and empirical systems, and transparent enough to expose uncertainty, class balance, collinearity, jitter status, and cross-mode support.

At the conceptual level, the framework is deliberately straightforward. Its target response is binary and its core inferential engine is Bayesian logistic regression, a familiar generalized linear model. Users can therefore relate coefficient signs, odds ratios, and posterior intervals directly to target increase or decrease without first learning a specialized black-box architecture. In practical terms, the learning curve for understanding the core logic of CIRN is not that steep, particularly for users who are already familiar with logistic regression. This accessibility does not remove the need to understand derivative estimation, temporal alignment, posterior computation, and model diagnostics, but it makes the main inferential logic approachable to a broad range of users.

A further practical advantage of CIRN is evidential traceability. The framework does not merely return a network graph: each retained edge can be traced to its specific predictor representation—state, first derivative, or second derivative—together with its temporal lag, model context, coefficient direction and magnitude, 95% HDI, and associated diagnostic record. Analysts can therefore ask not only whether a candidate interaction was retained, but also which aspect of the observed dynamics supports it, under which modeling assumptions it appears, and whether it recurs across inference modes. For research teams that possess time-series observations but do not yet have a complete mechanistic model, CIRN offers an accessible route from observed trajectories to transparent, uncertainty-qualified, and experimentally testable regulatory hypotheses.

CIRN is also intended to support an iterative exploratory workflow rather than a single, closed analysis. As domain expertise, prior knowledge, or new external information becomes available, analysts may refine scientifically admissible predictor sets, temporal lags, prior distributions, exclusion rules, preprocessing decisions, sensitivity scenarios, or topology-informed extensions, and then rerun the analysis transparently. Patterns recognized during exploratory data analysis—such as oscillation, delayed co-movement, regime changes, missingness, or scale differences—can likewise motivate better-informed analytical choices. Every refinement should be documented and subjected to diagnostic and sensitivity checks. When the same dataset is used both to suggest and to evaluate a refinement, the resulting support remains exploratory and should not be presented as independent confirmation.

Given the inherent uncertainty and possible non-identifiability of regulatory-network inference from observational time series, CIRN should not be treated as universally superior to other available methods or as the only analysis required. Whenever feasible, its findings should be compared with results from complementary algorithms that rely on different model structures or assumptions. Agreement among CIRN modes and across genuinely distinct methods provides convergent evidence and can increase confidence in a candidate interaction, whereas disagreement identifies edges whose interpretation depends on the analytical approach and therefore requires closer investigation. Cross-method agreement is not proof of a true regulatory relation, particularly when the methods share the same data limitations or related assumptions, but it is stronger evidence than an unsupported result from one method alone.

Thus, CIRN should not be viewed as a replacement for mechanistic modeling, intervention-based validation, equation discovery, or complementary network-inference algorithms. Its value is that it provides a principled, comparatively simple, and reproducible evidential layer for identifying which lagged dynamical representations support directed and signed

target responses, thereby guiding expert interpretation, benchmark comparison, method triangulation, and follow-up experimental or mechanistic investigation.

XVI. PRACTICAL END-TO-END CIRN WORKFLOW

The steps below summarize how a typical CIRN analysis moves from an organized project folder to reportable results. This workflow is intended for users running the current R script directly. It is a practical execution guide, not a separate inference rule; the statistical definition of CIRN remains the directional-response Bayesian logistic model and the 95% HDI edge-retention rule described in the earlier sections.

1. **Create one folder for one analysis.** Start with a dedicated folder for the system or dataset being analyzed. Place the input CSV file, the `CIRN_Algorithm.R` script, and any supporting notes in that folder. If the analysis uses a known signed reference network, keep the ground-truth adjacency file or data-generation notes in the same folder. This prevents outputs from different datasets, parameter settings, or sensitivity runs from being mixed.
2. **Prepare the input CSV file.** The file should contain one time column and numeric state-variable columns. The time column should be the first column (you may name it as `t`) unless `time_col` is set explicitly in the script. Use short and readable variable names, because the same names will appear in retained-edge tables, diagnostic sheets, and network figures.
3. **Look at the raw trajectories first.** Before running CIRN inference, plot or inspect the observed variables. Check for missing values, duplicate time points, extreme outliers, nearly constant variables, monotonic target trajectories, irregular sampling, or obvious data-entry errors. These issues can affect spline smoothing, derivative estimation, response construction, class balance, and the final interpretation of retained edges. The script also generates raw-data and derivative-related visual outputs that can be used as preprocessing checks.
4. **Choose the target and predictor variables.** Set the `targets` and `predictors` fields in the `cirn_config` block using the study question and relevant domain knowledge. Targets are the variables whose increasing/decreasing responses will be modeled. Predictors are the variables allowed to contribute candidate regulatory information. The target and predictor sets can be identical when all measured system variables are considered admissible as both responses and candidate regulators, but they can also differ when biological, ecological, environmental, or experimental knowledge suggests that only selected variables should be modeled as targets or allowed as predictors. If `targets = NULL`, the script uses all non-time numeric variables as targets. If `predictors = NULL`, the script uses all non-time numeric variables as candidate predictors. The time column should not be included in either set. These choices are recorded in the `settings` sheet of `CIRN_Results_Compendium.xlsx`.
5. **Set the time-lag and derivative settings.** Set `lag_units`, `points_per_interval`, `spar`, `outlier_method`, `outlier_action`, and `response_eps`. The lag ℓ, specified through `lag_units`, should match the research question, sampling frequency, and plausible response delay of the system being studied; for example, $\ell = 1$ asks whether predictor information at the immediately preceding time step supports the target's current increasing/decreasing response, whereas larger lags ask whether earlier predictor states or derivative patterns support later target responses. The `response_eps` setting is the tolerance used to remove near-zero raw target derivatives before fitting; it is not a p-value, posterior probability threshold, or statistical significance cutoff, but a small interval around zero within which the target derivative is treated as too close to stationary

or too numerically ambiguous to label confidently as increasing or decreasing. With the default `response_eps` $= 10^{-6}$, this step mainly acts as a numerical safeguard; larger values remove a wider band of near-stationary observations, which can reduce ambiguous labels but may also reduce usable sample size and alter class balance. These choices should be reported because changing the lag, smoothing, outlier handling, derivative-grid resolution, or response tolerance can change the model-ready data, response class balance, and retained network.

6. **Choose which CIRN models to fit.** Set the `representation_mode` to either `sublevel`, `all_predictors`, or `both` for the primary multivariable CIRN analysis. The `sublevel` mode fits three representation-specific multivariable models for each target: one using state predictors, one using first-derivative predictors, and one using second-derivative predictors. This mode is useful for identifying which type of temporal information supports a retained edge. The `all_predictors` mode fits one joint multivariable model per target using all eligible state, first-derivative, and second-derivative predictors together; this is useful for assessing whether a term remains supported after competing with other representations, but it can be more sensitive to collinearity and usable sample size. The `both` mode runs the sublevel and all-predictors models, which makes it computationally expensive. If `run_pairwise = TRUE`, the script also runs pairwise CIRN models, in which one predictor variable is evaluated at a time for each target, including their respective temporal abstractions, which makes this option more computationally expensive. The pairwise layer uses `pairwise_representation_mode`; if this is `NULL`, the script copies the primary `representation_mode`. Pairwise outputs are useful for marginal screening, sensitivity to predictor inclusion, and cross-mode comparison, but they should not be interpreted as equivalent to multivariable conditioning. The resulting retained edges are stored in `edges`, `pairwise_edges`, and `edges_combined`.
7. **Decide whether adaptive jitter and optional analyses are needed.** Keeping `adaptive_jitter = TRUE` is useful when some target trajectories may be monotonic or nearly one-class after derivative encoding. If adaptive jitter is used, it is recorded in `diagnostics`, `edges_combined`, and the adaptive-jitter diagnostic figures. Keep `jitter_predictors = FALSE` for primary analyses unless predictor-jitter sensitivity is intentionally being studied. Enable ground-truth validation only for simulated or benchmark systems with a known signed reference network. Enable sensitivity analysis only when robustness scenarios are intentionally being run.
8. **Set the Bayesian computation options.** Record `model_iter`, `model_warmup`, `model_chains`, `model_cores`, `prior_mean`, `prior_sd`, `adapt_delta`, and `seed`. These settings define how the Bayesian logistic CIRN models are sampled and regularized. The script fits each model using `brms` with the `cmdstanr` backend, a Bernoulli likelihood, and a logit link. The total number of saved posterior draws per completed model is `model_chains` $\times$ (`model_iter` $-$ `model_warmup`), because warm-up iterations are used for sampler adaptation and are not retained for posterior summaries. The `model_chains` setting controls the number of independent MCMC chains, while `model_cores` controls how many CPU cores are requested for running chains in parallel within a fitted model; it does not mean that all target-wise or pairwise models are launched simultaneously. The `prior_mean` and `prior_sd` settings define the normal prior used for regression coefficients, with `prior_sd` interpreted as the standard deviation rather than the variance. The `adapt_delta` setting controls the target acceptance behavior of the Stan NUTS sampler; increasing it can help when divergent transitions occur, but usually increases runtime. The `seed` stabilizes stochastic components of the analysis, including posterior sampling and target-specific jitter pathways when they are invoked. For final reported analyses, these settings should be fixed before running the script, recorded in the `settings` sheet, and interpreted together with `sessionInfo_CIRN.txt`.

9. **Run the script in a clean R session.** Run `CIRN_Algorithm.R` after confirming the input file and configuration. It is recommended to run it line by line, or function by function, or part by part, rather than simultaneously. While the script runs, check the console for messages about skipped targets, failed fits, single-class responses, adaptive jitter, retained edges, and runtime warnings. The main runtime summary is saved in `cirn_summary`.
10. **Start with the retained-edge table.** After the run finishes, first inspect `edges_combined`. This is the most convenient retained-edge sheet because it combines multivariable and pairwise retained edges. For each row, check the source term, target response, `analysis_mode`, `predictor_set`, ω, odds ratio, 95% HDI, regulation type, class counts, and jitter status. This sheet is the main source for the publication edge table.
11. **Inspect retained and non-retained coefficients.** Use `all_coefficients_combined` and `CIRN_All_Coefficients_Retained_and_Not_Retained.csv` when the full coefficient record is needed. These outputs include both retained and non-retained coefficients, allowing users to distinguish posterior-supported edges from coefficients whose HDI crossed zero. The same information can also be accessed directly inside RStudio through `all_coefficients_combined` or the formatted object `all_coefficients_table`, without relying on the exported CSV file.
12. **Check whether the fitted models are reliable.** After running the algorithm, inspect `diagnostics`, `pairwise_diagnostics`, and `effective_sample_size` before assigning strong interpretation to retained edges. These outputs are fitted-model-level summaries, not edge-level summaries, and they report whether each requested model was completed, skipped, or failed; whether the response was raw or adaptive-jitter based; how many usable observations remained after response construction, lagging, predictor filtering, and complete-row filtering; how balanced the decreasing and increasing response classes were; how many predictors entered the fitted model; and whether posterior sampling was acceptable based on maximum $\hat{R}$ and minimum ESS ratio. When LOO is computed, its summaries can help compare predictive adequacy across fitted model specifications, but LOO does not determine edge inclusion. Retained edges from models with failed or skipped status, poor convergence, low sampling efficiency, severe class imbalance, very small model-ready sample size, or very low minority-class support should be reported cautiously rather than as strong CIRN-inferred regulatory hypotheses.
13. **Check whether predictors were too similar to each other.** Inspect `vif_group`, `vif_pairs`, `pairwise_vif_group`, and `pairwise_vif_pairs`. These sheets help identify high collinearity, rank deficiency, aliasing, and near-redundant predictor pairs. This step is especially important for all-predictors models, where state, velocity, and curvature terms compete in the same fitted model.
14. **Check whether edges are consistent across modes.** Inspect `edge_consistency` and the file `CIRN_Edge_Consistency_Across_Modes.png/.pdf`. These outputs summarize whether the same term–target pair appears in pairwise, sublevel, and/or all-predictors results. Cross-mode consistency is useful for distinguishing robust signals from mode-specific or conditional results.
15. **Review optional validation and sensitivity outputs.** If ground-truth validation was enabled, inspect `evaluation_metrics`. If sensitivity analysis was enabled (this increases runtime), `sensitivity_plan`, `sensitivity_runs`, `sensitivity_edges`, `sensitivity_diagnostics`, `sensitivity_effective_ sample_size`, `sensitivity_edge_stability`, `sensitivity_feature_edge_stability`, and `sensitivity_ground_truth_metrics` (when available) must be thoroughly inspected. These outputs should be used only when the corresponding optional analysis was intentionally enabled.

16. **Use the publication-ready tables for reporting.** Use `CIRN_Publication_Tables.xlsx` for the two reader-facing manuscript tables: `Table1_Edges` and `Table2_Diagnostics`. The first sheet summarizes retained edges; the second summarizes fitted-model diagnostics. Keeping these tables separate avoids confusing edge-level rows with model-level rows.
17. **Use the full compendium as the audit trail.** Use `CIRN_Results_Compendium.xlsx` and `CIRN_Results_Compendium_CSV/` as the complete tabular record of the run. The Excel workbook is convenient for reading across sheets, whereas the CSV folder is useful for reproducible downstream analysis.
18. **Use figures as visual checks, not replacements for diagnostics.** Network figures, posterior-effect plots, edge-consistency plots, adaptive-jitter diagnostics, and MCMC diagnostic plots are useful for interpretation and communication. However, they should be read together with the corresponding tables and diagnostic sheets. A visual network alone is not sufficient evidence for reporting a retained edge.
19. **Write the results with the diagnostic context preserved.** Report retained edges as directed, signed, posterior-supported temporal-response regulatory hypotheses. Include the predictor representation, inference mode, ω, odds ratio, HDI bounds, class counts, jitter status, and any diagnostic qualification. Avoid claiming direct mechanism, intervention, or full equation recovery unless independent evidence supports that interpretation.

Following this workflow keeps the reported CIRN network connected to the decisions and diagnostics that produced it. The retained-edge table identifies posterior-supported coefficients, but it is only one layer of the analysis. Interpretation should also account for the declared target and predictor sets, lag specification, smoothing and derivative construction, response encoding, adaptive-jitter status, model-ready sample size, class balance, MCMC diagnostics, collinearity diagnostics, cross-mode consistency, and any available validation or sensitivity evidence. This is especially important because different output sheets have different units of analysis: retained-edge tables are coefficient-level, diagnostics are fitted-model-level, and VIF or correlation tables describe the predictor design.

Practical Rule. Do not move directly from `edges_combined` to biological interpretation. A reportable CIRN result should first be checked against the fitted-model diagnostics, effective-sample-size summary, response and jitter metadata, VIF and pairwise-correlation diagnostics, relevant network/posterior/diagnostic figures, and, when available, edge-consistency, validation, or sensitivity outputs. Stronger reporting language is appropriate only when the retained edge, model diagnostics, predictor design, and response construction all support the same interpretation.

XVII. CIRN STUDIO: INTERACTIVE SHINY APPLICATION

CIRN Studio is the interactive R Shiny interface accompanying the CIRN algorithm. It translates the implementation-level workflow in `CIRN_Algorithm.R` into a guided workspace for specialists and non-specialists without changing the underlying inferential definitions. Within one application, users can select or generate a time series, inspect its exploratory characteristics, configure preprocessing and Bayesian inference, review retained regulatory hypotheses and their diagnostics, run optional sensitivity and benchmark analyses, and export a documented analysis bundle. The interface keeps the active dataset, selected variables, analysis settings, model status, and available outputs visible so that each result remains connected to the choices that produced it. Its accessi-

bility is intended to support teaching, exploration, collaborative review, and reproducible analysis; it does not lower the scientific standards required for interpreting a CIRN edge.

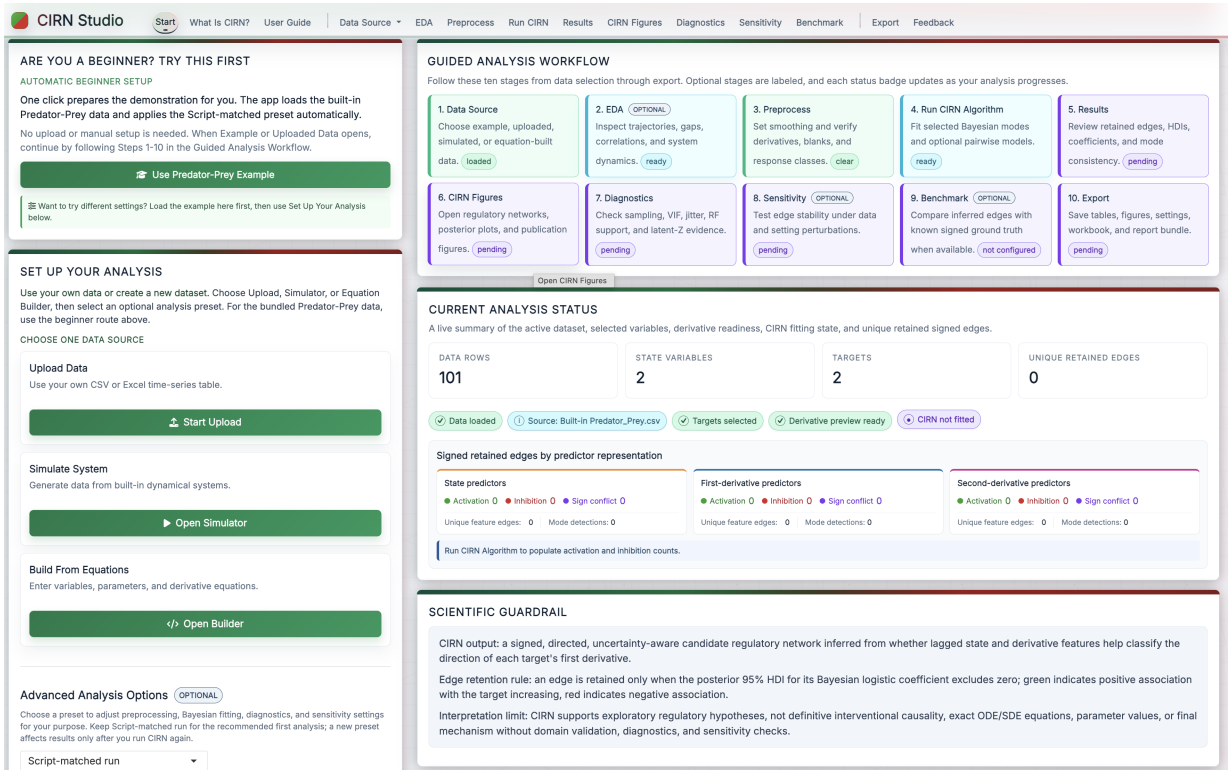
Relationship to the Core Algorithm. CIRN Studio is not a separate network-inference method and does not replace `CIRN_Algorithm.R`. At startup, the application loads and calls reusable functions from the core script while presenting their principal inputs and outputs through a guided interface. For each target, the modeled response remains the estimated first-derivative direction after application of the epsilon blank rule and, when required, the documented adaptive-response procedure. Candidate predictors remain temporally lagged state, first-derivative, and second-derivative features. A coefficient defines a retained edge only when its posterior 95% highest-density interval excludes zero. Presets configure these operations; they do not redefine them.

Numerical agreement between an app run and a script run requires the same data values and row order, time column, target and predictor sets, preprocessing choices, response-construction rules, representation and pairwise modes, lag, priors, MCMC controls, random seed, and software environment. The script-matched preset provides a convenient starting configuration for comparison, but it is not a guarantee of identical numerical output across platforms, package versions, compilers, or CmdStan toolchains.

A. Illustrative Interface and Example Analysis

Supplementary Figures 7–15 present representative views of the current CIRN Studio interface in workflow order, from entry and exploratory data analysis through preprocessing, inference, interpretation, diagnostic qualification, sensitivity analysis, and reproducible export. The screenshots use the bundled Predator–Prey example for demonstration. They document the interface and the scope of available analyses; they are not themselves scientific evidence. Because CIRN Studio is reactive, the controls, tables, figures, status indicators, and available downloads shown in a session depend on the active data source, selected settings, completed analyses, and installed software dependencies. Consequently, the screenshots illustrate representative workflow states rather than every possible input, model configuration, diagnostic outcome, or display condition supported by the application. Numerical findings should be interpreted from exported tables, saved settings, model diagnostics, and the underlying data rather than from screenshots alone. The displayed example should likewise not be interpreted as evidence of expected performance or generalizability for other systems or datasets. For reproducible reporting, users should preserve the CIRN Studio version, data provenance, complete analysis configuration, random seed, software environment, and relevant exported outputs together with any cited figure. Minor differences in wording, navigation, or styling may appear in later releases without changing the underlying CIRN definitions.

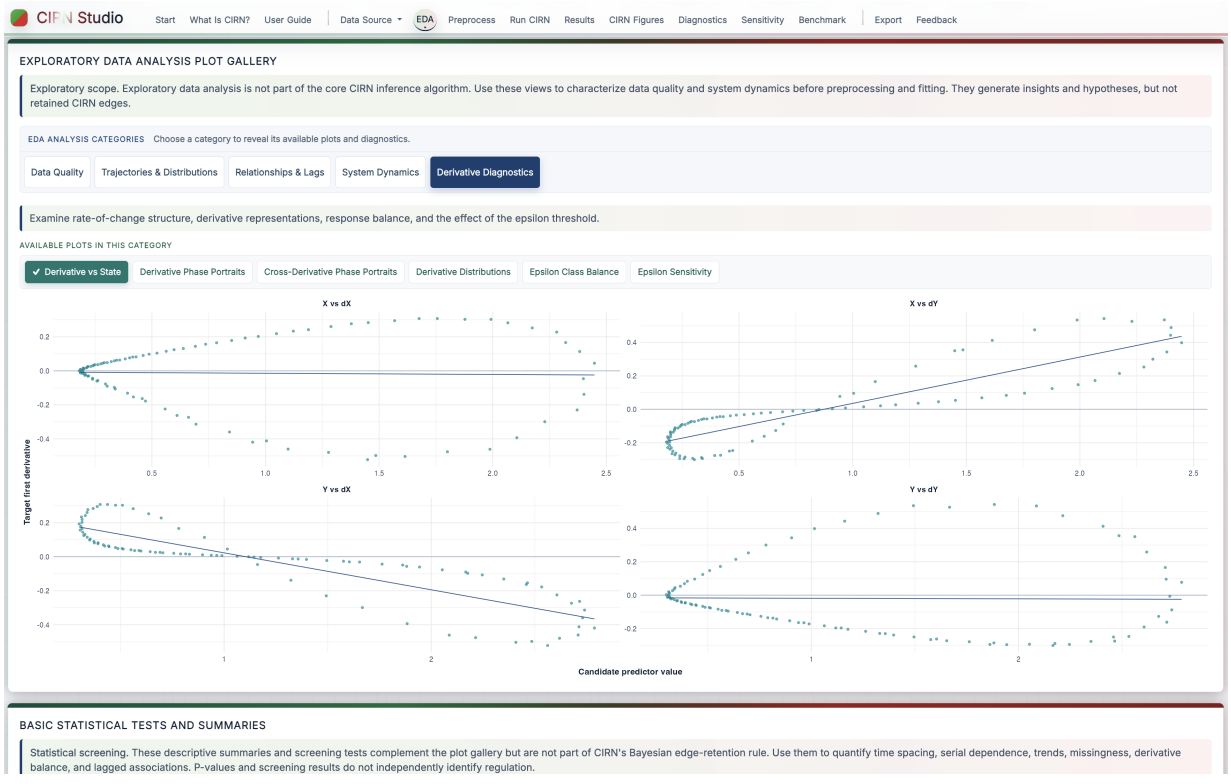
(a) Start-page overview of the CIRN framework, inferential question, authorship, and positive–negative signed-edge convention.



(b) Beginner pathway, data-source setup, guided ten-stage workflow, live analysis status, and scientific guardrail.

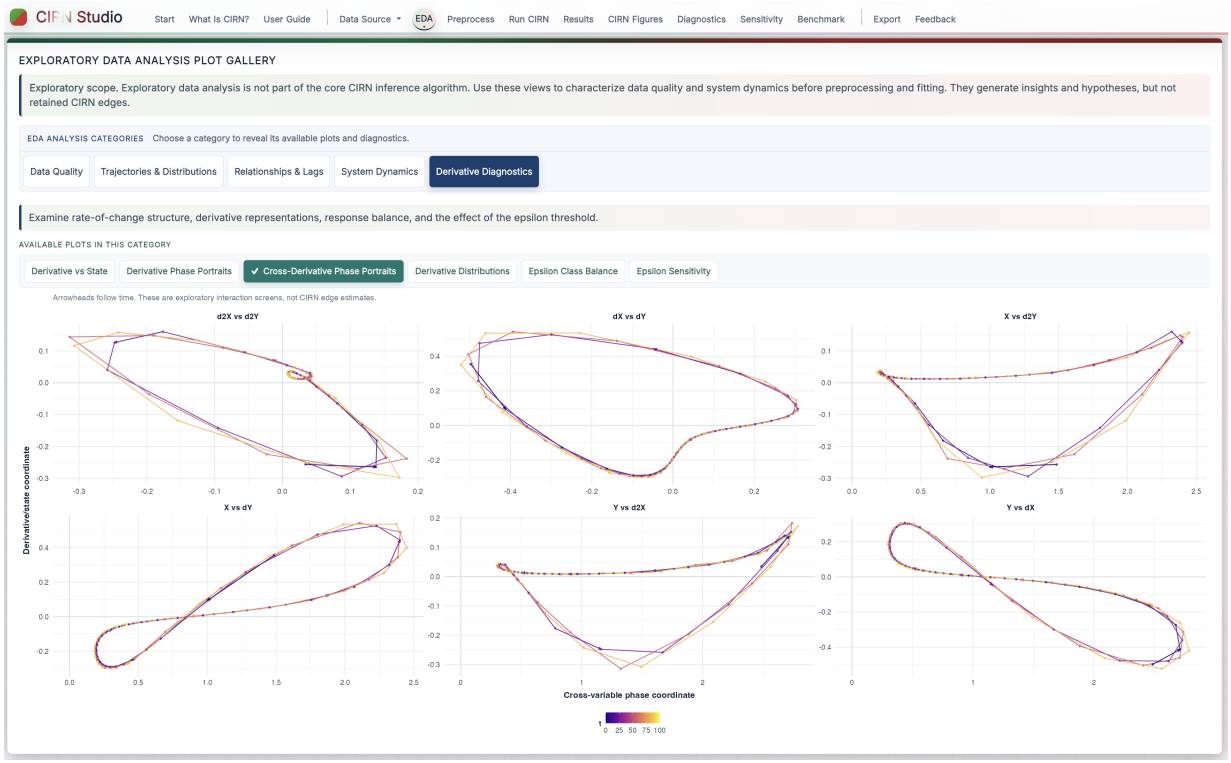
Supplementary Figure 7: CIRN Studio entry point and guided workflow. The Start tab introduces the method and directs users from data selection to interpretation and export while keeping the active source and analysis status visible.

(a) Processed time-series trajectories and the grouped exploratory and statistical-screening controls.

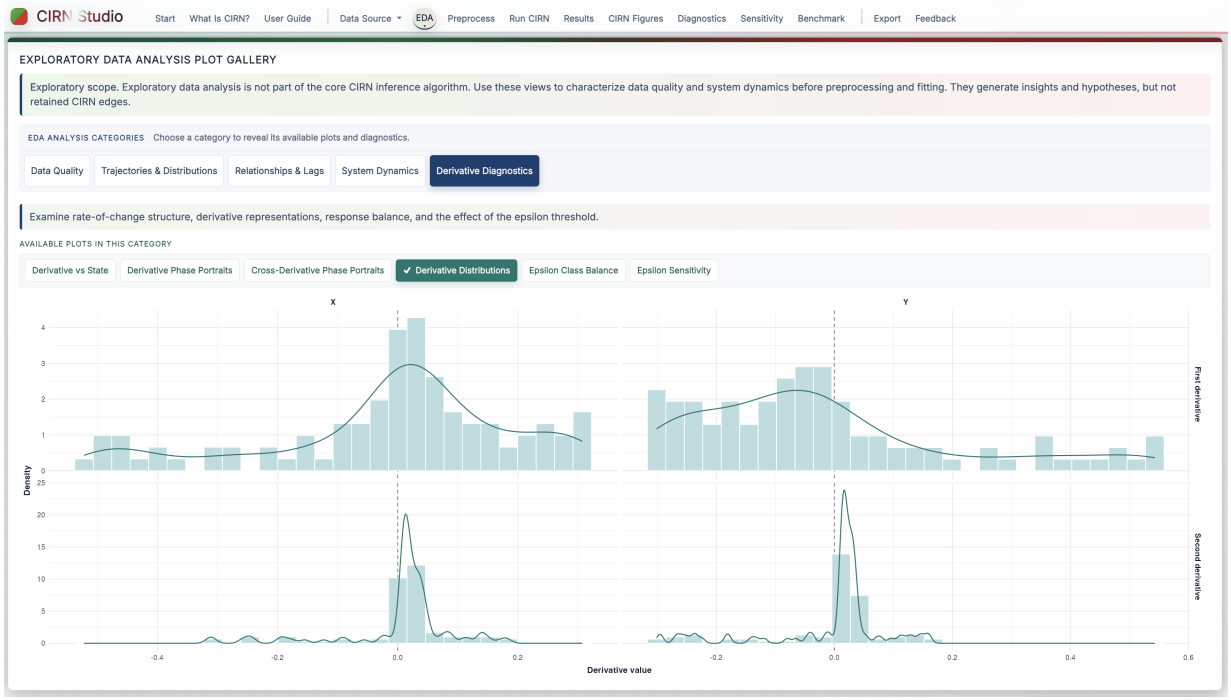


(b) System-agnostic derivative-versus-state views across the available state and target-derivative combinations.

Supplementary Figure 8: Exploratory trajectory and derivative–state views. These panels help users examine temporal form, scaling, derivative structure, and potential state-dependent changes before fitting CIRN.

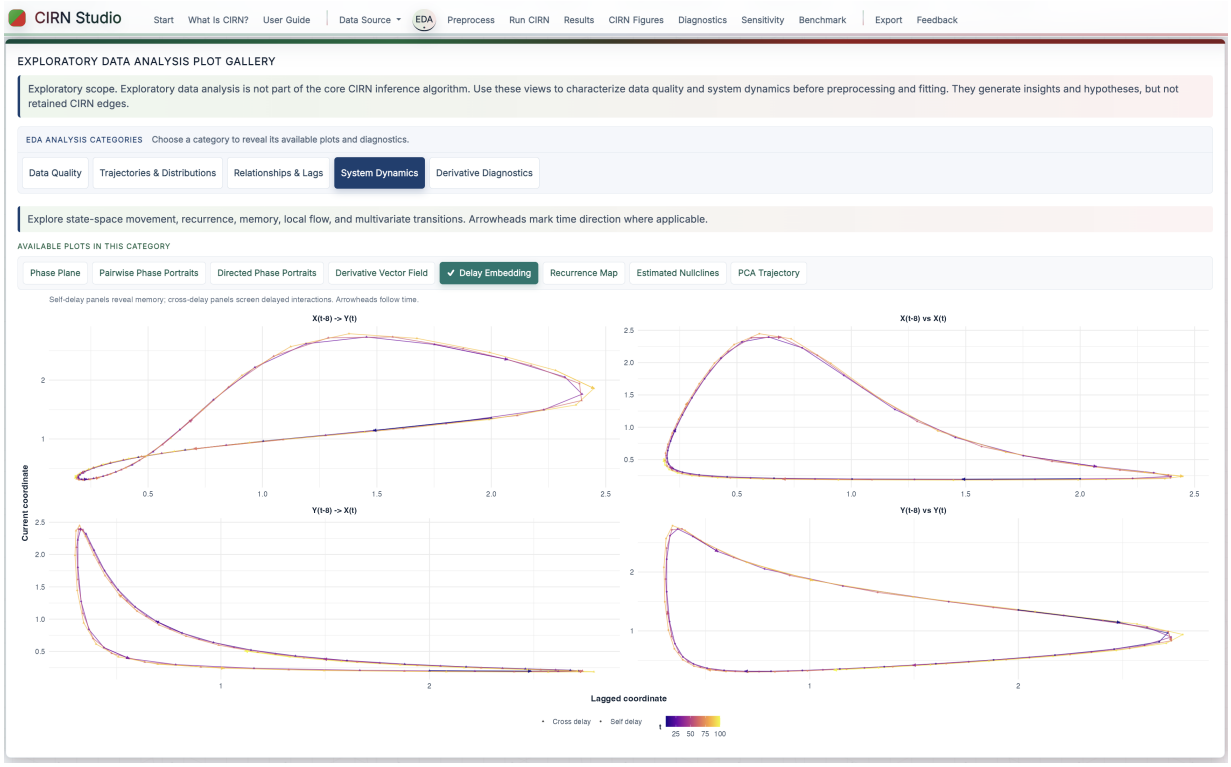


(a) Cross-derivative phase portraits; arrowheads indicate the direction of time along the displayed trajectories.

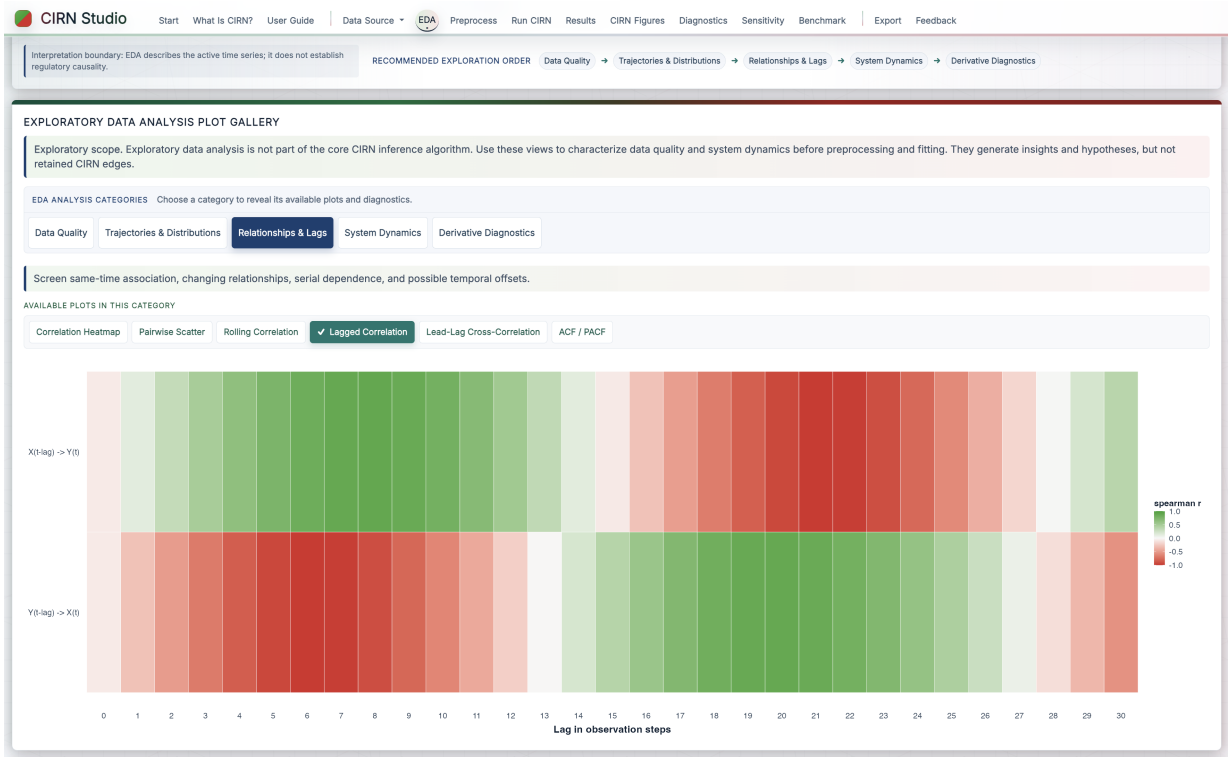


(b) First- and second-derivative distributions for the selected variables, with zero-reference lines.

Supplementary Figure 9: Derivative-focused exploratory views. Cross-representation portraits and derivative distributions reveal rate-of-change structure, asymmetry, concentration near zero, and possible differences among variables.

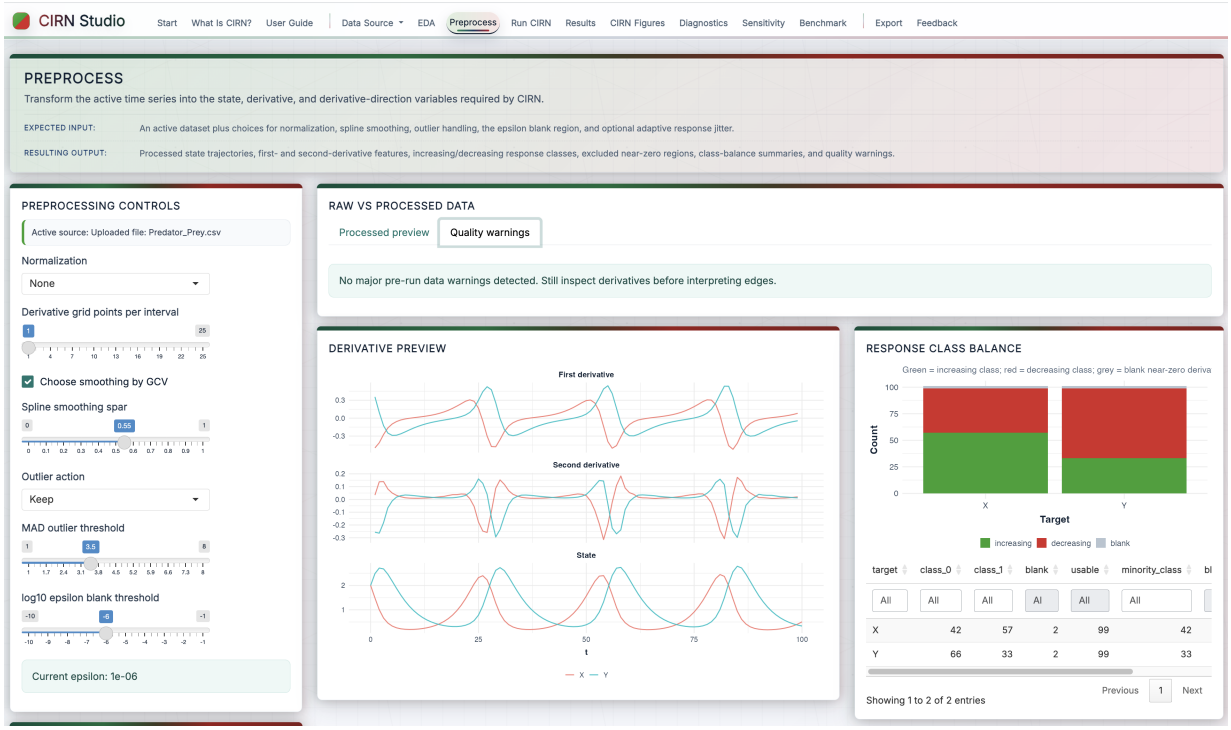


(a) Self- and cross-delay embeddings used to explore temporal memory and delayed interactions; arrowheads follow time.

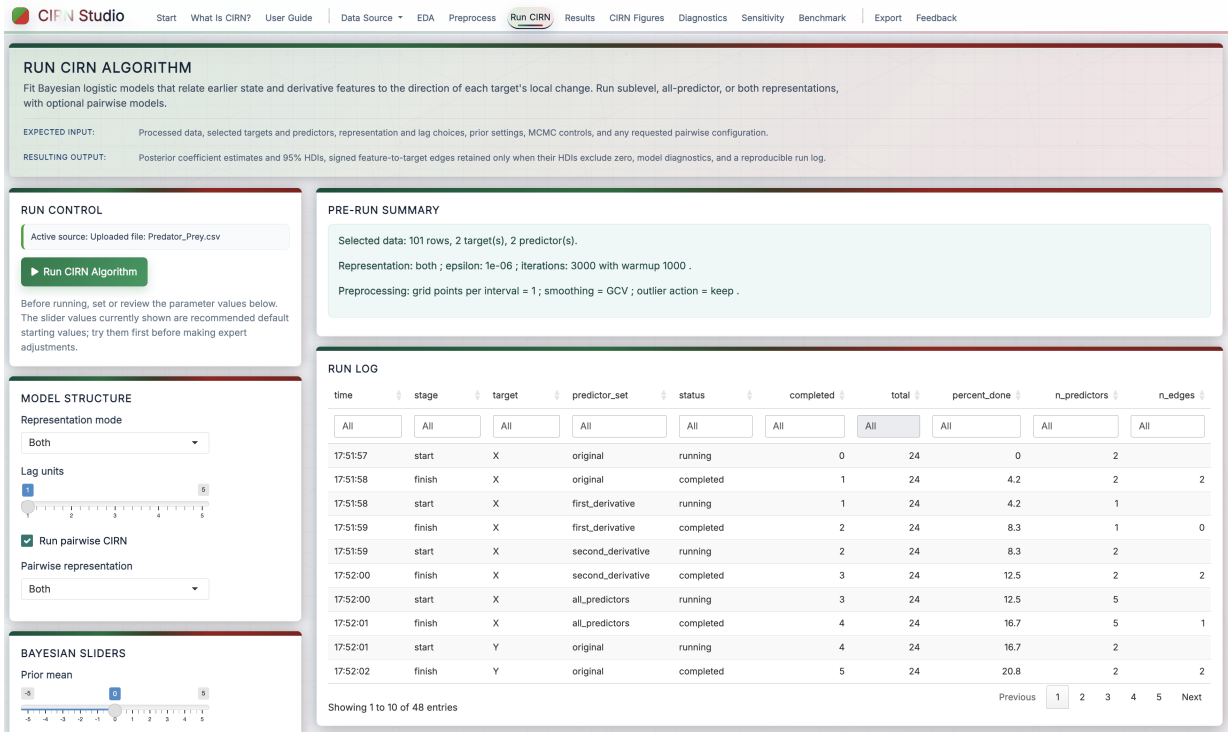


(b) Lagged-correlation heatmap across observation-step lags for the selected variable pairs.

Supplementary Figure 10: Delay and lagged-association views. These exploratory displays help identify memory, temporal offsets, and lag ranges that may warrant closer investigation; they do not themselves define retained CIRN edges.

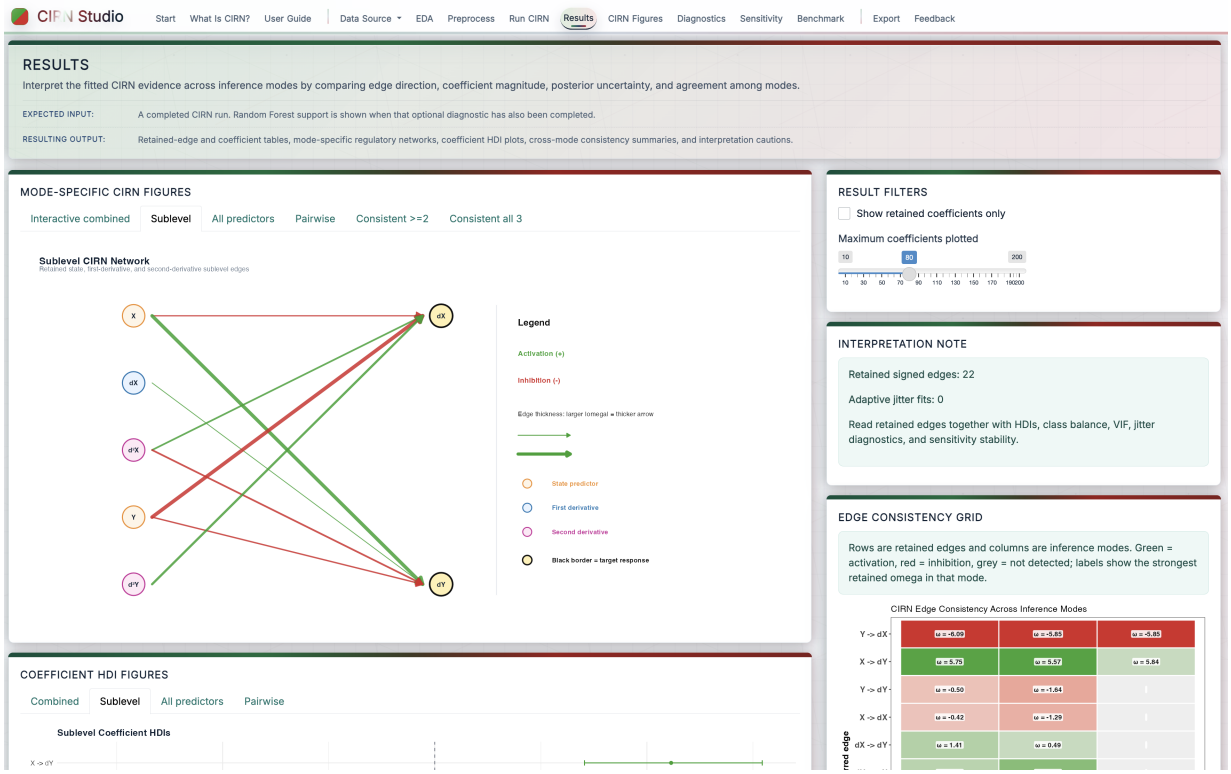


(a) Preprocessing controls, raw-versus-processed checks, derivative preview, and response-class balance.

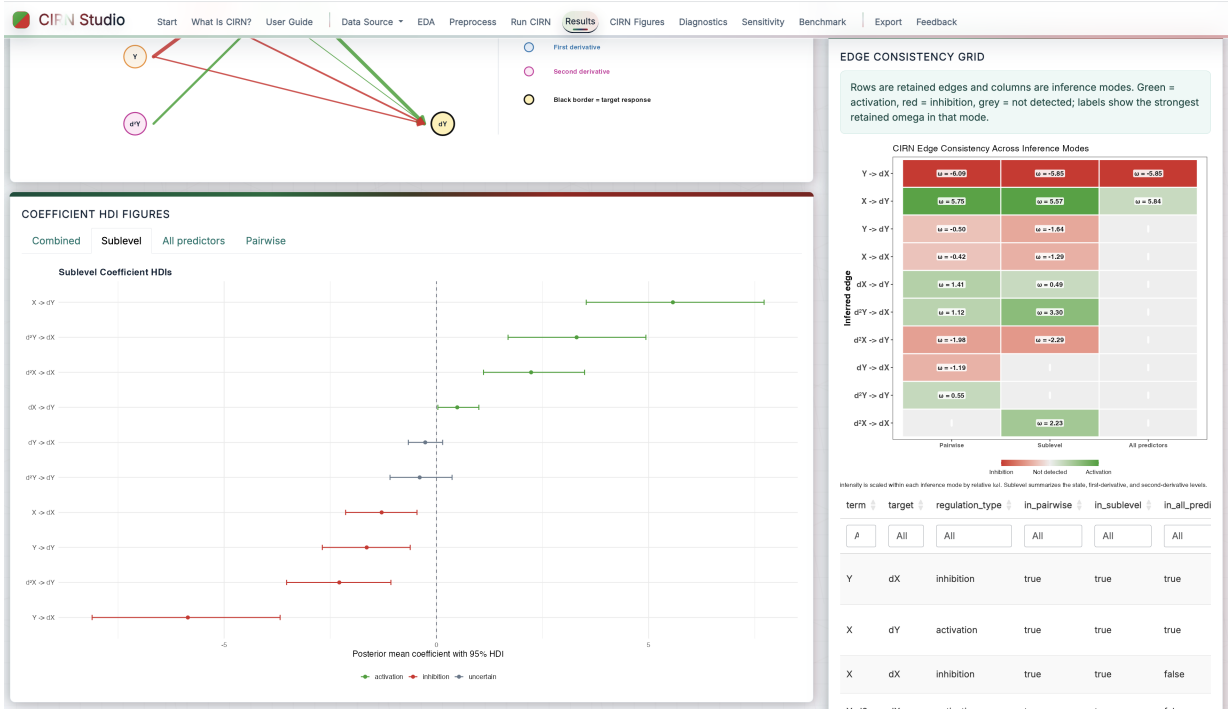


(b) Representation, lag, pairwise, prior, and MCMC controls together with the pre-run summary and execution log.

Supplementary Figure 11: Preprocessing and Bayesian CIRN execution. CIRN Studio exposes the data transformations used to construct derivative-direction responses and records the progress of the requested Bayesian model fits.

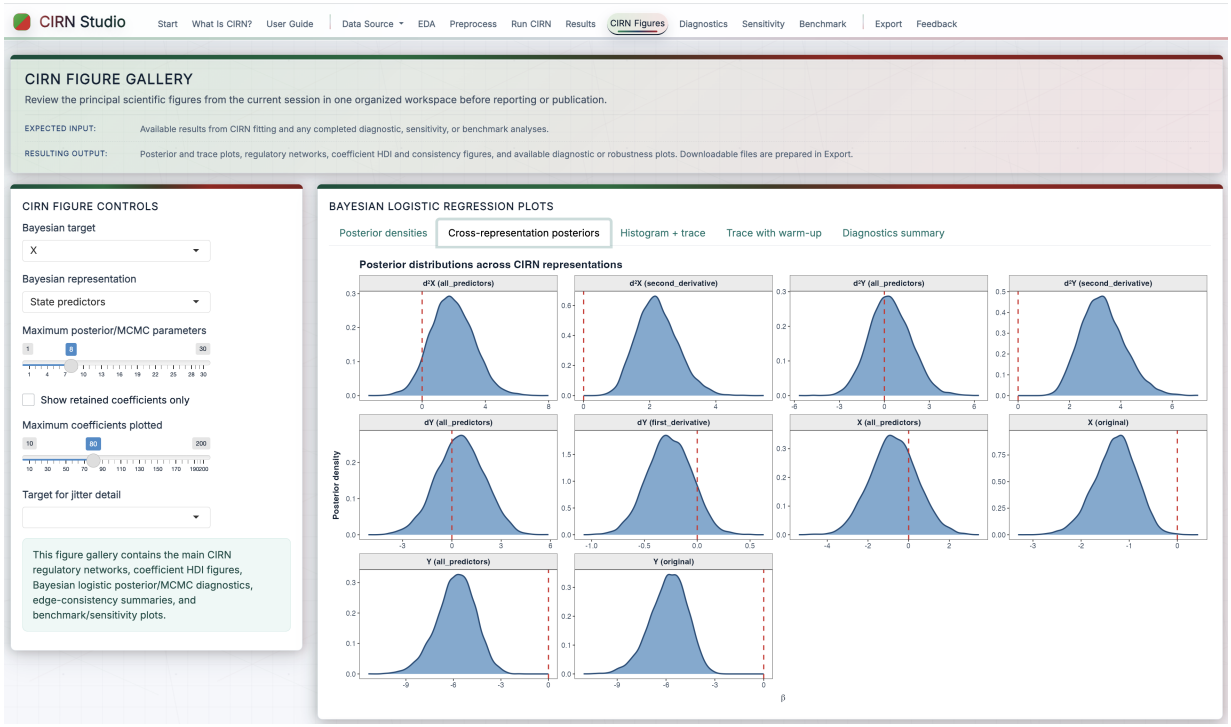


(a) Representation-specific signed regulatory network with interpretation controls and an edge-consistency overview.

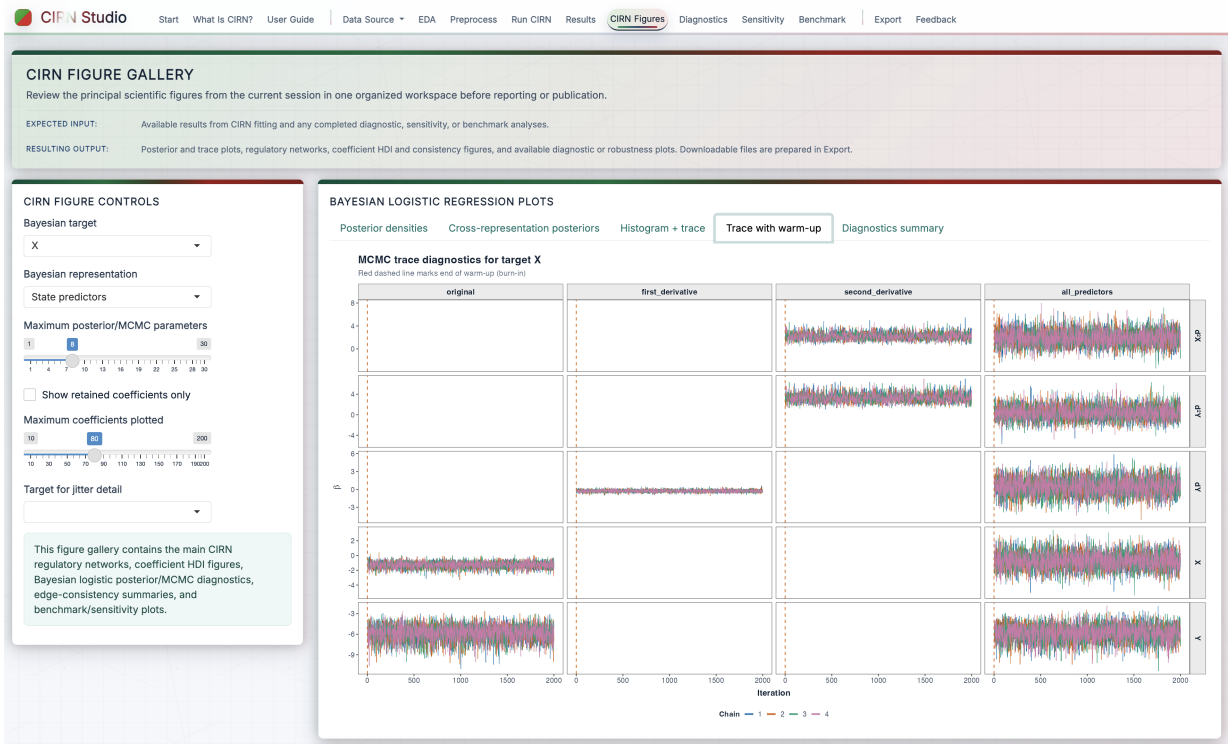


(b) Sublevel coefficient 95% HDIs alongside the edge-consistency grid and its supporting table.

Supplementary Figure 12: Representative retained-edge results and consistency evidence. Green denotes a positive association with the modeled probability of target increase, red denotes a negative association, and coefficient HDIs and cross-mode agreement qualify interpretation of the displayed edges. These colors do not by themselves establish biochemical activation, inhibition, or causality.

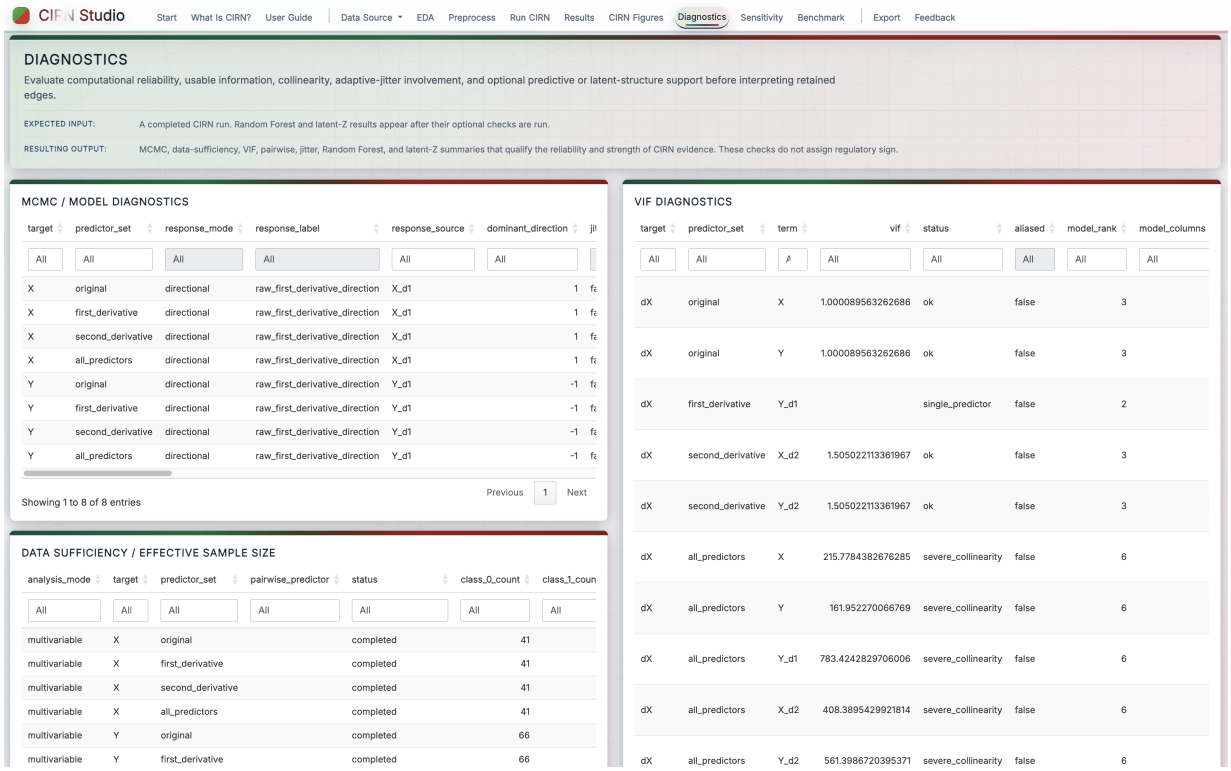


(a) Cross-representation posterior densities for selected Bayesian coefficients.

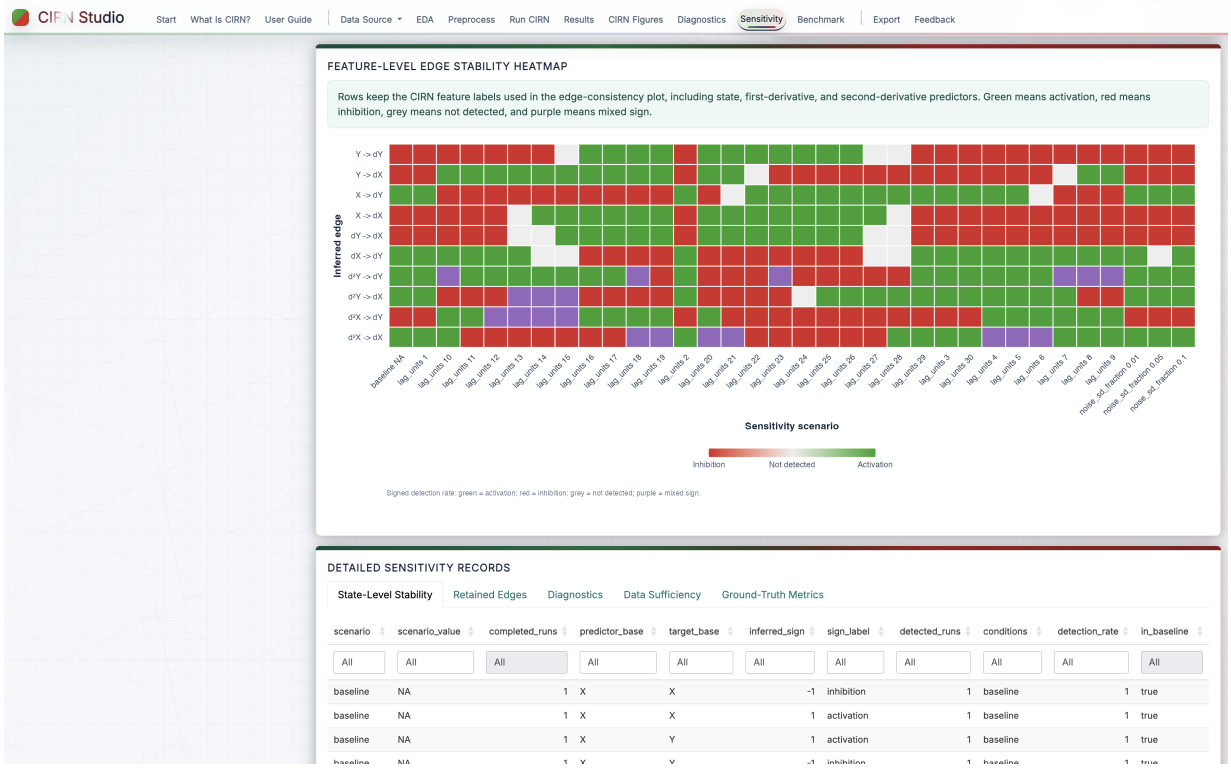


(b) MCMC trace plots with the warm-up boundary shown for the selected target and predictor representations.

Supplementary Figure 13: Bayesian posterior and MCMC figure gallery. Posterior distributions summarize parameter uncertainty, while trace views support inspection of chain mixing, stationarity, and warm-up behavior.

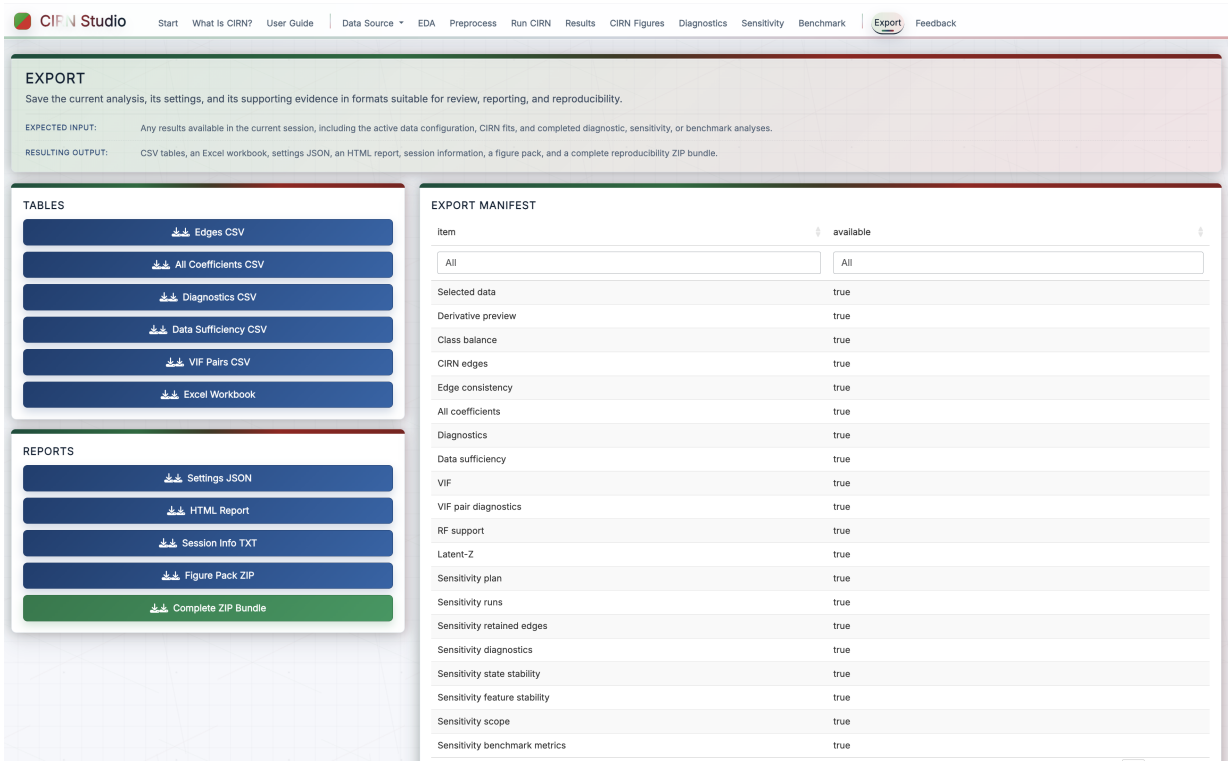


(a) MCMC/model, data-sufficiency, and variance-inflation diagnostics used to qualify fitted CIRN evidence.

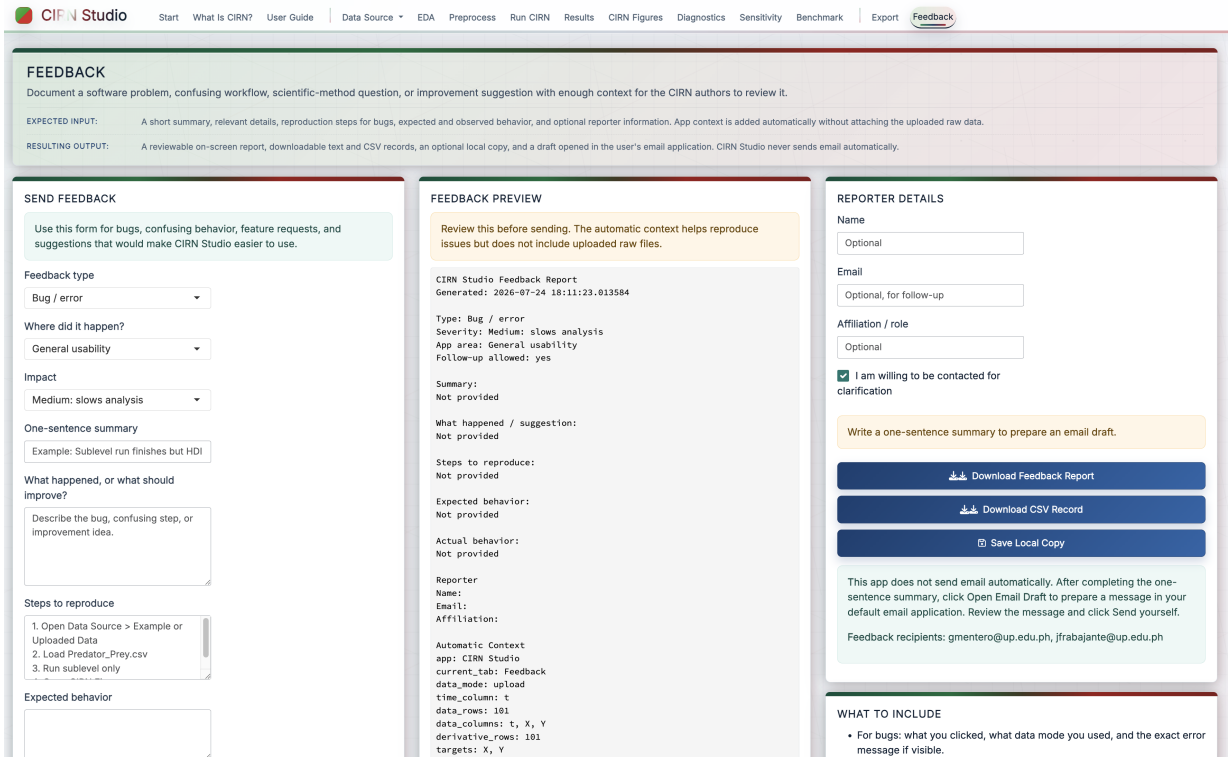


(b) Feature-level edge-stability heatmap and detailed records across configured sensitivity scenarios.

Supplementary Figure 14: Diagnostic qualification and sensitivity assessment. These views expose convergence, data sufficiency, collinearity, and perturbation stability so retained edges are not interpreted from HDI exclusion alone.



(a) Export manifest and download controls for tables, settings, reports, figures, and the reproducibility bundle.



(b) Structured feedback form, automatically generated context preview, and optional reporter details; preparing or downloading a report does not send an email automatically.

Supplementary Figure 15: Reproducibility export and user feedback tools. CIRN Studio packages available evidence for review and provides a structured, user-controlled route for documenting bugs, questions, and improvement suggestions.

B. Supported Data Sources and Guided Workflow

The application supports four routes into an analysis: (i) load the bundled Predator–Prey example; (ii) upload an observed CSV or Excel time-series table; (iii) simulate observations from a built-in dynamical system; or (iv) generate synthetic observations from user-specified differential equations and parameters. A signed ground-truth adjacency matrix may additionally be supplied when a legitimate reference network is available for benchmarking. Only one time-series dataset is active at a time. Before proceeding, users should verify the active-source badge, time column, selected targets, and admissible base predictors. Changing the active source changes the data used by all downstream tabs but does not retroactively refit an existing result.

CIRN Studio presents the computational workflow as ten connected stages:

1. **Data Source.** Select the bundled example, upload observed data, simulate a built-in system, or construct a system from equations. Confirm the time column, target variables, admissible base predictors, row order, numeric state columns, and active-data preview before analysis.
2. **Exploratory Data Analysis (optional).** Examine missingness, sampling gaps, trajectories, distributions, correlations, lagged relationships, phase portraits, recurrence structure, estimated nullclines, derivative behavior, and epsilon-dependent response balance. These views can reveal scale differences, oscillation, delayed co-movement, possible regime changes, and unusual observations that may inform later choices. They generate descriptive evidence and hypotheses, not retained CIRN edges, and are not part of the Bayesian edge-retention rule.
3. **Preprocess.** Configure normalization, spline smoothing, derivative-grid resolution, outlier handling, the epsilon-defined near-zero derivative region, and adaptive-response safeguards. Inspect the processed series, derivative previews, quality warnings, and increasing, decreasing, and blank response counts before fitting.
4. **Run CIRN Algorithm.** Select sublevel, all-predictors, or both multivariable representation modes; choose the lag; optionally enable pairwise fits; and configure priors, iterations, warmup, chains, cores, `adapt_delta`, seed, and optional LOO computation. Sublevel analysis fits the state, first-derivative, and second-derivative predictor levels separately, whereas all-predictors analysis fits the admissible representations jointly. Pairwise analysis fits one candidate predictor at a time and is therefore an unadjusted support view rather than a substitute for multivariable inference. The run log distinguishes completed, skipped, and failed fits so that an isolated failure does not erase valid completed results.
5. **Results.** Review coefficient summaries, 95% HDIs, odds ratios, retained feature-to-target edges, representation-specific networks, and cross-mode consistency. A positive coefficient means that larger values of the lagged predictor are associated with greater modeled odds that the target is increasing rather than decreasing; a negative coefficient means the opposite. In a multivariable model this interpretation is conditional on the other included predictors. Green and red are visual shorthand for these positive and negative temporal-response associations, not proof of a biochemical mechanism.
6. **CIRN Figures.** Inspect posterior densities, trace plots, coefficient-HDI displays, regulatory networks, and consistency figures in one gallery. State, first-derivative, and second-derivative predictors retain their feature-level identities so that the representation supporting each result remains visible.
7. **Diagnostics.** Evaluate MCMC behavior, usable sample size, class balance, collinearity, variance inflation, adaptive-response involvement, and pairwise versus multivariable evidence. Optional Random Forest checking measures reproducible nonlinear predictive support but neither

assigns regulatory sign nor defines a retained edge. Optional latent-Z screening assesses whether a low-dimensional latent summary yields useful predictive gain; it does not establish a hidden regulator. Neither diagnostic overrides the 95% HDI rule or repairs an unreliable Bayesian fit.

8. **Sensitivity (optional).** Perturb selected data or analysis conditions and specify which inference scope is rerun. State-level stability collapses derivative representations to their corresponding base regulator–target relation, whereas feature-level stability preserves the specific predictor representation, such as X, dX, or d^2X. The selected `sensitivity_inference_scope` determines whether sublevel, all-predictors, pairwise, or all configured modes contribute to the sensitivity results.
9. **Benchmark (optional).** Compare inferred signed relations with a known signed ground-truth adjacency matrix. Because reference adjacency is ordinarily defined at the state-variable level, feature-specific CIRN edges are collapsed to base-variable relations only for benchmark comparison; their original feature-level provenance remains available in the CIRN outputs. Benchmark scores assess agreement with the supplied reference and do not validate the reference itself.
10. **Export.** Save available tables, figures, settings, session information, an Excel workbook, an HTML report, and a complete ZIP bundle. The export manifest distinguishes available from unavailable items so that the contents of a reported analysis can be audited.

The **What Is CIRN?** and **User Guide** tabs provide method and workflow documentation but are not separate inference stages. The **Feedback** page prepares a reviewable report and, when requested, opens a draft in the user's email application. CIRN Studio does not attach the uploaded raw dataset to that report and does not send email automatically.

C. Reproducibility and Interpretation Safeguards

Several interface features reduce avoidable analysis errors. Active-source badges identify the dataset currently used downstream; pre-run summaries expose major settings before execution; status indicators distinguish data readiness, derivative readiness, and fitted results; and progress logs record requested, completed, skipped, and failed fits. Presets provide coherent starting configurations, including a script-matched option for closer comparison with the main implementation, while leaving the applied controls visible. Selecting a new preset changes controls but does not change an existing result until the algorithm is run again.

The app distinguishes a *unique feature edge* from a *mode detection*. The same feature-to-target relation detected in pairwise, sublevel, and all-predictors analyses is one unique candidate relation with evidence from several modes, not three biological interactions. Sign conflicts across modes are preserved and reported rather than silently resolved by majority vote. Pairwise and multivariable coefficients are also not interchangeable: pairwise fits describe one-predictor association, while sublevel and all-predictors coefficients are conditional on the other predictors included in the corresponding model.

A reproducible comparison requires more than selecting the same preset. Analysts should preserve the data values and row order, time column, target and predictor sets, preprocessing and response-construction settings, representation and pairwise modes, lag, priors, MCMC controls, random seed, package versions, R version, operating system, compiler, and CmdStan toolchain. A fixed seed improves reproducibility within a matched environment but does not guarantee bit-for-bit equality across platforms or software versions. The complete export bundle should accompany any result intended for formal reporting.

Interpretive Boundary. A green edge means a retained positive association with the modeled probability that the target is increasing; a red edge means a retained negative association. These labels do not by themselves establish biochemical activation or inhibition, interventional causality, direct physical interaction, an unmeasured regulator, or the exact governing ODE/SDE. An absent edge is likewise not proof that no relation exists. Every retained or absent edge is conditional on the observed data, temporal alignment, predictor set, response construction, prior and fitting choices, and model adequacy. Exploratory plots, Random Forest support, latent-Z screening, sensitivity results, and benchmark scores can qualify or triangulate the evidence, but they cannot rescue a poorly identified or diagnostically unreliable fit.

D. Local Use, Deployment, and Computational Requirements

For local use, the distributed application folder should retain `app.R`, `CIRN_Algorithm.R`, the `data/` and `www/` folders, the user manual, and the accompanying README file. The filename `app.R` follows the standard Shiny application convention. After installing the required R packages, platform build tools, and CmdStan toolchain described in the README, users may open the folder as the working directory and run `shiny::runApp()`. A console message of the form `Listening on http://127.0.0.1:PORT` means that the local R process is serving the app on the user's computer and that the displayed address may be opened in a browser.

During local execution, uploaded or generated data are handled by the user's local R session. Exports are created only when requested, and the feedback email feature opens a user-controlled draft rather than transmitting a message automatically. Nevertheless, settings files, reports, session information, and feedback records may contain file names, paths, software details, or analysis context. Users should review exported or drafted material before sharing it.

When CIRN Studio is published to a remote host, uploaded data are processed on that host rather than solely on the user's computer. A deployment administrator must therefore configure access control, transport security, privacy notices, upload and execution limits, temporary-file cleanup, logging policy, session isolation, error recovery, dependency versions, and sufficient memory and CPU resources. The application itself does not replace these administrative safeguards.

Computational cost depends on the number of observations, targets, predictors, requested representation modes, pairwise fits, MCMC chains and iterations, diagnostic options, sensitivity scenarios, replicates, and `sensitivity_inference_scope`. In particular, the `everything` sensitivity scope reruns substantially more models than `sublevel`. A short exploratory run is therefore recommended before a final high-iteration analysis. Results intended for publication should then be regenerated in a clean session with fixed settings and seeds and archived with the complete reproducibility bundle.

XVIII. DETAILED RESULTS AND SUPPORTING ANALYSES OF THE MAIN ARTICLE

A. Geometric Characterization of Representative Dynamics

To examine whether CIRN recovers interpretable dynamical structure rather than merely isolated statistical associations, we evaluated the framework on representative two-variable systems with known governing equations and visually distinct trajectory geometries. These examples were not intended to exhaustively catalogue every possible dynamical behavior, but rather to illustrate how CIRN responds to characteristic regimes such as reciprocal exchange, nonlinear saturation, inverse-

response dynamics, cooperative growth, and monotonic decay. For each system, the inferred network was examined together with the consistency of signed posterior coefficients across pairwise, sublevel, and all-predictors configurations. This allowed us to assess not only whether a directed interaction was retained, but also whether its sign, magnitude, and predictor representation were preserved across alternative modeling formulations. Interactions supported by multiple inference configurations generally exhibited greater stability in both sign and effect size than interactions recovered by only a single configuration, suggesting that consistency across CIRN formulations may serve as an additional indicator of regulatory robustness.

The clearest equation-level signatures emerge in the reciprocal exchange systems shown in Supplementary Figure 16a and Supplementary Figure 16b. In both systems, the governing equations impose opposite-signed derivative behavior between the two variables, such that one state increases while the other decreases through a shared interaction term. In Supplementary Figure 16a, this relationship is expressed as $dX/dt = -aXY$ and $dY/dt = aXY$, whereas Supplementary Figure 16b reverses the signs. Despite the reversal of the observed state trajectories, CIRN recovered a stable reciprocal inhibitory architecture between derivative-based responses in both systems. The interactions $dY \to dX$ and $dX \to dY$ remained negative across pairwise, sublevel, and all-predictors configurations, indicating that the inferred structure was not an artifact of a particular model formulation. Importantly, CIRN did not simply identify that X and Y were associated; rather, it recovered a regulatory signature consistent with opposing directions of change. This directly contrasts with association-based approaches such as WGCNA, MRNET, ARACNE, and PIDC, which can identify dependency or co-variation but generally do not provide intrinsic temporal directionality or signed regulatory interpretation [27, 15, 13, 3]. Thus, for reciprocal exchange dynamics, the key contribution of CIRN is not only edge detection but recovery of an interpretable signed relationship at the level of temporal change.

The comparison between Supplementary Figure 16a and Supplementary Figure 16b is particularly informative because, although the monotonic trends are reversed, the derivative-level regulatory architecture remains qualitatively stable. This suggests that CIRN captures a dynamical signature of reciprocal exchange rather than merely reproducing the visual direction of the state trajectories. In other words, the framework does not recover the symbolic term aXY directly, but it does recover interaction constraints compatible with the underlying equations through a stable anti-parallel derivative structure. This distinguishes CIRN from full equation-discovery methods such as SINDy, which seek explicit governing equations and may require suitable candidate libraries, dense sampling, or sufficiently informative trajectories [1]. CIRN instead occupies a structural inference role: it does not estimate the exact functional form of the dynamics, but identifies signed and directed dependency constraints that are compatible with observed temporal behavior.

In contrast, systems characterized by cooperative or mutually reinforcing dynamics produce markedly different regulatory signatures. The inverse-saturating growth system shown in Supplementary Figure 17g provides a representative example. Although the observed trajectories appear relatively simple and monotonic, CIRN retained a compact activating architecture dominated by derivative-mediated interactions. The positive regulatory relationships among derivative responses are consistent with a dynamical regime in which both variables evolve in the same overall direction while their rates of change remain coupled through the governing equations. Similarly, the cooperative growth system in Supplementary Figure 18k generated a predominantly activating regulatory structure. Because both governing equations depend positively on the combined state term $(X + Y)$, the consistent retention of positive derivative-mediated interactions across inference configurations indicates that the shared growth direction is strongly encoded within the temporal derivative structure. Compared with the reciprocal inhibition observed in Supplementary Figure 16a and Supplementary Figure 16b, these systems function as positive-control-like examples

in which cooperative dynamics produce coherent activating architectures rather than antagonistic exchange patterns.

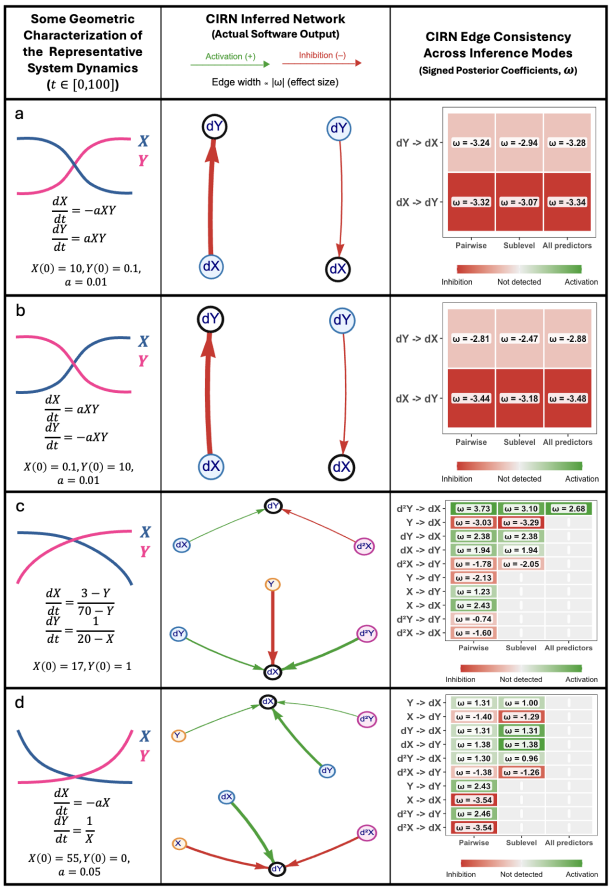
Several systems exhibit related behavior and can be interpreted under the same dynamical principle. Rows with reciprocal exchange or opposing monotonic change, such as Supplementary Figure 16a–b, tend to produce anti-parallel inhibitory derivative-level structures because the governing equations encode opposite directions of temporal motion. Rows with cooperative growth or shared monotonic increase, such as Supplementary Figure 18j–k, tend to produce activating structures because both variables move along a common directional regime. Systems with saturation, inverse response, or stronger curvature, such as Supplementary Figure 17e–g, tend to shift regulatory support toward derivative- and curvature-based predictors because the relevant dynamical information is expressed less through absolute state magnitude and more through local rate and acceleration. The remaining systems may be interpreted analogously: when trajectories share directionality, CIRN tends to favor reinforcing interactions; when trajectories oppose each other, CIRN tends to recover antagonistic interactions; and when nonlinear curvature dominates, derivative-mediated effects become more informative than state-level associations alone.

A recurring observation across the representative systems is that many of the most stable and consistently retained interactions arise from first- and second-derivative predictor representations rather than from the original state variables alone. This finding supports the central premise underlying CIRN that regulatory information is distributed across multiple temporal representations. While original state variables describe the instantaneous configuration of the system, first derivatives encode local rates of change and second derivatives encode local curvature or acceleration. Consequently, derivative-based predictors often preserve information about the mechanisms governing system evolution that is not readily apparent from the state trajectories themselves. This point also distinguishes CIRN from predictive machine-learning approaches such as GENIE3 and GRNBoost2, where regulatory influence is commonly summarized through feature importance and may not directly encode signed temporal state change [9, 16]. CIRN instead links predictor representations to the probability of target-state increase, enabling signed regulatory interpretation grounded in the temporal evolution of the system.

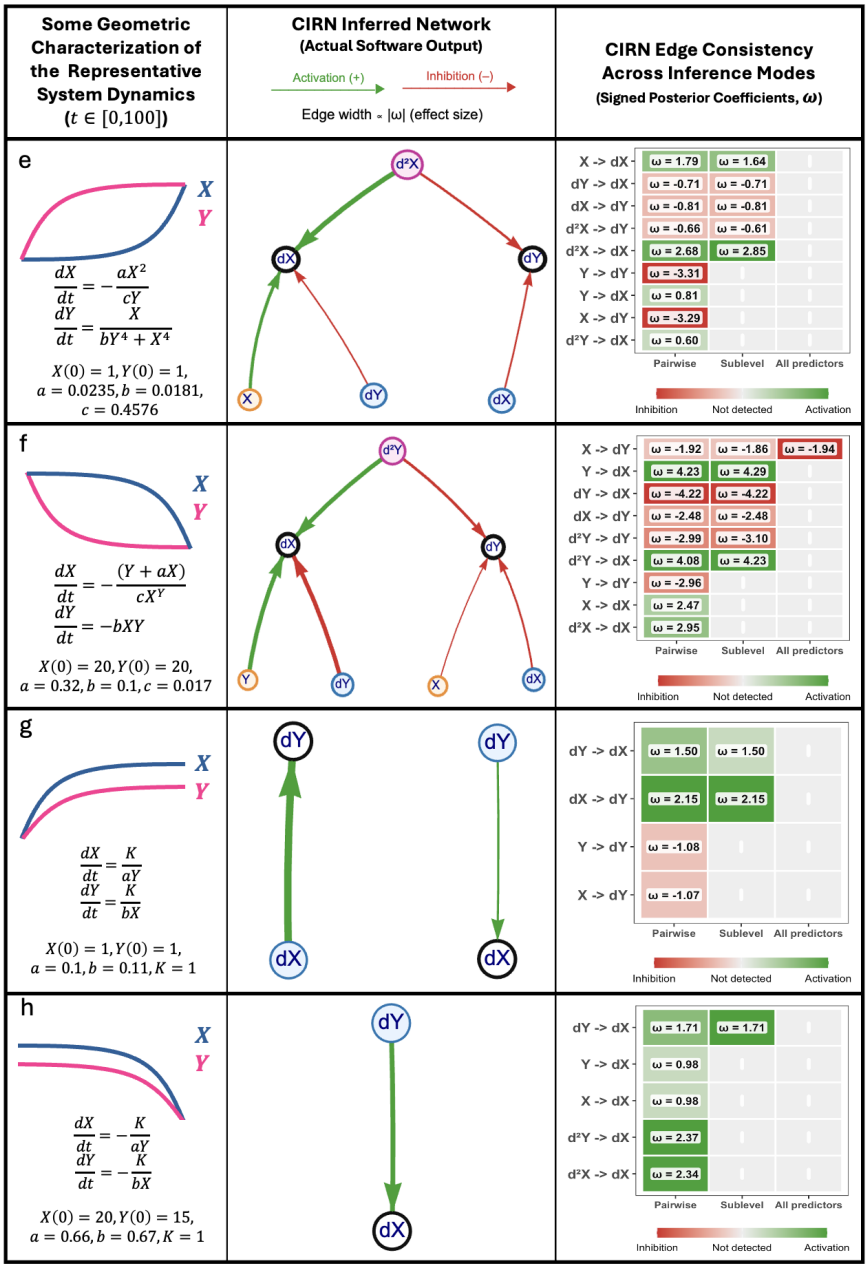
The results also clarify how CIRN differs from temporal and mechanistic inference frameworks. Granger causality imposes directionality through lagged predictability, but its standard formulation is typically linear and does not directly encode regulatory sign in terms of directional state change [7, 21]. CCM and PCM infer asymmetric dynamical influence through state-space reconstruction, but their outputs are not naturally expressed as signed regulatory edges or posterior-supported coefficient distributions [11, 24]. ODE-based and GOBI-type approaches provide stronger mechanistic interpretation by embedding interactions within dynamical equations, but they require explicit modeling assumptions and parameter estimation [17]. By contrast, CIRN preserves temporal ordering and signed interpretation without requiring explicit specification of the global functional form of the governing system. Its posterior HDI-based edge inclusion also provides a principled uncertainty-aware alternative to purely heuristic edge thresholds, aligning with the broader motivation for probabilistic network inference [19, 25].

Across the examined representative systems, the results demonstrate that CIRN provides substantially more information than a binary edge-detection procedure. The framework distinguishes reciprocal antagonism, cooperative activation, nonlinear derivative-mediated regulation, and curvature-dependent interactions through the joint analysis of state, velocity, and acceleration information. The inferred networks should not be interpreted as exact symbolic reconstructions of the governing differential equations. Rather, they represent probabilistic structural approximations of the dependency architecture governing system evolution. Consequently, CIRN occupies an intermediate position between purely associative network inference and full equation-discovery approaches:

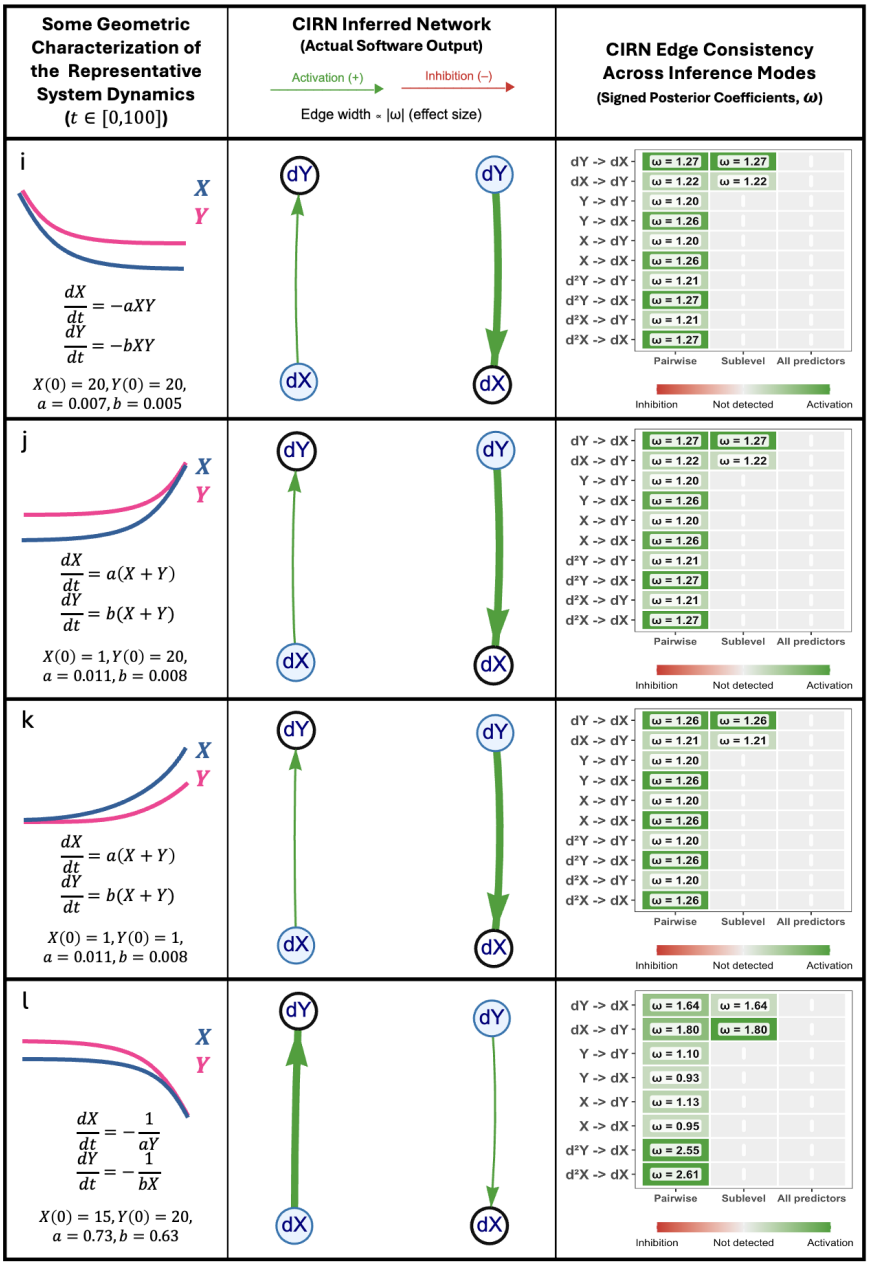
it does not recover the explicit functional form of the dynamics, but it identifies the directed and signed interaction constraints that shape the evolution of the system. Across diverse dynamical regimes, the results demonstrate that temporal geometry leaves characteristic regulatory signatures that can be recovered through the CIRN framework, providing interpretable and uncertainty-aware structural insight into the mechanisms underlying complex dynamical systems.



Supplementary Figure 16: Geometric characterization of representative two-variable dynamical systems and corresponding CIRN-inferred regulatory architectures across inference configurations. Rows (a–d) present four benchmark systems selected to represent distinct classes of temporal behavior, including reciprocal monotonic exchange dynamics (a,b), nonlinear saturation and threshold-like responses (c), and asymmetric inverse-response dynamics (d). These systems span a range of temporal geometries and interaction structures, providing illustrative examples of how differences in system behavior are reflected in the regulatory architectures inferred by CIRN. The left column summarizes the simulated state trajectories, with X shown in blue and Y shown in magenta, together with the governing differential equations, initial conditions, and parameter values. The middle column displays the corresponding directed and signed regulatory networks inferred by CIRN. Orange, blue, and magenta nodes denote original-state, first-derivative, and second-derivative predictor representations, respectively, whereas black-bordered nodes denote target directional responses. Directed edges represent posterior-supported regulatory influences from predictor to target. Green arrows indicate activation (⟶), red arrows indicate inhibition (⟞), and edge thickness is proportional to the magnitude of the posterior mean edge weight $|\omega|$. The right column summarizes inferred regulatory interactions across three CIRN configurations: pairwise fitting, representation-specific multivariable fitting (*sublevel*), and joint multivariable fitting using all candidate predictors (*all-predictors*). Each row corresponds to a candidate directed interaction and each cell reports the signed posterior mean edge weight ω. Green cells indicate activating regulation, red cells indicate inhibitory regulation, and grey cells denote interactions not retained under the posterior HDI criterion. Color intensity is proportional to the magnitude of the corresponding posterior mean effect relative to the inference configuration. Across the representative systems, CIRN recovers regulatory architectures ranging from sparse monotonic interactions to richer derivative-mediated structures associated with nonlinear temporal behavior. Differences among inference configurations further demonstrate that the inferred regulatory architecture depends not only on the underlying system dynamics but also on the predictor representation used during inference. Collectively, these results illustrate how CIRN translates temporal geometry into probabilistic, directed, and signed regulatory hypotheses through the joint exploitation of state, velocity, and curvature information. Node sizes are adjusted solely for visualization purposes and do not correspond to any quantitative network property.



Supplementary Figure 17: (continued) Geometric characterization of representative two-variable dynamical systems and corresponding CIRN-inferred regulatory architectures across inference configurations. Rows (e–h) extend the analysis to additional nonlinear dynamical regimes characterized by saturating growth, interaction-driven responses, inverse-saturating behavior, and monotonic inverse-decay dynamics. Compared with the systems shown in Supplementary Figure 16, these examples exhibit stronger nonlinearities and more pronounced curvature effects, providing a richer setting for evaluating the contribution of derivative-based predictor representations to regulatory inference. Figure layout, node and edge definitions, inference configurations, and heatmap interpretation are identical to those described in Supplementary Figure 16. Rows (e) and (f) generate asymmetric regulatory architectures containing mixed activating and inhibitory influences distributed across state-, velocity-, and curvature-based representations. These systems illustrate how nonlinear interaction terms can produce regulatory signatures that are not apparent from the original state variables alone. Row (g) demonstrates that inverse-saturating growth dynamics generate compact but highly consistent regulatory structures dominated by derivative-mediated interactions, highlighting the ability of CIRN to recover informative higher-order temporal dependencies even when trajectory geometry appears relatively simple. In contrast, row (h) produces a sparse regulatory architecture in which only a small subset of derivative-mediated interactions remain consistently supported, illustrating the capacity of CIRN to selectively retain the most structurally informative relationships when regulatory complexity is limited. These examples further demonstrate that inferred regulatory architecture emerges from the temporal geometry and nonlinear structure of the underlying dynamics and that CIRN flexibly adapts to diverse classes of nonlinear systems while preserving directional and signed regulatory interpretation. Node sizes are adjusted solely for visualization purposes and do not correspond to any quantitative network property.



Supplementary Figure 18: (continued) Geometric characterization of representative two-variable dynamical systems and corresponding CIRN-inferred regulatory architectures across inference configurations. Rows (i–l) complete the collection of representative systems by illustrating symmetric decay, cooperative growth, accelerated expansion, and inverse-response dynamics. These systems are characterized by comparatively simple and highly structured trajectory geometries, allowing assessment of CIRN under strongly monotonic dynamical conditions. Figure layout, node and edge definitions, inference configurations, and heatmap interpretation are identical to those described in Supplementary Figure 16. Rows (i) and (l) exhibit decay-dominated dynamics that yield highly coherent derivative-mediated regulatory signatures, indicating that temporal rates of change may provide stronger regulatory information than the original state variables in systems approaching asymptotic decline. Rows (j) and (k) display cooperative growth dynamics that generate remarkably stable activating structures across inference configurations, reflecting the strong temporal consistency embedded within the underlying equations. In particular, the accelerated growth regime in row (k) amplifies velocity- and curvature-related contributions while preserving the overall coherence of the inferred regulatory architecture. Taken together with the preceding figures, these final examples reinforce the close correspondence between trajectory geometry and inferred regulatory structure. Across a wide spectrum of dynamical behaviors, CIRN consistently translates information contained in system state, velocity, and curvature into probabilistic, directed, and signed regulatory hypotheses. Node sizes are adjusted solely for visualization purposes and do not correspond to any quantitative network property.

B. Classical Lotka–Volterra Dynamics

Building on the geometric characterization of representative two-variable systems, we evaluated CIRN on a classical Lotka–Volterra predator–prey model. The deterministic predator–prey dynamics were generated from

$$\frac{dX}{dt} = \alpha X - \beta XY, \qquad \frac{dY}{dt} = \delta XY - \gamma Y,$$

with initial conditions $X(0) = Y(0) = 2.0$, parameter values $\alpha = 0.29$, $\beta = 0.26$, $\gamma = 0.22$, and $\delta = 0.25$, and numerical integration over $t \in [0, 100]$ using an RK45 solver. In this formulation, prey abundance contributes positively to predator growth through δXY, whereas predator abundance contributes negatively to prey growth through $-\beta XY$. The expected signed interaction structure is therefore $X \longrightarrow Y$ and $Y \longdashv X$, making this system a useful benchmark for assessing whether CIRN can recover biologically meaningful directional and signed dependencies from observational time-series data.

As shown in Supplementary Figure 19A, the simulated trajectories exhibit the characteristic oscillatory behavior of the Lotka–Volterra system, with predator and prey populations displaying recurring phase-shifted cycles. The first- and second-derivative representations reveal alternating periods of increase, decrease, acceleration, and deceleration throughout the simulation. Because CIRN models the direction of temporal change rather than state magnitude alone, these derivative representations provide additional information about the timing and geometry of the oscillatory dynamics.

The network-level summaries in Supplementary Figure 19B–C show that the two canonical predator–prey interactions are stable across CIRN inference configurations. Predator abundance is inferred to inhibit the prey directional response ($Y \longdashv dX$), whereas prey abundance is inferred to activate the predator directional response ($X \longrightarrow dY$). In the edge-consistency summary, $Y \longdashv dX$ produced posterior mean effects of $\omega = -6.09$, -5.85, and -5.85 across pairwise, sublevel, and all-predictor configurations, respectively. Similarly, $X \longrightarrow dY$ produced effects of $\omega = 5.75$, 5.57, and 5.84. The persistence of both sign and effect magnitude despite changes in model structure indicates that these edges represent dominant features of the underlying dynamics rather than artifacts of a particular predictor representation.

To make the CIRN output traceable while avoiding confusion between edge-level and model-level quantities, we summarize the deterministic Lotka–Volterra results in two companion tables. Supplementary Table 16 reports the retained posterior-supported edge rows from `edges_combined`, preserving the raw `analysis_mode` and `predictor_set` labels used by the algorithm. Each row corresponds to an edge that passed the HDI-based retention criterion for a specific fitted representation, whereas the Support column summarizes whether the same term–target relation also appeared across pairwise, sublevel/multivariable, and all-predictor analyses. Thus, the table should be read as an edge-list summary of the CIRN output rather than as a list of fitted models.

Because quantities such as usable sample size, number of predictors, class balance, convergence, effective sample size, and VIF are indexed by fitted model configuration rather than by retained edge, they are reported separately in Supplementary Table 17. This separation is important because the number of retained edges is not expected to match the number of diagnostic rows. The diagnostics table therefore provides the audit trail for the fitted models that generated the edge-level output, including completed pairwise and multivariable fits and the expected `skipped_no_predictors` cases where no admissible predictor remained for a target–predictor-set combination.

The evidence in Supplementary Figure 19 and Supplementary Tables 16–17 indicates that CIRN recovers the canonical signed architecture of the predator–prey system. The prey state X is con-

Supplementary Table 16: Expanded CIRN edge-list output for the deterministic Lotka–Volterra predator–prey system. The table reports all posterior-supported rows from `edges_combined`, with cross-mode support summarized from `edge_consistency`.

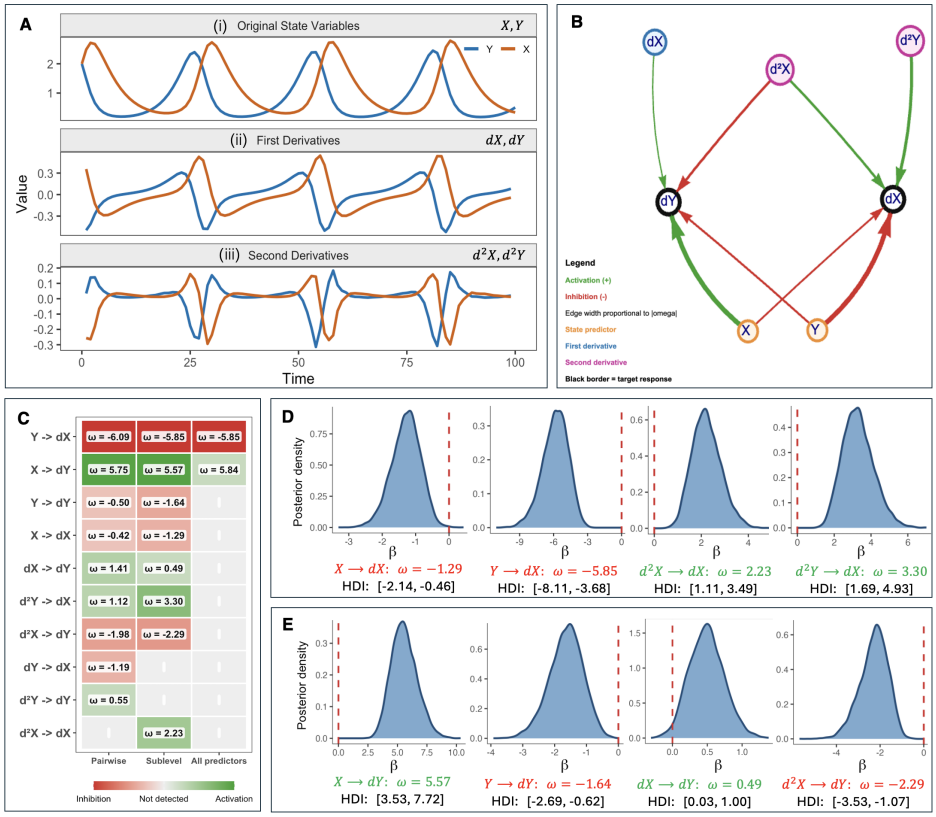
Mode	Predictor set	CIRN edge	ω	OR	95% HDI	Strength	Support	Class (0/1)	Jitter
multivariable	original	$X \longrightarrow\!\!\!\!\shortmid\ dX$	-1.29	0.28	$[-2.14, -0.46]$	weak	P,S	41/57	No (raw)
pairwise	original	$X \longrightarrow\!\!\!\!\shortmid\ dX$	-0.42	0.66	$[-0.85, -0.01]$	credible	P,S	41/57	No (raw)
multivariable	original	$X \longrightarrow dY$	5.57	262.66	$[3.53, 7.72]$	strong	P,S,A	66/32	No (raw)
multivariable	all_predictors	$X \longrightarrow dY$	5.84	343.00	$[3.60, 8.28]$	strong	P,S,A	66/32	No (raw)
pairwise	original	$X \longrightarrow dY$	4.29	72.82	$[2.78, 6.02]$	strong	P,S,A	66/32	No (raw)
pairwise	all_predictors	$X \longrightarrow dY$	5.75	314.02	$[3.67, 7.86]$	strong	P,S,A	66/32	No (raw)
multivariable	original	$Y \longrightarrow\!\!\!\!\shortmid\ dX$	-5.85	2.87e-3	$[-8.11, -3.68]$	strong	P,S,A	41/57	No (raw)
multivariable	all_predictors	$Y \longrightarrow\!\!\!\!\shortmid\ dX$	-5.85	2.89e-3	$[-8.27, -3.49]$	strong	P,S,A	41/57	No (raw)
pairwise	original	$Y \longrightarrow\!\!\!\!\shortmid\ dX$	-4.80	8.20e-3	$[-6.60, -3.08]$	strong	P,S,A	41/57	No (raw)
pairwise	all_predictors	$Y \longrightarrow\!\!\!\!\shortmid\ dX$	-6.09	2.26e-3	$[-8.34, -4.02]$	strong	P,S,A	41/57	No (raw)
multivariable	original	$Y \longrightarrow\!\!\!\!\shortmid\ dY$	-1.64	0.19	$[-2.69, -0.62]$	weak	P,S	66/32	No (raw)
pairwise	original	$Y \longrightarrow\!\!\!\!\shortmid\ dY$	-0.50	0.61	$[-0.99, -0.05]$	weak	P,S	66/32	No (raw)
multivariable	first_derivative	$dX \longrightarrow dY$	0.49	1.63	$[0.03, 1.00]$	weak	P,S	66/32	No (raw)
pairwise	first_derivative	$dX \longrightarrow dY$	0.49	1.63	$[0.03, 1.00]$	weak	P,S	66/32	No (raw)
pairwise	all_predictors	$dX \longrightarrow dY$	1.41	4.11	$[0.39, 2.44]$	weak	P,S	66/32	No (raw)
pairwise	all_predictors	$dY \longrightarrow\!\!\!\!\shortmid\ dX$	-1.19	0.30	$[-2.59, -0.05]$	moderate	P	41/57	No (raw)
multivariable	second_derivative	$d^2X \longrightarrow dX$	2.23	9.31	$[1.11, 3.49]$	weak	S	41/57	No (raw)
multivariable	second_derivative	$d^2X \longrightarrow\!\!\!\!\shortmid\ dY$	-2.29	0.10	$[-3.53, -1.07]$	moderate	P,S	66/32	No (raw)
pairwise	second_derivative	$d^2X \longrightarrow\!\!\!\!\shortmid\ dY$	-1.98	0.14	$[-3.10, -0.99]$	moderate	P,S	66/32	No (raw)
multivariable	second_derivative	$d^2Y \longrightarrow dX$	3.30	27.20	$[1.69, 4.93]$	moderate	P,S	41/57	No (raw)
pairwise	second_derivative	$d^2Y \longrightarrow dX$	1.12	3.05	$[0.51, 1.74]$	weak	P,S	41/57	No (raw)
pairwise	second_derivative	$d^2Y \longrightarrow dY$	0.55	1.73	$[0.02, 1.14]$	strong	P	66/32	No (raw)

Notes: Mode is the raw `analysis_mode` value from `edges_combined`; Predictor Set is the raw `predictor_set` value. Rows are ordered by predictor term: X, Y, dX, dY, d^2X, and d^2Y. Support gives the cross-mode evidence for the same term–target pair (P=pairwise, S=sublevel/representation-specific multivariable, A=all predictors). ω is the posterior mean coefficient, OR is the odds ratio, and the 95% HDI is the highest-density interval used for edge retention. Numerical entries are rounded to at most two decimal places; OR values below 0.01 are shown in scientific notation to avoid rounding to zero. Class (0/1) reports the number of decreasing/increasing target-direction labels used in the fitted edge model. Model-level diagnostics such as usable sample size, number of predictors, $\hat{R}$, ESS ratio, and VIF are reported separately because they are indexed by fitted model rather than by retained edge. Jitter reports whether adaptive response or predictor jitter was required; No (raw) means the raw directional response was used without jitter. Yes (response) indicates adaptive jitter was applied to recover a usable target response, Yes (predictor) indicates predictor jitter was applied, and Yes (response + predictor) indicates both were required.

Supplementary Table 17: Model-level diagnostics for deterministic Lotka–Volterra CIRN fits. Rows are fitted model configurations rather than retained edges, preventing row-count confusion with the edge-list output.

Mode	Target	Pairwise predictor	Predictor set	Status	Class (0/1)	n	p	Min./p	$\hat{R}_{\max}$	$\mathrm{ESS}_{\min}$	Max VIF	VIF flag
multivariable	dX	–	original	completed	41/57	98	2	20.50	1.00	0.41	1.00	ok
multivariable	dX	–	first_derivative	completed	41/57	98	1	41.00	1.00	0.43	–	single_predictor
multivariable	dX	–	second_derivative	completed	41/57	98	2	20.50	1.00	0.24	1.51	ok
multivariable	dX	–	all_predictors	completed	41/57	98	5	8.20	1.00	0.37	783.42	severe_collinearity
pairwise	dX	X	original	completed	41/57	98	1	41.00	1.00	0.48	–	single_predictor
pairwise	dX	X	first_derivative	skipped_no_predictors	41/57	98	0	–	–	–	–	–
pairwise	dX	X	second_derivative	completed	41/57	98	1	41.00	1.00	0.42	–	single_predictor
pairwise	dX	X	all_predictors	completed	41/57	98	2	20.50	1.00	0.38	1.69	ok
pairwise	dX	Y	original	completed	41/57	98	1	41.00	1.00	0.40	–	single_predictor
pairwise	dX	Y	first_derivative	completed	41/57	98	1	41.00	1.00	0.43	–	single_predictor
pairwise	dX	Y	second_derivative	completed	41/57	98	1	41.00	1.00	0.42	–	single_predictor
pairwise	dX	Y	all_predictors	completed	41/57	98	3	13.67	1.00	0.28	1.99	ok
multivariable	dY	–	original	completed	66/32	98	2	16.00	1.00	0.44	1.00	ok
multivariable	dY	–	first_derivative	completed	66/32	98	1	32.00	1.00	0.46	–	single_predictor
multivariable	dY	–	second_derivative	completed	66/32	98	2	16.00	1.00	0.44	1.51	ok
multivariable	dY	–	all_predictors	completed	66/32	98	5	6.40	1.00	0.39	635.37	severe_collinearity
pairwise	dY	X	original	completed	66/32	98	1	32.00	1.00	0.37	–	single_predictor
pairwise	dY	X	first_derivative	completed	66/32	98	1	32.00	1.00	0.46	–	single_predictor
pairwise	dY	X	second_derivative	completed	66/32	98	1	32.00	1.00	0.40	–	single_predictor
pairwise	dY	X	all_predictors	completed	66/32	98	3	10.67	1.00	0.40	1.71	ok
pairwise	dY	Y	original	completed	66/32	98	1	32.00	1.00	0.45	–	single_predictor
pairwise	dY	Y	first_derivative	skipped_no_predictors	66/32	98	0	–	–	–	–	–
pairwise	dY	Y	second_derivative	completed	66/32	98	1	32.00	1.00	0.44	–	single_predictor
pairwise	dY	Y	all_predictors	completed	66/32	98	2	16.00	1.00	0.46	1.97	ok

Notes: This table is indexed by fitted CIRN model configuration, not by retained edge. Class (0/1) reports decreasing/increasing target-direction labels. Min./p is the minority class count divided by the number of predictors. $\hat{R}_{\max}$ and $\mathrm{ESS}_{\min}$ summarize MCMC convergence and effective-sample-size ratio. Max VIF and VIF flag summarize collinearity diagnostics within the fitted predictor design; VIF is undefined for single-predictor models.



Supplementary Figure 19: CIRN-based regulatory inference for a classical Lotka–Volterra predator–prey system, whose canonical mechanistic interaction structure is characterized by prey-supported predator activation ($X \longrightarrow Y$) and predator-induced prey inhibition ($Y \longrightarrow\!\!\!\!\shortmid\ X$). (A) Time-series representations of the simulated predator–prey dynamics over $t \in [0, 100]$, including the original state variables (X, Y), first derivatives (dX, dY), and second derivatives (d^2X, d^2Y). The original trajectories exhibit the characteristic oscillatory and phase-shifted behavior of the classical Lotka–Volterra system, whereas the derivative-based representations capture temporal rate-of-change and curvature dynamics. (B) Full CIRN-inferred directed and signed regulatory structure obtained from posterior-supported interactions recovered across fitted predictor representations. Nodes correspond to original-state, first-derivative, and second-derivative predictors, whereas directed edges denote inferred regulatory influences on target directional responses. **Black-bordered** nodes represent the target responses being modeled, while predictor representations are distinguished by node-outline color: orange for original-state predictors, blue for first-derivative predictors, and magenta for second-derivative predictors. Regulatory sign is encoded through edge color (green: activation or $\longrightarrow$; red: inhibition or $\longrightarrow\!\!\!\!\shortmid$), and edge thickness is proportional to the magnitude of the posterior mean edge weight $|\omega|$. (C) CIRN edge-consistency summary across pairwise fitting, representation-specific multivariable fitting (*sublevel*), and joint multivariable fitting using all candidate predictors (*all-predictors*). Each row corresponds to a candidate directed interaction, and each cell reports the signed posterior mean coefficient ω associated with a posterior-supported edge. Green cells indicate activating regulation, red cells indicate inhibitory regulation, and gray/blank cells indicate interactions not retained under the posterior HDI criterion. Color intensity is proportional to the magnitude of the corresponding posterior mean effect. (D–E) Posterior distributions of selected regression coefficients associated with the target responses dX and dY, respectively. Dashed vertical red lines indicate the null effect, and highest density intervals (HDIs) summarize posterior uncertainty. Coefficients whose HDIs exclude zero are retained as credible regulatory interactions, whereas coefficients whose HDIs contain zero are treated as uncertain and excluded from the inferred network. Node sizes are adjusted solely for visualization and do not correspond to any quantitative network property.

sistently associated with increased predator growth direction (dY), while the predator state Y is consistently associated with decreased prey growth direction (dX). These two dependencies are among the strongest retained effects and are supported across pairwise, sublevel/multivariable, and all-predictor analyses, indicating that they are not artifacts of a single predictor set or fitting mode.

The model diagnostics support a cautious interpretation of the retained edges. All completed fits used 98 usable observations, with class counts of 41/57 for dX and 66/32 for dY, and the maximum $\hat{R}$ values round to 1.00, indicating stable MCMC convergence. Effective-sample-size ratios were

adequate for reporting posterior summaries, although some maximal models showed lower values. The severe VIF values in the multivariable all-predictor fits are expected in this deterministic oscillatory system because state, velocity, and curvature predictors are strongly coupled. For this reason, the all-predictor rows are best interpreted as stress-test evidence for sign and support consistency, whereas the main biological interpretation should emphasize effects that remain stable across the pairwise and sublevel/multivariable representations. All retained edge rows are marked as *No (raw)* under Jitter, indicating that the raw directional-response coding was sufficient and no adaptive response or predictor jitter was required for these reported fits.

The additional interactions in Supplementary Figure 19B should be interpreted differently from the two canonical predator–prey relationships. The Lotka–Volterra equations contain only the ecological coupling terms βXY and δXY at the species level. Consequently, derivative- and curvature-mediated edges do not imply additional biological entities or independent ecological mechanisms. Instead, they reflect statistical and dynamical dependencies among alternative temporal representations of the same oscillatory process. Because predator–prey cycles involve tightly coupled changes in state, velocity, and curvature, information about future directional responses can be encoded not only in population abundance but also in local rate-of-change and acceleration patterns.

Several retained interactions support this temporal interpretation. The positive effects associated with $d^2X \longrightarrow dX$ and $d^2Y \longrightarrow dX$ suggest that curvature contains information about impending changes in prey growth direction, whereas the negative interaction $d^2X \longdashv dY$ indicates that acceleration patterns in the prey trajectory contribute to predicting subsequent predator directional change. These relationships are consistent with the organization of oscillatory systems, where peaks, troughs, and turning points emerge before changes in growth direction are fully expressed in the state variables. Thus, second-derivative predictors may act as early indicators of transitions between growth and decline phases within the predator–prey cycle.

The posterior summaries in Supplementary Figure 19D–E further reinforce this interpretation. For the prey directional response dX, the posterior distributions associated with $Y \longdashv dX$ and $X \longdashv dX$ remain negative, indicating credible inhibitory effects. Conversely, the curvature-derived predictors $d^2X \longrightarrow dX$ and $d^2Y \longrightarrow dX$ exhibit positive posterior means, suggesting that local acceleration patterns carry predictive information about future prey growth direction. For the predator directional response dY, the dominant state-level interaction $X \longrightarrow dY$ remains strongly positive, whereas $Y \longdashv dY$ and $d^2X \longdashv dY$ exhibit negative posterior means. These results indicate that state-level and higher-order temporal representations both contribute to prediction, although their relative importance differs across targets.

A final pattern concerns how network complexity changes as predictor representations are integrated. Pairwise fitting recovered several derivative- and curvature-mediated interactions, while the sublevel formulation retained a subset of these effects after accounting for competing predictors within the same representation class. In contrast, the all-predictor formulation produced a sparser network dominated by the two canonical state-level interactions. Rather than indicating loss of information, this sparsification suggests that many secondary temporal signals are partially redundant once state, velocity, and curvature predictors compete in the same model. The persistence of $Y \longdashv dX$ and $X \longrightarrow dY$ under this more stringent formulation provides additional evidence that these are the fundamental structural components recovered by CIRN.

These findings highlight the distinction between structural and mechanistic recovery. CIRN does not reconstruct the explicit nonlinear terms βXY and δXY, nor does it estimate the Lotka–Volterra parameters. Instead, it recovers the signed directional constraints implied by those terms: predator abundance decreases the likelihood of continued prey increase, and prey abundance increases the likelihood of predator increase. The deterministic Lotka–Volterra experiment therefore demonstrates that CIRN can recover the expected predator–prey dependency architecture while

also identifying representation-specific temporal signatures associated with oscillatory dynamics. This provides a probabilistic and interpretable summary of both dominant regulatory structure and higher-order temporal organization from observational time-series data.

C. Stochastic Predator–Prey Dynamics with Demographic Noise

To evaluate the robustness of CIRN under stochastic dynamics, we next considered a stochastic Lotka–Volterra predator–prey system obtained by extending the deterministic model of the previous section with multiplicative demographic noise. The dynamics were generated using the classical Langevin-type stochastic differential equations

$$dX = (\alpha - \beta Y)X\,dt + \sqrt{(\alpha + \beta Y)X}\,dW_X,$$
$$dY = (\delta X - \gamma)Y\,dt + \sqrt{(\delta X + \gamma)Y}\,dW_Y,$$

with initial conditions $X(0) = Y(0) = 2.0$, parameter values $\alpha = 0.29$, $\beta = 0.26$, $\gamma = 0.22$, and $\delta = 0.25$, and simulated using the Euler–Maruyama scheme. Importantly, the introduction of stochasticity does not alter the underlying biological interaction structure. Prey abundance continues to promote predator growth through the positive interaction term δXY, whereas predator abundance suppresses prey growth through the negative interaction term $-\beta XY$. Consequently, the canonical regulatory architecture remains characterized by prey-supported predator activation ($X \longrightarrow Y$) and predator-induced prey inhibition ($Y \longrightarrow\!\!\!\!\shortmid\ X$), allowing a direct comparison with the deterministic case.

As shown in Supplementary Figure 20A, the stochastic system retains the oscillatory predator–prey organization observed in the deterministic model but exhibits substantial local variability in both amplitude and timing. Fluctuations introduced by the stochastic forcing propagate through the derivative representations, producing increasingly irregular patterns in the first- and second-derivative trajectories. This behavior is expected because differentiation amplifies local variation, causing stochastic perturbations that are only moderately visible in the state variables to become more pronounced in the velocity and curvature representations. Consequently, the stochastic system provides a more demanding test of CIRN than the deterministic model, since informative regulatory signals must be distinguished from noise-induced temporal fluctuations.

Despite this additional complexity, CIRN successfully recovered the dominant predator–prey interaction structure. The full inferred regulatory network shown in Supplementary Figure 20B preserves the two principal state-level interactions identified in the deterministic system. Predator abundance exerts a negative influence on the prey directional response, whereas prey abundance exerts a positive influence on the predator directional response. These interactions correspond directly to the biological interpretation of the stochastic Lotka–Volterra equations and therefore demonstrate that CIRN remains capable of recovering meaningful directed and signed regulatory relationships even when the observed trajectories are subject to stochastic perturbations.

The robustness of these interactions is further evident in the edge-consistency summary (Supplementary Figure 20C). The inhibitory interaction $Y \longrightarrow\!\!\!\!\shortmid\ dX$ was retained across the pairwise, sublevel, and all-predictors configurations with posterior mean coefficients of $\omega = -3.32$, -3.18, and -3.46, respectively. Similarly, the activating interaction $X \longrightarrow dY$ remained consistently supported with posterior mean coefficients of $\omega = 3.83$, 3.80, and 4.17. Although the magnitudes are somewhat smaller than those observed in the deterministic system, the preservation of both sign and effect size across substantially different model formulations indicates that these edges represent stable structural features of the underlying dynamics rather than artifacts of a particular predictor representation. The persistence of the canonical interactions in the all-predictors configu-

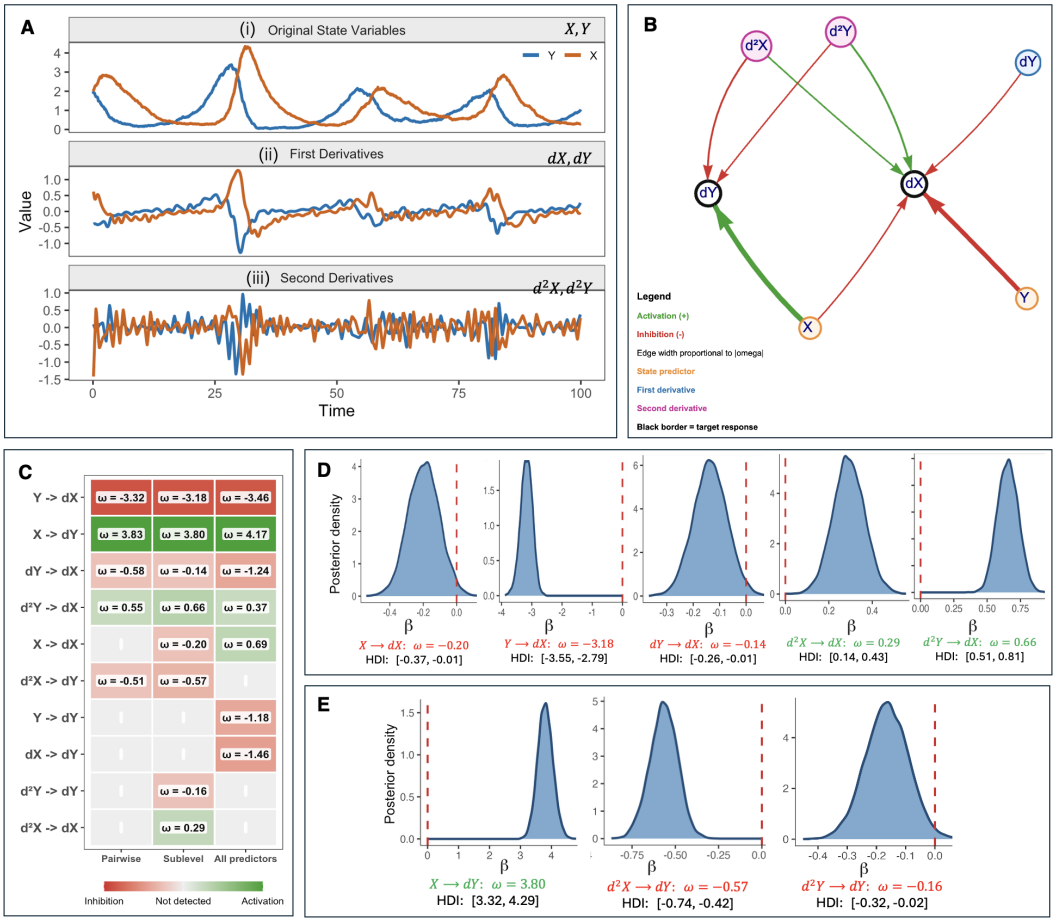
ration is especially notable because this formulation simultaneously evaluates state-, velocity-, and curvature-based predictors, thereby providing the most stringent test of edge robustness.

Comparison with the deterministic system reveals an additional effect of stochasticity. Whereas the deterministic model was dominated by a relatively compact set of highly stable interactions, the stochastic system exhibits a richer collection of derivative- and curvature-mediated edges. Examples include the activating interactions $d^2Y \longrightarrow dX$ and $d^2X \longrightarrow dX$, together with the inhibitory interactions $d^2X \longmapsto dY$ and $d^2Y \longmapsto dY$. These interactions should not be interpreted as additional biological mechanisms absent from the governing equations. Rather, they reflect how stochastic fluctuations redistribute information across multiple temporal representations. Because noise perturbs not only the state variables but also their local rates of change and curvature, derivative-based predictors become increasingly informative regarding future directional changes in the system. The resulting edges therefore represent temporal signatures of the stochastic dynamics rather than new mechanistic predator–prey relationships.

The posterior distributions shown in Supplementary Figure 20D–E further support this interpretation. The dominant state-level effects remain clearly separated from zero, with narrow highest density intervals excluding the null effect. At the same time, several derivative- and curvature-mediated interactions also exhibit credible posterior support. The coexistence of these effects suggests that stochastic perturbations introduce regulatory information that is distributed across multiple temporal scales. Whereas state variables continue to capture the principal biological interactions, derivative and curvature representations encode additional information about the local dynamical response of the system to stochastic fluctuations.

These results highlight an important distinction between CIRN and many existing network-inference approaches. Correlation- and association-based methods would primarily detect the persistent statistical dependence between predator and prey populations, but they do not naturally distinguish the opposing biological roles of predator-induced suppression and prey-supported growth. Likewise, methods based on state-space reconstruction can identify causal coupling but generally do not recover explicit regulatory sign. In contrast, CIRN simultaneously preserves directionality, regulatory sign, and posterior uncertainty while remaining agnostic to the explicit functional form of the governing equations. The stochastic Lotka–Volterra system therefore demonstrates that the framework is capable of recovering biologically interpretable regulatory structure even when the observed dynamics deviate substantially from ideal deterministic behavior.

The stochastic predator–prey experiment shows that CIRN is not merely recovering geometric regularities present in smooth oscillatory trajectories. Instead, the framework remains capable of identifying the dominant signed dependency architecture after the introduction of demographic noise. Although stochastic perturbations generate additional derivative- and curvature-mediated regulatory signatures, the principal predator–prey interaction pattern remains stable across inference configurations. This finding suggests that CIRN possesses a degree of robustness to stochastic variability while simultaneously exploiting information contained in system state, velocity, and curvature, thereby providing an interpretable probabilistic representation of regulatory structure in both deterministic and stochastic dynamical systems.



Supplementary Figure 20: CIRN-based regulatory inference for a stochastic Lotka–Volterra predator–prey system with demographic noise. This system extends the deterministic predator–prey model in Supplementary Figure 19 by incorporating stochastic demographic fluctuations while preserving the canonical interaction structure characterized by prey-supported predator activation ($X \longrightarrow Y$) and predator-induced prey inhibition ($Y \longrightarrow\!\!\!\!\!\!\cdot\ \ X$). (A) Time-series representations of the stochastic dynamics over $t \in [0, 100]$, including the original state variables (X, Y), first derivatives (dX, dY), and second derivatives (d^2X, d^2Y). Stochastic perturbations introduce irregular fluctuations and increased temporal variability relative to the deterministic system, particularly in the derivative-based representations. (B) Full CIRN-inferred directed and signed regulatory network obtained from all posterior-supported interactions across fitted predictor representations. Predictor representations are distinguished by node-outline color (orange: original state; blue: first derivative; magenta: second derivative), whereas black-bordered nodes denote target responses. Edge color indicates regulatory sign (green: activation; red: inhibition), and edge thickness is proportional to the magnitude of the posterior mean effect size $|\omega|$. (C) Edge-consistency summary across pairwise, sublevel, and all-predictors CIRN configurations. Each row corresponds to a candidate directed interaction and each cell reports the signed posterior mean coefficient ω associated with a posterior-supported edge. Green cells indicate activating effects, red cells indicate inhibitory effects, and gray/blank cells indicate interactions not retained under the posterior HDI criterion. Color intensity is proportional to the magnitude of the corresponding posterior mean effect. (D–E) Posterior distributions of representative coefficients associated with the target responses dX and dY, respectively. Dashed vertical red lines indicate the null effect ($\beta = 0$), and highest density intervals (HDIs) summarize posterior uncertainty. The figure illustrates that CIRN retains the dominant predator–prey regulatory organization while simultaneously identifying derivative- and curvature-mediated temporal dependencies under stochastic nonlinear dynamics. Node sizes are adjusted solely for visualization purposes and do not correspond to any quantitative network property.

D. Temporal Reorganization of Regulatory Structure under Lagged Interactions

To examine whether the inferred regulatory architecture remains stable under temporal displacement, CIRN was applied to the classical Lotka–Volterra predator–prey system across increasing lag units incorporated directly within the algorithmic predictor–response alignment. This analysis extends the preceding deterministic and stochastic predator–prey experiments by asking not

only whether CIRN can recover the canonical signed interactions, but also how those interactions reorganize when predictors are evaluated at progressively earlier time points relative to future directional responses. Because predator–prey oscillations are inherently phase-shifted, lag variation provides a stringent test of whether CIRN can distinguish persistent regulatory constraints from lag-dependent changes in sign, edge support, and temporal representation. The resulting heatmap in Supplementary Figure 21 therefore serves as a lag-resolved map of regulatory rewiring across state-, velocity-, and curvature-based predictors.

The lag-dependent analysis in Supplementary Figure 21 reveals that CIRN-inferred regulatory structure is highly sensitive to temporal alignment, even when the underlying dynamical mechanism remains fixed. As the lag unit increases, posterior-supported interactions do not merely weaken uniformly; instead, they undergo structured sign reversals, disappearance, re-emergence, and redistribution across original-state, first-derivative, and second-derivative predictor representations. This behavior indicates that temporal displacement changes the inferable regulatory signature of the system. In an oscillatory predator–prey model, the predictive meaning of a variable depends strongly on its phase position relative to the future directional response being modeled. CIRN captures this dependence as lag-conditioned rewiring of the inferred regulatory network.

It is important to emphasize that the rewiring observed in Supplementary Figure 21 does not imply that the underlying Lotka–Volterra mechanism itself changes across lag units. The governing equations remain fixed throughout the analysis, with predator abundance suppressing prey growth through the negative interaction term $-\beta XY$ and prey abundance supporting predator growth through the positive interaction term δXY. Rather, the rewiring occurs at the level of the inferred temporal regulatory representation. As the predictor–response alignment changes, CIRN detects different signed dependencies because each lag samples a different phase relationship among abundance, velocity, curvature, and future directional change. Thus, lag-dependent rewiring should be interpreted as a change in the observable and inferable regulatory signature of the system, not as a change in the underlying ecological law.

At short lags, CIRN recovers the expected predator–prey dependency structure. In the target-dX block, predator abundance Y exerts a strong inhibitory effect on prey directional change, with posterior mean edge weights decreasing from $\omega = -5.85$ at lag 1 to $\omega = -5.40$, -4.60, and -3.58 at lags 2–4. This is consistent with the canonical Lotka–Volterra term $-\beta XY$, in which predator abundance suppresses prey growth. Similarly, in the target-dY block, prey abundance X exerts a strong activating effect on predator directional change, with $\omega = 5.57$ at lag 1, followed by $\omega = 4.53$, 3.22, and 1.98 at lags 2–4. This corresponds to the positive interaction term δXY, in which prey availability supports predator growth. Thus, under near-immediate temporal alignment, the CIRN-inferred architecture closely reflects the biological interaction structure encoded in the governing equations: $Y \longrightarrow\!\!\!\!\!\!\mid\ dX$ and $X \longrightarrow dY$.

The behavior of the canonical predator-to-prey interaction under increasing lag provides one of the clearest examples of lag-dependent network rewiring. The inhibitory effect $Y \longrightarrow\!\!\!\!\!\!\mid\ dX$ weakens from $\omega = -5.85$ at lag 1 to $\omega = -1.13$ at lag 6, disappears at lag 7, and then reappears with the opposite sign at lag 8 ($\omega = 1.96$). It continues as a strong activating association through the middle lag regime, increasing to $\omega = 3.74$ at lag 9, $\omega = 4.49$ at lag 10, and reaching a maximum of $\omega = 6.72$ at lag 13. The effect then gradually declines, remains activating across subsequent lags, and disappears again at lags 21–22. At later lags, the interaction reverses back to inhibition, with $\omega = -1.56$, -3.39, and -4.09 at lags 23–25. This sequence—inhibition, disappearance, activation, disappearance, and return to inhibition—is a concrete example of regulatory rewiring induced by temporal displacement. Biologically, this does not mean that predators become beneficial to prey at intermediate lags. Rather, predator abundance observed at different lag positions indexes different phases of the predator–prey cycle, ranging from immediate prey suppression to delayed

prey recovery after predator-driven depletion.

A complementary pattern appears for the prey-to-predator interaction $X \longrightarrow dY$. At short lags, prey abundance strongly activates predator directional change, with $\omega = 5.57$ at lag 1. This activation weakens across lags 2–5 and disappears at lag 6. The interaction then reverses sign, becoming inhibitory from lag 7 onward, with increasingly strong negative effects from $\omega = -0.91$ at lag 7 to $\omega = -2.89$ at lag 9, $\omega = -5.53$ at lag 12, and $\omega = -6.35$ at lag 15. After this mid-lag inhibitory regime, the effect weakens, disappears at lag 21, and re-emerges as activation at later lags, increasing from $\omega = 1.05$ at lag 22 to $\omega = 2.14$, 3.14, and 4.05 at lags 23–25. This trajectory mirrors the behavior of $Y \longrightarrow\!\!\!\!\!\!\mid\ dX$ and illustrates how the canonical prey-supported predator growth relationship is transformed by temporal offset. In ecological terms, prey abundance at short lags directly supports predator growth, but prey abundance observed too far in the past may instead mark a preceding phase of the cycle that is followed by predator saturation, decline, or delayed negative feedback.

The gradual increase, peak, and subsequent decline of posterior mean edge weights across lag units suggest that each retained interaction has an optimal temporal window of regulatory informativeness. As the lag increases, a predictor may become progressively better aligned with the future directional response, producing increasing values of $|\omega|$. The peak of $|\omega|$ therefore identifies the lag regime in which that predictor carries the strongest sign-consistent information for the target response. Beyond this window, the effect weakens as the predictor becomes increasingly phase-misaligned with the response or as regulatory information shifts to alternative state-, velocity-, or curvature-based representations. In the Lotka–Volterra system, this pattern is especially visible for the canonical interactions. The transition of $Y \longrightarrow\!\!\!\!\!\!\mid\ dX$ from strong inhibition at short lags to a positive peak around the intermediate lag regime indicates that past predator abundance changes its predictive meaning as the predator–prey cycle advances. Similarly, the transition of $X \longrightarrow dY$ from short-lag activation to a strong negative intermediate-lag regime indicates that prey abundance observed farther in the past may correspond to a phase preceding predator saturation or decline rather than immediate predator growth.

When an interaction transitions from one regulatory sign to the opposite sign, the corresponding peaks should be interpreted as sign-specific phase-alignment maxima. A negative peak indicates the lag at which the predictor is most strongly aligned with future decline of the target response, whereas a positive peak indicates the lag at which the same predictor is most strongly aligned with future increase. Thus, the transition from inhibition to activation, or from activation to inhibition, does not imply a change in the underlying ecological mechanism. Rather, it reflects movement across different phases of the predator–prey cycle, where the same variable can carry different predictive information depending on its temporal position relative to the response.

These sign reversals are a natural consequence of phase-shifted oscillatory dynamics. In predator–prey cycles, a high value of a state variable does not have a single predictive meaning across all temporal offsets. High prey abundance at a short lag may precede predator growth, yielding an activating effect on dY. At a longer lag, however, the same prey signal may correspond to an earlier phase that is followed by predator saturation or decline, producing an apparent inhibitory association with future predator directional change. Similarly, high predator abundance may immediately predict prey decline at short lags, but at later offsets it may index a phase after prey depletion, where prey recovery is beginning. CIRN captures these phase-dependent changes as sign transitions in posterior-supported regulatory edges.

The state-level self-effects also exhibit strong lag-dependent transitions. For the prey response, $X \to dX$ is inhibitory at short lags, strengthening from $\omega = -1.29$ at lag 1 to $\omega = -5.98$ at lag 7. The interaction weakens thereafter, disappears at lag 13, and reverses to activation from lag 14 onward, reaching $\omega = 3.33$ at lag 16, $\omega = 5.10$ at lag 18, and a maximum of $\omega = 6.30$ at lag 21 before

declining to $\omega = 4.08$ at lag 25. For the predator response, $Y \to dY$ follows an analogous pattern: it is inhibitory from lag 1 ($\omega = -1.64$) through lag 14 ($\omega = -0.78$), disappears at lag 15, and then becomes activating from lag 16 onward, reaching $\omega = 3.18$ at lag 18 and $\omega = 5.44$ at lag 21 before gradually weakening toward lag 25 ($\omega = 3.18$). These transitions are characteristic of oscillatory dynamics. A high population level at one phase may predict imminent decline because the system is approaching or has passed a peak, whereas the same variable observed at a different temporal offset may predict future increase because the system is positioned near a recovery phase. CIRN therefore captures not only cross-species regulation but also phase-conditioned temporal memory within each population trajectory.

Derivative-mediated interactions provide further evidence that regulatory information is redistributed across temporal representations as lag increases. In the target-dX block, the velocity predictor dY initially exerts an inhibitory effect on prey directional change, with $\omega = -0.60$ at lag 2 and a much stronger $\omega = -6.18$ at lag 7. This interaction then disappears across several intermediate lags and re-emerges as activating from lag 16 ($\omega = 0.52$) onward, increasing to $\omega = 0.86$ at lag 18 and $\omega = 6.13$ at lag 23 before weakening again. The corresponding velocity interaction in the target-dY block, $dX \to dY$, shows the opposite temporal organization: it is activating at short lags, reaching $\omega = 6.28$ at lag 4, disappears across lags 12–17, and then reappears as inhibitory at later lags, with a strong negative effect of $\omega = -5.70$ at lag 21. These paired velocity-level sign reversals indicate that the rates of change of prey and predator populations contain phase-specific information about subsequent directional responses. In ecological terms, whether increasing prey predicts future predator growth or decline depends on the temporal distance from the predator response and the phase of the predator–prey cycle.

Curvature-based predictors exhibit their own structured patterns of lag-dependent rewiring. For the prey response dX, $d^2X \to dX$ is activating at short lags, with effects ranging from $\omega = 2.23$ to $\omega = 2.90$ across lags 1–4, disappears at lag 10, and becomes inhibitory from lag 11 ($\omega = -1.68$) onward, with a peak negative effect of $\omega = -2.97$ at lag 13. Similarly, $d^2Y \to dX$ is activating at early lags, with $\omega = 3.30$ at lag 1 and $\omega = 3.39$ at lag 2, disappears at lag 8, and then becomes inhibitory from lag 9 ($\omega = -1.68$) through lag 23 ($\omega = -1.19$). For the predator response dY, $d^2X \to dY$ begins as inhibitory at lags 1–2 (resp., $\omega = -2.29, -1.36$), becomes activating from lag 4 ($\omega = 1.66$) through lag 19 ($\omega = 1.61$), and then reverses again to inhibition from lag 21 ($\omega = -1.16$) onward. The predictor $d^2Y \to dY$ is mostly activating from lags 2–17, disappears at lag 18, and becomes inhibitory across lags 19–24. These curvature-level transitions show that acceleration information is not merely auxiliary; it becomes a major carrier of regulatory signal in particular lag regimes. Curvature predictors appear to encode information about turning points, acceleration toward peaks, and deceleration toward troughs, which are central features of predator–prey oscillations.

The simultaneous involvement of state, velocity, and curvature predictors indicates that lag-dependent rewiring is not confined to isolated edges. Rather, it occurs as a coordinated reorganization of the inferred regulatory architecture. Canonical state-level interactions dominate at short lags, while derivative- and curvature-mediated interactions become more prominent as the predictor–response alignment moves through later phases of the oscillation. The transitions in $Y \longrightarrow\!\!\!\!\!\mid\ dX$, $X \longrightarrow dY$, $X \to dX$, and $Y \to dY$ occur in a mutually structured manner, with short-lag canonical regulation giving way to intermediate-lag sign inversion and late-lag partial recurrence. This coordinated behavior indicates that the inferred rewiring is not random variation in posterior estimates, but a systematic consequence of moving the predictor–response alignment through the phase structure of the oscillator. In this sense, Supplementary Figure 21 does not simply display a collection of changing coefficients. It reveals a lag-resolved regulatory landscape in which different temporal representations become informative at different offsets. Each row can

therefore be interpreted as a distinct temporal view of the same dynamical system, with the inferred network topology changing as CIRN samples different phase relationships between predictors and responses.

Blank cells in Supplementary Figure 21 are also informative. They indicate lag regimes in which a candidate interaction was not retained under the posterior credibility criterion, but they should not be interpreted as definitive biological absence. Instead, they mark temporal offsets where a predictor no longer contributes a stable, distinguishable, or credible signal for the target response. For example, the disappearance of $Y \longrightarrow\!\!\!\!\!\mid\ dX$ at lag 7 occurs precisely between a short-lag inhibitory regime and a mid-lag activating regime, suggesting a transition zone where the predator signal becomes phase-misaligned with prey directional change. Similarly, the disappearance of $X \longrightarrow dY$ at lags 6 and 21 occurs between activating and inhibitory phases of the prey-to-predator relationship. These gaps therefore represent intervals of regulatory ambiguity or transition, where information may be attenuated, redistributed to derivative or curvature predictors, or made indistinguishable by competing temporally correlated variables.

Viewed as a whole, the heatmap reveals three major lag regimes. The early-lag regime preserves the canonical Lotka–Volterra architecture, with $Y \longrightarrow\!\!\!\!\!\mid\ dX$ and $X \longrightarrow dY$ strongly supported. The intermediate-lag regime is characterized by widespread sign reversal, in which state-level interactions often switch polarity and derivative or curvature predictors assume stronger explanatory roles. The late-lag regime exhibits partial recurrence of the original sign structure, consistent with the cyclic nature of the underlying dynamics. This progression suggests that the inferred network undergoes lag-conditioned rewiring as the temporal offset moves through different phases of the predator–prey cycle.

Ecologically, this result is consistent with the delayed and phase-structured nature of predator–prey systems. Predator abundance does not influence prey growth only at a single instant; its observed value is embedded within a cycle of pursuit, depletion, predator decline, and prey recovery. Likewise, prey abundance may signal immediate resource availability for predators at short lags, but at longer lags it may correspond to a phase that precedes predator saturation or decline. The alternating signs observed in the heatmap therefore reflect the ecological reality that the predictive meaning of abundance, growth rate, and curvature depends on timing. In this sense, lag-dependent CIRN analysis provides a way to distinguish direct near-term regulatory effects from delayed, phase-conditioned, and representation-dependent predictive relationships.

These findings strengthen the central argument of the CIRN framework. Regulatory networks inferred from time-series data should not always be treated as fixed summaries of pairwise dependence. In nonlinear oscillatory systems, the inferred architecture may change as a function of temporal alignment, producing systematic rewiring across lag regimes. CIRN makes this behavior explicit by evaluating signed and directed interactions across state, velocity, and curvature representations under varying temporal displacement. The resulting patterns show that the framework can recover canonical ecological interactions at biologically meaningful short lags while also revealing how regulatory information migrates across temporal representations as the system evolves. Thus, Supplementary Figure 21 demonstrates that CIRN is capable not only of reconstructing signed interaction structure, but also of exposing the temporal organization and lag-dependent rewiring of regulatory influence in oscillatory dynamical systems.

The lagged analysis therefore extends CIRN from static signed network reconstruction toward a temporally resolved view of regulatory inference. Instead of producing a single graph that summarizes all interactions, the framework exposes how regulatory evidence changes as the temporal separation between putative cause and response is varied. This is particularly relevant for biological, ecological, and environmental systems, where delayed responses, feedback cycles, and phase shifts are common. In such systems, a single inferred network may obscure important temporal

structure, whereas the lag-dependent CIRN representation distinguishes persistent regulatory constraints from transient, phase-specific, and representation-dependent interactions. Consequently, lag-dependent CIRN analysis provides a principled way to evaluate whether an inferred regulatory interaction is stable across temporal scales, restricted to specific phase regimes, or redistributed across alternative temporal representations. Temporal lag should therefore not be regarded merely as a technical parameter of the inference procedure, but as a biologically and dynamically meaningful quantity capable of shaping the inferred architecture of regulatory systems.

No. of Lag Units	Probable Edge Weight (ω) & Sign — Activation (+) — Inhibition (−)									
	Target X (i.e., $\longrightarrow dX$)					Target Y (i.e., $\longrightarrow dY$)				
	X	Y	dY	d^2X	d^2Y	X	Y	dX	d^2X	d^2Y
1	$\omega = -1.29$	$\omega = -5.85$		$\omega = 2.23$	$\omega = 3.30$	$\omega = 5.57$	$\omega = -1.64$	$\omega = 0.49$	$\omega = -2.29$	
2	$\omega = -2.54$	$\omega = -5.40$	$\omega = -0.60$	$\omega = 2.78$	$\omega = 3.39$	$\omega = 4.53$	$\omega = -3.44$	$\omega = 1.63$	$\omega = -1.36$	$\omega = 1.63$
3	$\omega = -3.69$	$\omega = -4.60$	$\omega = -0.97$	$\omega = 2.90$	$\omega = 3.31$	$\omega = 3.22$	$\omega = -4.82$	$\omega = 3.61$		$\omega = 3.72$
4	$\omega = -4.69$	$\omega = -3.58$	$\omega = -1.49$	$\omega = 2.80$	$\omega = 3.08$	$\omega = 1.98$	$\omega = -5.75$	$\omega = 6.28$	$\omega = 1.66$	$\omega = 4.02$
5	$\omega = -5.45$	$\omega = -2.40$	$\omega = -2.42$	$\omega = 2.56$	$\omega = 2.64$	$\omega = 0.96$	$\omega = -6.20$	$\omega = 5.51$	$\omega = 2.40$	$\omega = 3.44$
6	$\omega = -5.93$	$\omega = -1.13$	$\omega = -4.40$	$\omega = 2.36$	$\omega = 2.07$		$\omega = -6.32$	$\omega = 3.43$	$\omega = 2.92$	$\omega = 2.98$
7	$\omega = -5.98$		$\omega = -6.18$	$\omega = 2.36$	$\omega = 1.35$	$\omega = -0.91$	$\omega = -6.26$	$\omega = 2.14$	$\omega = 2.59$	$\omega = 2.42$
8	$\omega = -5.63$	$\omega = 1.96$	$\omega = -3.50$	$\omega = 2.26$		$\omega = -1.86$	$\omega = -5.94$	$\omega = 1.47$	$\omega = 1.98$	$\omega = 1.90$
9	$\omega = -4.46$	$\omega = 3.74$	$\omega = -1.58$	$\omega = 1.32$	$\omega = -1.68$	$\omega = -2.89$	$\omega = -5.43$	$\omega = 1.05$	$\omega = 1.52$	$\omega = 1.59$
10	$\omega = -3.02$	$\omega = 4.49$	$\omega = -0.89$		$\omega = -3.65$	$\omega = -3.94$	$\omega = -4.74$	$\omega = 0.79$	$\omega = 1.30$	$\omega = 1.48$
11	$\omega = -1.77$	$\omega = 5.76$	$\omega = -0.46$	$\omega = -1.68$	$\omega = -3.95$	$\omega = -4.84$	$\omega = -3.80$	$\omega = 0.60$	$\omega = 1.21$	$\omega = 1.48$
12	$\omega = -0.74$	$\omega = 6.20$		$\omega = -2.46$	$\omega = -3.45$	$\omega = -5.53$	$\omega = -2.75$		$\omega = 1.21$	$\omega = 1.57$
13		$\omega = 6.72$		$\omega = -2.97$	$\omega = -3.04$	$\omega = -6.01$	$\omega = -1.73$		$\omega = 1.29$	$\omega = 1.70$
14	$\omega = 1.17$	$\omega = 6.07$		$\omega = -2.70$	$\omega = -2.45$	$\omega = -6.28$	$\omega = -0.78$		$\omega = 1.42$	$\omega = 1.79$
15	$\omega = 2.20$	$\omega = 5.72$		$\omega = -2.03$	$\omega = -1.80$	$\omega = -6.35$			$\omega = 1.55$	$\omega = 1.71$
16	$\omega = 3.33$	$\omega = 5.13$	$\omega = 0.52$	$\omega = -1.61$	$\omega = -1.46$	$\omega = -6.17$	$\omega = 0.97$		$\omega = 1.71$	$\omega = 1.41$
17	$\omega = 4.31$	$\omega = 4.28$	$\omega = 0.67$	$\omega = -1.35$	$\omega = -1.35$	$\omega = -5.80$	$\omega = 1.97$		$\omega = 2.06$	$\omega = 0.94$
18	$\omega = 5.10$	$\omega = 3.33$	$\omega = 0.86$	$\omega = -1.18$	$\omega = -1.37$	$\omega = -4.81$	$\omega = 3.18$	$\omega = -0.96$	$\omega = 2.28$	
19	$\omega = 5.75$	$\omega = 2.38$	$\omega = 1.17$	$\omega = -1.01$	$\omega = -1.37$	$\omega = -3.33$	$\omega = 4.54$	$\omega = -1.92$	$\omega = 1.61$	$\omega = -1.31$
20	$\omega = 6.11$	$\omega = 1.39$	$\omega = 1.70$	$\omega = -1.03$	$\omega = -1.53$	$\omega = -1.64$	$\omega = 5.23$	$\omega = -3.50$		$\omega = -2.75$
21	$\omega = 6.30$		$\omega = 2.65$	$\omega = -1.17$	$\omega = -1.66$		$\omega = 5.44$	$\omega = -5.70$	$\omega = -1.16$	$\omega = -3.25$
22	$\omega = 6.26$		$\omega = 4.28$	$\omega = -1.35$	$\omega = -1.57$	$\omega = 1.05$	$\omega = 5.28$	$\omega = -5.07$	$\omega = -2.01$	$\omega = -3.51$
23	$\omega = 5.98$	$\omega = -1.56$	$\omega = 6.13$	$\omega = -1.60$	$\omega = -1.19$	$\omega = 2.14$	$\omega = 4.81$	$\omega = -3.07$	$\omega = -2.67$	$\omega = -3.59$
24	$\omega = 5.30$	$\omega = -3.39$	$\omega = 4.75$	$\omega = -2.20$		$\omega = 3.14$	$\omega = 4.09$	$\omega = -1.88$	$\omega = -2.91$	$\omega = -3.52$
25	$\omega = 4.08$	$\omega = -4.09$	$\omega = 2.08$	$\omega = -1.99$		$\omega = 4.05$	$\omega = 3.18$	$\omega = -1.24$	$\omega = -2.93$	

Supplementary Figure 21: Lag-dependent rewiring of CIRN-inferred regulatory architecture in the classical Lotka–Volterra predator–prey system. The figure summarizes posterior-supported regulatory interactions inferred by CIRN across increasing lag units incorporated directly within the predictor–response alignment procedure of the framework. Rows correspond to CIRN-specified lag units, whereas columns correspond to candidate predictor representations grouped according to the target directional responses dX and dY. For a specified lag unit k, CIRN internally aligns predictors observed at earlier time points with future directional responses, e.g., $Y(t-k) \longrightarrow dX(t)$ or $X(t-k) \longrightarrow dY(t)$, thereby preserving the temporal ordering assumption that putative regulatory causes precede their associated effects. Each non-empty cell reports the posterior mean edge weight $\omega = \mathbb{E}[\beta \mid \text{data}]$ for an interaction whose highest density interval excludes zero. Green cells denote activating effects ($\omega > 0$), red cells denote inhibitory effects ($\omega < 0$), and blank cells indicate interactions that were not retained under the posterior credibility criterion at the corresponding lag unit. Such omissions do not necessarily imply the complete absence of regulatory dependence; rather, they indicate lag regimes in which the corresponding predictor no longer provides a stable or uniquely identifiable contribution to the target response, potentially because of phase misalignment, attenuation of predictive signal, redistribution of regulatory information across alternative temporal representations, or competition among correlated state-, velocity-, and curvature-based predictors. Several regulatory relationships persist across broad lag intervals, whereas others undergo sign reversals, temporary disappearance, or subsequent re-emergence as temporal displacement increases. The canonical predator–prey dependencies remain evident throughout much of the lag spectrum, with predator-related predictors exhibiting predominantly inhibitory effects on prey directional change and prey-related predictors exhibiting predominantly activating effects on predator directional change. At the same time, state-, first-derivative-, and second-derivative predictors display distinct lag-dependent transition patterns, revealing systematic rewiring of the inferred network topology as regulatory support shifts among alternative temporal representations. These transitions reflect the phase-shifted oscillatory geometry of the Lotka–Volterra dynamics, in which the apparent influence of a predictor depends strongly on the temporal offset at which the interaction is evaluated. The results demonstrate that CIRN-derived regulatory networks are temporally conditioned representations of system dynamics rather than static structures, and that state, velocity, and curvature information contribute differently across lag regimes and temporal scales.

E. Structural Dependency Recovery in Damped Harmonic Oscillator Dynamics

To further evaluate whether CIRN can recover structural dependencies beyond first-order population interactions, the framework was applied to a damped harmonic oscillator governed by the second-order differential equation

$$\underbrace{\frac{d^2x}{dt^2}}_{\substack{\text{acceleration/inertial}\\ \text{acceleration/curvature}}} + \underbrace{1.1\frac{dx}{dt}}_{\substack{\text{damping term/}\\ \text{velocity/directional change}}} + \underbrace{20x}_{\substack{\text{restoring/stiffness}\\ \text{state/displacement}}} = 0$$

This system provides an important test case because its dynamics are not defined by interactions among multiple biological species or molecular variables, but by the internal relationship among displacement, velocity, and acceleration. In contrast to the Lotka–Volterra examples, where CIRN was expected to recover signed dependencies among interacting state variables, the damped oscillator tests whether the framework can reveal the structural organization embedded within a single second-order dynamical variable when it is represented through multiple temporal descriptions. The result is therefore a stringent assessment of whether CIRN can extract meaningful regulatory architecture from state, rate-of-change, and curvature information.

The oscillator trajectory in Supplementary Figure 22A exhibits the expected exponentially damped oscillatory behavior. The displacement $x(t)$ alternates around the equilibrium point with progressively decreasing amplitude, while the first derivative dx/dt and second derivative d^2x/dt^2 capture the associated velocity and acceleration patterns. These three temporal representations are not independent dynamical entities; rather, they describe different aspects of the same underlying motion. However, the governing equation imposes a precise structural relationship among them. Rearranging the oscillator equation gives

$$\frac{dx}{dt} = -\frac{1}{1.1}\frac{d^2x}{dt^2} - \frac{20}{1.1}x$$

which shows that both displacement x and acceleration d^2x/dt^2 should contribute negatively to the velocity response dx/dt. Thus, although CIRN does not attempt to estimate the explicit functional form of the differential equation, the expected signed dependency pattern is clear: $x \longrightarrow\!\!\!\!\!\!\mid\ dx/dt$ and $d^2x/dt^2 \longrightarrow\!\!\!\!\!\!\mid\ dx/dt$.

The joint all-predictors CIRN model successfully recovered this expected second-order dependency structure. As shown in Supplementary Figure 22B–C, both the displacement predictor x and the curvature predictor d^2x/dt^2 exerted strong inhibitory effects on the target response dx/dt. The posterior mean effect for $x \longrightarrow\!\!\!\!\!\!\mid\ dx/dt$ was $\omega = -15.93$, with a 95% highest density interval of $[-18.02, -13.94]$, while the posterior mean effect for $d^2x/dt^2 \longrightarrow\!\!\!\!\!\!\mid\ dx/dt$ was $\omega = -15.24$, with a 95% highest density interval of $[-17.24, -13.31]$. Both posterior distributions are sharply concentrated away from zero, indicating highly credible inhibitory effects. This result is notable because the inferred signed regulatory structure agrees with the rearranged second-order equation: velocity is negatively constrained by both displacement and acceleration.

This finding highlights a key strength of the CIRN framework. Although the method is formulated as a classification-based inference procedure rather than an equation-discovery algorithm, it was still able to recover the directional and signed dependency architecture implied by the governing differential equation. In this sense, CIRN does not reconstruct the oscillator equation symbolically, nor does it estimate the physical coefficients 1.1 and 20 directly. Instead, it identifies the structural

dependency skeleton that is compatible with the equation. The strong inhibitory edges from x and d^2x/dt^2 to dx/dt therefore represent a probabilistic, signed, and directed approximation of the dynamical constraints governing the oscillator.

The comparison between the joint all-predictors formulation and the separate predictor-representation formulation further strengthens this interpretation. When state, velocity, and curvature predictors were modeled jointly, CIRN recovered both inhibitory dependencies expected from the rearranged oscillator equation. However, when temporal representations were evaluated separately, only the state-level interaction $x \longrightarrow\!\!\!\!\shortmid\ dx/dt$ was retained, with a posterior mean effect of $\omega = -0.26$ and a 95% highest density interval of $[-0.37, -0.15]$. The curvature-based effect was not retained under this separate configuration because its highest density interval overlapped zero, with $\omega = 0.03$ and a 95% highest density interval of $[-0.07, 0.13]$. This contrast indicates that the second-order dependency becomes detectable when temporal representations are placed in a shared inferential context, but may remain unresolved when each representation is analyzed in isolation.

This behavior is consistent with the mathematical structure of the damped oscillator. The velocity response is not governed by displacement or acceleration separately; it is constrained by their joint relationship. When x and d^2x/dt^2 are considered together, they provide complementary information about the direction of motion. Displacement encodes position relative to equilibrium, while acceleration encodes local curvature and restoring tendency. Their combined inhibitory effects on dx/dt reflect the coordinated constraint imposed by the second-order dynamics. By contrast, modeling representations separately may remove the relational context needed to reveal this structure, causing the curvature contribution to appear uncertain even though it is part of the governing equation.

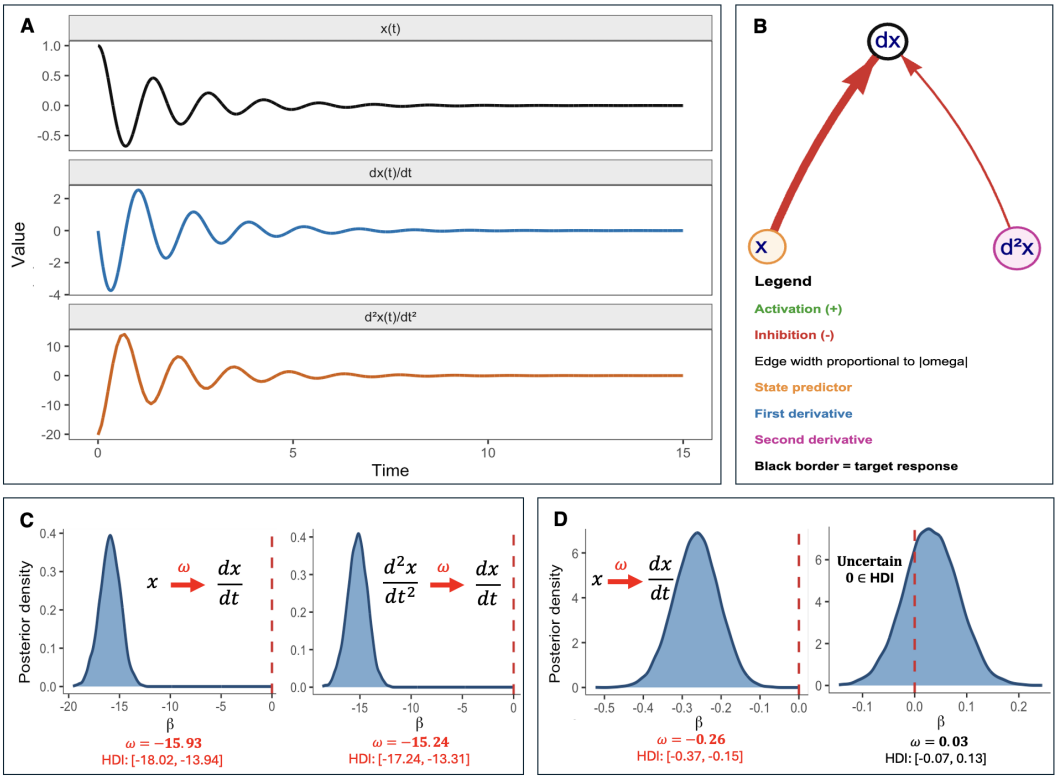
From a dynamical systems perspective, the result is particularly meaningful because it shows that CIRN can identify structural dependencies associated with feedback and damping using only temporal representations of the observed trajectory. The damped oscillator is characterized by a restoring tendency toward equilibrium and progressive loss of amplitude through damping. These mechanisms are expressed geometrically through the coupling among displacement, velocity, and acceleration. The recovered network captures this organization as a compact inhibitory regulatory architecture directed toward the velocity response. Thus, CIRN translates the geometry of damped oscillation into a signed regulatory representation, where displacement and curvature jointly constrain future directional change.

The result also clarifies the interpretive scope of CIRN. The framework should not be viewed as a replacement for symbolic regression or sparse equation-discovery methods when the objective is to recover exact differential equations. Rather, CIRN occupies a complementary position: it infers the signed and directed structural dependencies that are consistent with the observed dynamics, while retaining posterior uncertainty around each candidate interaction. In the damped oscillator example, CIRN does not recover the analytic equation in its original form; however, it recovers the critical signed dependency implied by the equation after rearrangement. This distinction is important because many biological, ecological, and environmental systems may not admit simple closed-form equations, yet their temporal trajectories may still contain recoverable structural dependency information.

The novelty of this result lies in the ability of a classification-based network inference framework to reveal a second-order dynamical dependency from temporal representations of a single variable. In most network-inference settings, regulatory structure is interpreted as a relationship among distinct observed variables. Here, CIRN reveals that regulatory architecture can also emerge across derivative orders of the same dynamical quantity. The inferred edges $x \longrightarrow\!\!\!\!\shortmid\ dx/dt$ and $d^2x/dt^2 \longrightarrow\!\!\!\!\shortmid\ dx/dt$ show that state and curvature jointly regulate the directional evolution of the system. This expands the interpretation of regulatory networks from interactions among variables to interactions

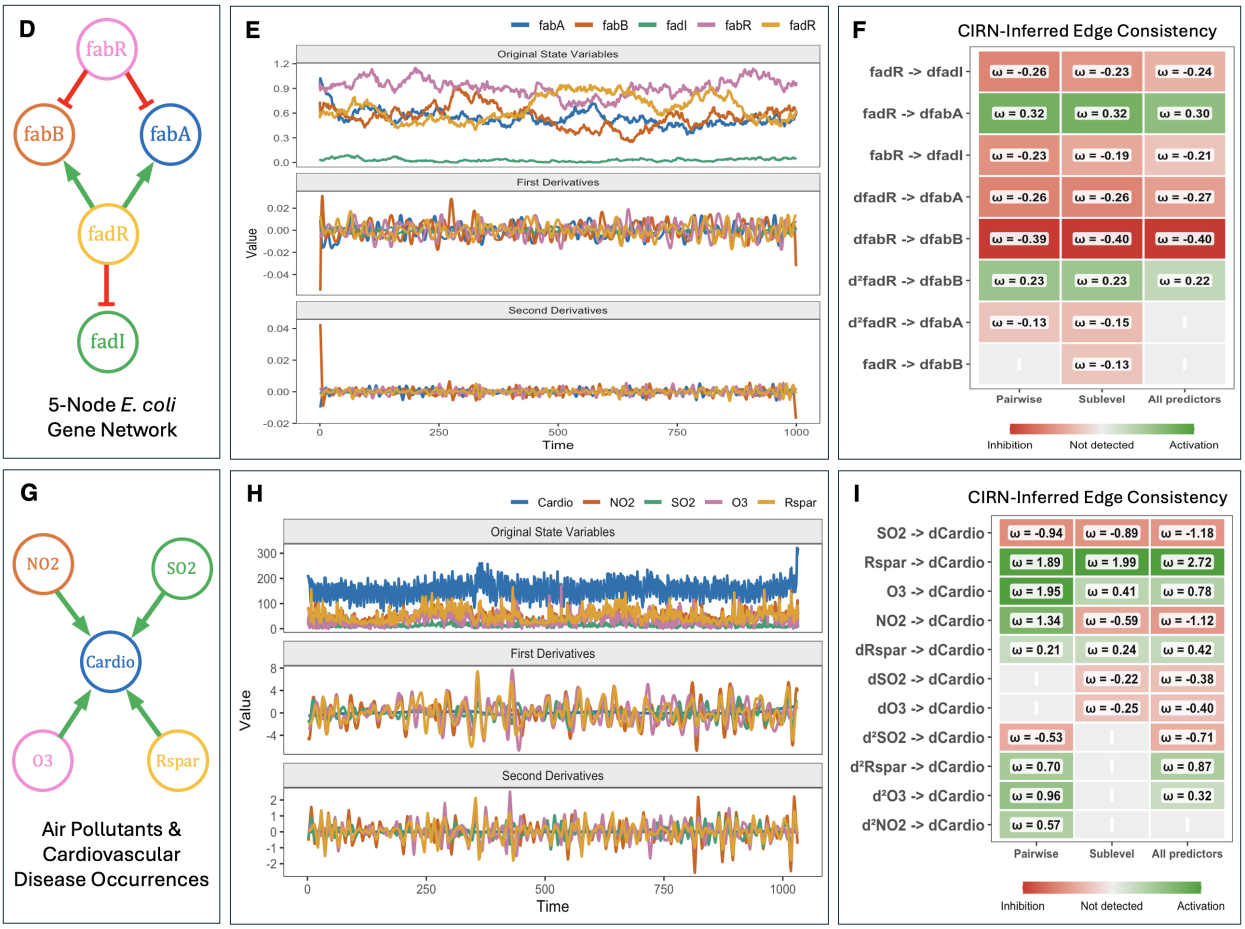
among temporal representations of system behavior.

Taken together, the damped oscillator experiment demonstrates that CIRN is capable of recovering structural dependency patterns embedded in second-order dynamics. The framework identifies the inhibitory organization implied by the governing equation, distinguishes credible from uncertain effects through posterior uncertainty, and shows that joint modeling of state, velocity, and curvature can reveal dynamical structure that may be missed under representation-specific analysis. This result strengthens the broader argument that CIRN does not merely detect associations among time-series variables; it extracts interpretable, signed, and directed regulatory constraints from the temporal geometry of dynamical systems. In doing so, CIRN provides a novel route for translating observed trajectories into probabilistic structural hypotheses, even when the underlying dynamics are governed by higher-order differential relationships.



Supplementary Figure 22: CIRN-based regulatory inference for a damped harmonic oscillator governed by $\frac{d^2x}{dt^2} + 1.1\frac{dx}{dt} + 20x = 0$**.** (A) Temporal representations supplied to CIRN, showing the original state variable $x(t)$, its first derivative dx/dt, and its second derivative d^2x/dt^2 over $t \in [0, 15]$. These trajectories correspond to displacement, velocity, and acceleration, respectively, and exhibit the characteristic exponentially decaying oscillations of a damped second-order system. (B) Directed and signed regulatory network inferred by CIRN when state-, velocity-, and curvature-based predictors are modeled jointly to explain the target directional response dx/dt. The inferred network retains inhibitory effects from both the state variable x and the second-derivative representation d^2x/dt^2 toward dx/dt, consistent with the negative feedback structure implied by the damped oscillator equation. Node-outline color denotes predictor representation (orange: state; blue: first derivative; magenta: second derivative), black-bordered nodes denote target responses, edge color denotes regulatory sign (green: activation; red: inhibition), and edge thickness is proportional to the magnitude of the posterior mean edge weight $|\omega|$. (C) Posterior coefficient distributions from the joint all-predictors formulation. Both $x \longrightarrow dx/dt$ and $d^2x/dt^2 \longrightarrow dx/dt$ exhibit strongly negative posterior means, $\omega = -15.93$ and $\omega = -15.24$, with highest density intervals excluding zero, indicating credible inhibitory contributions to the velocity response. (D) Posterior coefficient distributions obtained when temporal predictor representations are evaluated separately. Under this configuration, CIRN retains the state-level inhibitory interaction $x \longrightarrow dx/dt$ ($\omega = -0.26$), whereas the curvature-based interaction is not retained because its highest density interval overlaps zero. This contrast shows that joint modeling of multiple temporal representations can reveal second-order regulatory structure that may be weak or unresolved when state, velocity, and curvature predictors are analyzed independently. Node sizes are adjusted solely for visualization purposes and do not correspond to any quantitative network property.

F. Extended Applications Across Dynamical Systems



Supplementary Figure 23: Extended CIRN applications across synthetic genetic, *in silico* biological, and environmental-health dynamical systems *(cont.).* CIRN was evaluated across three multivariate systems that differ in regulatory structure, temporal behavior, stochasticity, and degree of mechanistic prior knowledge. (D–F) Five-node *Escherichia coli* fatty-acid regulatory network generated using GeneNetWeaver [12, 20] and analyzed using a 2-unit lag selected from lag-sensitivity assessment. (G–I) Air pollutant–cardiovascular disease occurrence system for Hong Kong [11, 17] analyzed using the default 1-unit lag. Panels D and G show the reference, simulated, or domain-supported candidate structures, where directed edges denote expected regulatory influences and edge color indicates regulatory sign: green for activation and red for inhibition. Panels E and H show the observed multivariate time series together with first- and second-derivative representations, allowing CIRN to evaluate information encoded in system state, rate of change, and local curvature. Panels F and I summarize posterior-supported CIRN interactions across pairwise, representation-specific multivariable (*sublevel*), and joint all-predictor configurations. Each cell reports the posterior mean edge weight, $\omega = \mathbb{E}[\beta \mid \text{data}]$, for retained interactions. Green cells indicate activating effects ($\omega > 0$), red cells indicate inhibitory effects ($\omega < 0$), and gray/blank cells indicate interactions not retained under the posterior HDI criterion. Color intensity reflects $|\omega|$. Because predictors were standardized before fitting, coefficients are comparable within a fitted model, but should not be interpreted as direct effect-size comparisons across systems or CIRN configurations. The figure illustrates CIRN's ability to recover signed and directed temporal dependencies across heterogeneous dynamical settings while revealing mode-consistent, representation-dependent, and conditioning-sensitive regulatory hypotheses.

The extended applications in Supplementary Figure 23 were designed to evaluate whether CIRN remains interpretable outside low-dimensional benchmark systems and under increasingly heterogeneous scientific conditions. The three examples differ in topology, temporal scale, signal-to-noise structure, and degree of prior mechanistic knowledge: a synthetic three-gene repressilator [17], a stochastic five-node *E. coli* fatty-acid regulatory network generated using GeneNetWeaver [12, 20], and an observational environmental-health system linking air pollutants to cardiovascular disease occurrence [11, 17]. This progression is useful because it moves from a closed synthetic regulatory

circuit, to an *in silico* biological network with known upstream and downstream variables, to an observational system in which causal interpretation requires especially careful constraint. Across these settings, CIRN is evaluated not as a fully automatic network-recovery algorithm, but as a probabilistic framework for generating signed and directed temporal regulatory hypotheses from expert-specified candidate structures.

A central lesson from these applications is that expert knowledge is not optional in CIRN; it is part of the inferential design. In the repressilator, all three genes can reasonably be treated as both predictors and targets because the system is a closed cyclic circuit. In the *E. coli* fatty-acid network, however, the candidate graph should not be fully symmetric: *fabR* and *fadR* are biologically plausible upstream regulators, whereas *fabA*, *fabB*, and *fadI* are downstream response genes. The cardiovascular application makes this point even more strongly. Cardio was specified as the only target, while NO_2, SO_2, O_3, and Rspar were treated as candidate predictors. If all variables were allowed to serve as both predictors and targets, CIRN could produce statistically supported but scientifically implausible edges, including reverse-direction relationships in which cardiovascular disease occurrence appears to regulate pollutant concentrations. The value of CIRN therefore depends not only on posterior edge estimation, but also on disciplined definition of the admissible target and predictor sets before model fitting.

The repressilator analysis in the main article provides a controlled test of whether CIRN can recover an inhibition-dominated temporal signature in an oscillatory genetic circuit. Using the 4-unit predictor–response lag, CIRN retained seven interactions across pairwise, sublevel, and all-predictors modes, all with consistent inhibitory sign and no cross-mode sign conflict. These included LacI $\rightarrow$ dLacI ($\omega = -1.28$, -1.26, and -1.98), TetR $\rightarrow$ dLacI ($\omega = -3.80$, -3.83, and -3.97), TetR $\rightarrow$ dTetR ($\omega = -0.92$, -1.06, and -1.65), $d\lambda cI \rightarrow d$TetR ($\omega = -2.25$, -0.66, and -1.92), LacI $\rightarrow$ $d\lambda cI$ ($\omega = -1.61$, -1.14, and -2.13), dLacI $\rightarrow$ $d\lambda cI$ ($\omega = -2.87$, -2.61, and -2.72), and $\lambda cI \rightarrow d\lambda cI$ ($\omega = -0.99$, -1.13, and -1.37). The strongest cross-mode effect was TetR $\rightarrow$ dLacI, indicating that TetR carries a particularly strong negative temporal signal for the subsequent directional response of LacI.

These repressilator results support the expected qualitative behavior of the system, but they also show why CIRN should not be read as a literal biochemical wiring diagram. In a phase-shifted oscillator, the sign and detectability of an effect depend on lag, phase position, and whether the predictive information is carried by state, velocity, or curvature (*see* Supplementary Figure 21). Thus, some expected relationships appear through derivative-mediated terms, while additional self-effects and mode-dependent CIRN edges also appear. For example, TetR $\rightarrow$ $d\lambda cI$ is retained as an activating effect in pairwise and sublevel modes, whereas $\lambda cI \rightarrow d$TetR is retained as an inhibitory effect in pairwise and all-predictors modes. These edges should not be interpreted as evidence for new direct molecular mechanisms. Rather, they indicate that the observed oscillatory trajectories contain phase-structured directional-change information that is only partially reducible to the static repression diagram. This is a strength of CIRN when interpreted correctly, because it reveals dynamically expressed regulatory signatures; it is also a limitation, because dynamic temporal dependencies are not identical to direct biochemical edges.

The *E. coli* fatty-acid network offers a more difficult biological test [12]. The trajectories exhibit compact stochastic fluctuations and relatively low-amplitude derivative structure, making edge recovery less visually obvious than in the repressilator. The results indicate that a 2-unit predictor–response lag was selected, which is biologically plausible because downstream gene-expression responses need not be maximally visible at immediate temporal alignment. Under this lag, CIRN retained six cross-mode interactions with no sign conflicts. Several of these correspond to the displayed reference architecture. The clearest state-level recovery was the activation of *fabA* by *fadR*, retained across pairwise, sublevel, and all-predictors modes with $\omega = 0.32$, 0.32, and 0.30. CIRN

also retained a negative first-derivative effect from *dfadR* to *dfabA* ($\omega = -0.26$, -0.26, and -0.27), suggesting that CIRN support for *fadR* is distributed across both its expression level and its local rate of change.

For *fabB*, the most stable recovered signal was *dfabR* → *dfabB*, which was inhibitory across all three modes ($\omega = -0.39$, -0.40, and -0.40). This agrees in sign with the reference inhibition of *fabB* by *fabR*, but CIRN detects the relationship primarily through the first derivative of the regulator rather than through raw expression level. CIRN also retained d^2*fadR* → *dfabB* as an activating effect ($\omega = 0.23$, 0.23, and 0.22), consistent with the positive role of *fadR* in the reference network but expressed through local curvature. These results illustrate one of the main advantages of CIRN over a state-only network model: it can indicate whether regulatory evidence is most strongly encoded in the current value of a variable, its direction of change, or its local acceleration.

For *fadI*, CIRN recovered the reference-consistent inhibition *fadR* → *dfadI* across all three modes ($\omega = -0.26$, -0.23, and -0.24). It also identified a stable inhibitory CIRN edge from *fabR* to *dfadI* ($\omega = -0.23$, -0.19, and -0.21). Because this latter edge is not part of the displayed reference diagram, it should be treated as an off-reference temporal dependency rather than definitive evidence of direct biochemical regulation. This is an important transparency point. CIRN can produce stable edges that are dynamically informative but mechanistically ambiguous. In this *E. coli* example, the VIF diagnostics do not indicate severe predictor collinearity: all group VIFs are close to 1, with the largest all-predictors VIF approximately 1.15 and the largest pairwise correlation approximately 0.31. Thus, the off-reference *fabR* → *dfadI* edge is less likely to be a simple collinearity artifact and more plausibly reflects an indirect, dynamically induced, or lag-specific temporal dependency. Reporting such edges is valuable, but they should be clearly separated from reference-confirming recoveries.

The cardiovascular disease occurrence application demonstrates why expert-constrained target selection is essential in observational settings. Unlike the repressilator or the GeneNetWeaver system, the environmental-health example is not a closed regulatory circuit. It would be scientifically inappropriate to allow Cardio to regulate air pollutants in the candidate graph simply because a statistical procedure could estimate such edges. By treating Cardio as the sole target and restricting NO_2, SO_2, O_3, and Rspar to predictor roles, the analysis focuses CIRN on scientifically admissible pollutant-to-Cardio temporal dependencies and reduces the risk of spurious network expansion. This design choice should be emphasized as a strength of the application: CIRN is most credible when the candidate graph is constructed from domain knowledge before posterior edge selection is performed.

Using the default 1-unit lag, the cardiovascular disease occurrence analysis results retained five cross-mode interactions, one of which displayed a sign conflict. SO_2 was consistently inhibitory for *d*Cardio across pairwise, sublevel, and all-predictors modes ($\omega = -0.94$, -0.89, and -1.18). Rspar was consistently activating at the state level ($\omega = 1.89$, 1.99, and 2.72), and *d*Rspar was also activating across all three modes ($\omega = 0.21$, 0.24, and 0.42). O_3 was retained as a positive state-level predictor across all modes, although the magnitude decreased from pairwise ($\omega = 1.95$) to sublevel and all-predictors configurations ($\omega = 0.41$ and 0.78). The effective-sample diagnostics suggest that these models were not limited primarily by sample size: the Cardio response had 1,029 usable observations, 432 minority-class observations, no adaptive jitter requirement, and an adequate sample-size flag even in the 12-predictor all-predictors model. These effects are therefore best interpreted as CIRN-inferred temporal directional-change effects on Cardio under the specified candidate graph, with the principal caution arising from predictor collinearity and observational confounding rather than from insufficient sample size.

The NO_2 result is especially important because it reveals both the value and the limitation of CIRN. In pairwise mode, NO_2 produced a positive CIRN edge into *d*Cardio ($\omega = 1.34$). Un-

der sublevel and all-predictors conditioning, however, the same pollutant produced negative CIRN edges ($\omega = -0.59$ and -1.12). This sign reversal is directly supported by the VIF and correlation diagnostics. At the original-state level, NO_2 is strongly correlated with Rspar ($r = 0.88$), moderately correlated with O_3 ($r = 0.61$), and enters a predictor set in which Rspar is flagged for high collinearity (VIF = 5.06) while NO_2 itself has an elevated VIF (VIF = 4.61). In the all-predictors model, the collinearity becomes more pronounced for the state variables, with Rspar and NO_2 flagged for high collinearity (VIF = 7.64 and 6.94, respectively). The derivative representations show the same structure: $d\mathrm{NO}_2$ and dRspar are strongly correlated ($r = 0.74$), and $d^2\mathrm{NO}_2$ and d^2Rspar remain correlated ($r = 0.68$). Thus, the NO_2 sign reversal should not be treated as an arbitrary instability; it indicates that NO_2 carries marginal temporal signal in pairwise mode, but that its conditional CIRN edge changes after accounting for highly collinear pollutant and meteorological predictors. Here, the mode comparison is not merely a robustness check; it functions as a diagnostic for conditional sensitivity. CIRN therefore helps identify which edges are stable and which require more cautious interpretation.

Derivative and curvature effects in the cardiovascular system further demonstrate the utility of CIRN's multi-representation design. The first derivative of O_3 was inhibitory in sublevel and all-predictors modes ($\omega = -0.25$ and -0.40), whereas the second derivative of O_3 was activating in pairwise and all-predictors modes ($\omega = 0.96$ and 0.32). SO_2 showed derivative- and curvature-mediated inhibitory effects, with $d\mathrm{SO}_2$ retained in sublevel and all-predictors modes and $d^2\mathrm{SO}_2$ retained in pairwise and all-predictors modes. Rspar showed state-level, first-derivative, and second-derivative activation. These findings suggest that CIRN may detect pollutant-to-Cardio temporal effects not only from pollutant levels, but also from changes and accelerations in pollutant trajectories. At the same time, these higher-order effects should be interpreted cautiously because derivatives can amplify measurement noise and may be sensitive to smoothing, sampling frequency, and lag choice.

The comparison of CIRN modes is therefore one of the most important methodological contributions illustrated by these applications. Rather than treating the three modes as interchangeable alternatives, CIRN should be used as a staged interpretive workflow. Pairwise mode is best used as a sensitive screening layer: it identifies marginal temporal dependencies that may be weak, lagged, or diluted in larger multivariable models. Its weakness is that it is most vulnerable to confounding, shared trends, and indirect temporal dependencies because each predictor is considered with limited conditioning. Sublevel mode should then be used to determine which temporal representation carries the strongest signal by evaluating predictors within the same representation class. This helps distinguish whether evidence is primarily state-based, velocity-based, or curvature-based while avoiding some cross-representation competition. Its weakness is that it can still retain representation-specific edges that disappear once all predictors compete together. All-predictors mode should be used as the most stringent conditional check because state, first-derivative, and second-derivative terms enter a joint model. Edges retained across all three modes should be treated as the highest-confidence CIRN hypotheses, whereas pairwise-only edges should be interpreted as exploratory marginal signals, sublevel-only edges as representation-dependent signals, and all-predictors-retained edges as conditionally supported signals. However, absence from all-predictors mode should not automatically be interpreted as absence of regulatory information, because real signals can be suppressed when predictors are collinear, derivatives are noisy, or the sample size is insufficient relative to the candidate predictor set. The results diagnostics illustrate these trade-offs empirically: the *E. coli* system has low VIFs and no sign conflicts, the cardiovascular system has adequate sample size but clear pollutant collinearity, and the repressilator all-predictors models show low minority-observation-per-predictor flags because only 79 usable observations are available for 8 predictors.

Thus, the three CIRN modes should not be ranked simply as good, better, and best. They answer different inferential questions. Pairwise mode asks whether a predictor contains marginal directional information about a future target response. Sublevel mode asks which temporal representation carries that information. All-predictors mode asks whether the information remains after competing predictors and representations are considered jointly. Cross-mode agreement strengthens confidence in an edge, as seen for several repressilator, *E. coli*, and cardiovascular interactions. Mode-specific detection, attenuation, or sign reversal is also informative because it reveals dependence on representation, conditioning, collinearity, or system context. This multi-mode structure makes CIRN more transparent than methods that output a single final graph without showing how edge support changes under different levels of conditioning.

The extended applications also expose several limitations that should be stated clearly. First, CIRN does not guarantee complete recovery of a known reference network all the time. Recovery depends on dynamic excitation, lag alignment, sampling frequency, noise level, derivative quality, and whether the candidate predictors contain the relevant causal signal–these are already embedded in CIRN and users are encouraged to explore different combinations of these during analysis. Second, CIRN infers temporal directional-change dependencies, not direct mechanistic causation. A retained edge means that a predictor helps classify or explain the future direction of a target response under a specified lag and model configuration. It does not, by itself, prove a direct biochemical, ecological, or environmental mechanism. Third, derivative and curvature terms can be powerful but fragile. They may reveal regulatory information that is invisible in state variables, but they can also magnify stochastic fluctuations and become sensitive to preprocessing choices. Fourth, all-predictors models are more conservative but may be affected by multicollinearity among state, derivative, and curvature terms. The cardiovascular VIF diagnostics provide a concrete example: high NO_2–Rspar collinearity changes the interpretation of NO_2 once multivariable conditioning is introduced. Fifth, small effective sample size relative to predictor count can make stringent multivariable modes fragile, as seen in the repressilator all-predictors configurations where minority-observation-per-predictor values fall to 3.0–4.75. Finally, expert knowledge is required not only to interpret edges after inference, but also to define the admissible candidate graph before inference.

These results support a balanced interpretation of CIRN. The method is valuable because it does more than estimate signed edges: it shows whether edge support is stable across inferential modes, whether a relationship is state-, velocity-, or curvature-mediated, and whether a candidate interaction is robust or conditioning-sensitive. The repressilator demonstrates recovery of an inhibition-dominated, phase-structured synthetic circuit. The *E. coli* network demonstrates biologically plausible recovery under stochastic perturbation while revealing representation-specific regulatory signals. The cardiovascular example demonstrates how CIRN can be used for expert-constrained hypothesis generation in observational environmental-health data, while also revealing the dangers of unconstrained target selection and the interpretive importance of mode-sensitive sign changes. CIRN's strength, therefore, is not that it always produces a positive or complete reconstruction; its strength is that it makes the stability, uncertainty, and limitations of temporal regulatory hypotheses visible.

G. System-Level Comparator Recovery Across Benchmark Systems

Supplementary Table 18 summarizes method recovery across all benchmark systems using a simplified interpretability scale. Rather than treating the comparator audit as a numerical ranking, the table asks whether each method recovered a usable system-level signal for the benchmark question. A "Yes" indicates interpretable recovery, "Partial" indicates useful but incomplete or nondirectional evidence, and "Cannot compute" identifies the specific data-structure limitation that prevented a

Supplementary Table 18: Simplified recovery summary for CIRN and comparator methods across all benchmark systems. Entries indicate whether each method recovered an interpretable system-level signal.

Benchmark	CIRN	Pearson	Spearman	Granger	CCM	PCM
Deterministic LV*	Yes	No	No	Yes	Partial	n/a
Stochastic LV	Yes	No	Partial	Yes	Partial	n/a
Figure 16a*	Yes	Partial	Partial	Cannot compute (collinearity)	Partial	n/a
Figure 16b*	Yes	Partial	Partial	Cannot compute (collinearity)	Partial	n/a
Figure 16c*	Yes	Partial	Partial	Yes	Partial	n/a
Figure 16d*	Yes	Partial	Partial	Cannot compute (mixed stationarity)	Partial	n/a
Figure 17e	Yes	Partial	Partial	Yes	Partial	n/a
Figure 17f	Yes	Partial	Partial	Yes	Partial	n/a
Figure 17g	Yes	Partial	Partial	Cannot compute (collinearity)	Partial	n/a
Figure 17h	Yes	Partial	Partial	Cannot compute (collinearity)	Partial	n/a
Figure 18i	Yes	Partial	Partial	Cannot compute (collinearity)	Partial	n/a
Figure 18j	Yes	Partial	Partial	Cannot compute (collinearity)	Partial	n/a
Figure 18k	Yes	Partial	Partial	Cannot compute (collinearity)	Partial	n/a
Figure 18l	Yes	Partial	Partial	Yes	Partial	n/a
Repressilator*	Yes	Partial	Partial	Partial	Partial	Partial
E. coli GNW-5 (stoch. 0.10)	Yes	Partial	Partial	Partial	Cannot compute (slow embedding)	Partial
Cardiovascular–air pollution	Partial	Partial	Partial	Partial	Cannot compute (noisy series)	Partial
Damped oscillator*	Yes	n/a	n/a	n/a	n/a	n/a

Notes: * means used in the main article; Yes=recovered an interpretable benchmark-level signal; Partial=returned useful but incomplete, nondirectional, overly dense, or assumption-limited evidence; No=no useful recovery in the parsed output; n/a=not applicable. Other entries give the one- or two-word reason a comparator was unavailable or not usable. CCM=convergent cross mapping; PCM=partial correlation mapping (at least 3 variables); LV=Lotka–Volterra; GNW-5=five-node GeneNetWeaver E. coli network with 10% stochasticity level.

comparator from being applied. This framing is appropriate because the comparator methods target different inferential objects: Pearson and Spearman correlation measure undirected association, Granger causality evaluates lagged predictive direction, CCM assesses nonlinear state-space cross-map skill, and PCM summarizes conditional association [7, 11, 21, 24]. CIRN instead evaluates whether predictor representations support the subsequent signed temporal response of a target variable.

Across the Lotka–Volterra benchmarks, CIRN recovered the expected predator–prey response structure in both deterministic and stochastic settings. This recovery is notable because the state-level correlation measures did not provide signed regulatory interpretation, while CCM returned bidirectional coupling evidence. Granger causality recovered lag-predictive signal in these two systems, but unlike CIRN, it does not directly encode activation versus inhibition in the target-response scale.

The geometric benchmark systems further illustrate the advantage of reporting recovery at the system level. CIRN produced interpretable retained edges across all Supplementary Figure 16a–18l systems. In contrast, Pearson, Spearman, and CCM frequently returned saturated or nondirectional dependence signals, and Granger causality could not be computed in several systems because of collinearity or mixed stationarity. These failures are informative rather than incidental: smooth

deterministic trajectories can contain clear temporal-response structure while simultaneously violating assumptions required by lag-regression comparators.

The multivariable benchmarks show that CIRN provides complementary evidence rather than simply duplicating existing methods. In the repressilator, CIRN recovered signed circuit-level response edges, whereas the comparator methods produced either sparse lag-predictive results or denser association/manifold summaries. In the *E. coli* GeneNetWeaver stochastic 0.10 system, CIRN recovered signed gene-response edges, while CCM could not be computed because embedding selection was unstable for the weak and noisy multivariable trajectories. In the cardiovascular–air pollution analysis, CIRN provided a signed observational response summary, but the result is conservatively marked as partial because the data are noisy, observational, and include one sign-conflict flag.

The damped oscillator represents a boundary case rather than a multivariable network benchmark. CIRN remained applicable for summarizing single-variable self-dynamics, whereas Granger causality, CCM, and PCM were not applicable in the same network-inference sense.

The comparator audit supports the central methodological claim that CIRN provides a signed, uncertainty-aware temporal-response framework. Standard comparators remain useful for association screening, lag prediction, nonlinear coupling assessment, and conditional association analysis [4, 14, 18]. However, CIRN contributes a distinct layer of evidence by producing interpretable signed response edges across ecological, synthetic, gene-network, observational, and single-variable dynamical systems.

XIX. SUMMARY OF THE SUPPLEMENTARY MATERIAL

This supplementary information consolidates the theoretical, statistical, computational, diagnostic, and interpretive foundations of CIRN. It explains how multivariate time-series trajectories are smoothed and transformed into state, first-derivative, and second-derivative representations; how the sign of the unstandardized target first derivative defines an increasing-versus-decreasing response after near-stationary observations are removed; and how temporally lagged predictor representations are standardized and fitted in target-wise Bayesian logistic regression models. Retained feature-to-target-response edges are identified when the posterior 95% highest density interval excludes zero, with the posterior mean coefficient determining the reported sign and effect magnitude on the target-response scale.

The supplement also documents the conditions required for defensible interpretation. These include explicit target and predictor sets, prevention of response leakage, scientifically justified lag selection, adequate increasing and decreasing response classes, stable derivative estimation, acceptable MCMC behavior, and careful review of collinearity and model-ready sample size. Pairwise, representation-specific sublevel, and joint all-predictors modes are presented as complementary inferential resolutions rather than interchangeable rankings. The adaptive response-jitter pathway is described as a qualified response-construction procedure for one-class or nearly one-class targets and must be disclosed whenever used. Sensitivity analysis, ground-truth benchmarking, cross-mode consistency, and the output compendium provide additional evidence for distinguishing stable findings from lag-sensitive, representation-dependent, collinearity-affected, or otherwise cautious regulatory hypotheses.

The computational sections provide an implementation-level algorithm, a complete guide to the scripted R workflow, a reproducibility checklist, and documentation of CIRN Studio as a guided interface to the same underlying method. The supporting applications show how CIRN behaves across deterministic and stochastic predator–prey dynamics, representative nonlinear systems, an oscillatory repressilator, an *E. coli* GeneNetWeaver network, an observational cardiovascular–air-

pollution system, and a damped harmonic oscillator. Collectively, these examples illustrate recovery of canonical signed interactions, representation-specific temporal information, conditioning-sensitive effects, and systematic lag-dependent changes in the inferred regulatory signature, while also exposing limitations arising from noise, phase alignment, collinearity, finite sample size, and incomplete mechanistic knowledge.

Taken together, the material positions CIRN as a comparatively accessible, equation-agnostic, directed, signed, uncertainty-aware, and representation-aware framework for generating temporally ordered regulatory hypotheses from observational or simulated time-series data. Its outputs should be interpreted as posterior-supported evidence conditional on the declared data, preprocessing, lag, predictor library, inference mode, prior, response construction, and diagnostic context. They do not, by themselves, establish interventional causality, direct biochemical or physical mechanism, complete governing equations, or unconditional regulatory structure. The principal contribution of CIRN is therefore not a claim of universal or complete network recovery, but a transparent and reproducible way to identify which lagged dynamical representations support target increase or decrease and to preserve the uncertainty and provenance needed for subsequent scientific evaluation.

REFERENCES

[1] Steven L. Brunton, Joshua L. Proctor, and J. Nathan Kutz. Discovering governing equations from data by sparse identification of nonlinear dynamical systems. *Proceedings of the National Academy of Sciences*, 113(15):3932–3937, 2016. doi: 10.1073/pnas.1517384113.

[2] Paul-Christian Bürkner. brms: An R package for bayesian multilevel models using Stan. *Journal of Statistical Software*, 80(1):1–28, 2017. doi: 10.18637/jss.v080.i01.

[3] Thalia E Chan, Michael P. H Stumpf, and Ann C Babtie. Gene regulatory network inference from single-cell data using multivariate information measures. *Cell Systems*, 5(3):251–267, 2017. doi: 10.1016/j.cels.2017.08.014.

[4] Riet De Smet and Kathleen Marchal. Advantages and limitations of current network inference methods. *Nature Reviews Microbiology*, 8(10):717–729, 2010. doi: 10.1038/nrmicro2419.

[5] Jonah Gabry and Rok Češnovar. cmdstanr: R interface to cmdstan. https://mc-stan.org/cmdstanr/, 2021. R package version 0.4.0.

[6] Andrew Gelman, Aleks Jakulin, Maria Grazia Pittau, and Yu-Sung Su. A weakly informative default prior distribution for logistic and other regression models. *The Annals of Applied Statistics*, 2(4):1360–1383, 2008. doi: 10.1214/08-AOAS191.

[7] Clive W. J. Granger. Investigating causal relations by econometric models and cross-spectral methods. *Econometrica*, 37(3):424–438, 1969. doi: 10.2307/1912791.

[8] Matthew D. Hoffman and Andrew Gelman. The no-u-turn sampler: adaptively setting path lengths in hamiltonian monte carlo. *Journal of Machine Learning Research*, 15(1):1593–1623, 2014.

[9] Vân Anh Huynh-Thu, Alexandre Irrthum, Louis Wehenkel, and Pierre Geurts. Inferring regulatory networks from expression data using tree-based methods. *PLOS ONE*, 5(9):e12776, 2010. doi: 10.1371/journal.pone.0012776.

[10] Nathan P. Lemoine. Moving beyond noninformative priors: why and how to choose weakly informative priors in bayesian analyses. *Oikos*, 128(7):912–928, 2019. doi: 10.1111/oik.05985.

[11] Siyang Leng, Huanfei Ma, Jürgen Kurths, Ying-Cheng Lai, Wei Lin, Kazuyuki Aihara, and Luonan Chen. Partial cross mapping eliminates indirect causal influences. *Nature Communications*, 11:2632, 2020. doi: 10.1038/s41467-020-16238-0.

[12] Daniel Marbach, Thomas Schaffter, Claudio Mattiussi, and Dario Floreano. Generating realistic in silico gene networks for performance assessment of reverse engineering methods. *Journal of Computational Biology*, 16(2):229–239, 2009. doi: 10.1089/cmb.2008.09TT.

[13] Adam A Margolin, Ilya Nemenman, Katia Basso, Chris Wiggins, Gustavo Stolovitzky, Riccardo Dalla Favera, and Andrea Califano. Aracne: an algorithm for the reconstruction of gene regulatory networks in a mammalian cellular context. *BMC Bioinformatics*, 7:S7, 2006. doi: 10.1186/1471-2105-7-S1-S7.

[14] Malvina Marku and Vera Pancaldi. From time-series transcriptomics to gene regulatory networks: A review on inference methods. *PLOS Computational Biology*, 19(8):e1011254, 2023. doi: 10.1371/journal.pcbi.1011254.

[15] Patrick E Meyer, Kevin Kontos, Frédéric Lafitte, and Gianluca Bontempi. Information-theoretic inference of large transcriptional regulatory networks. *EURASIP Journal on Bioinformatics and Systems Biology*, 2007:79879, 2007. doi: 10.1155/2007/79879.

[16] Thomas Moerman, Sara Aibar Santos, Carmen Bravo González-Blas, Jaak Simm, Yves Moreau, Jan Aerts, and Stein Aerts. Grnboost2 and arboreto: efficient and scalable inference of gene regulatory networks. *Bioinformatics*, 35(12):2159–2161, 2019. doi: 10.1093/bioinformatics/bty916.

[17] Seong Hoon Park, Seung Ha, and Jae Kyoung Kim. A general model-based causal inference method overcomes the curse of synchrony and indirect effect. *Nature Communications*, 14: 4287, 2023. doi: 10.1038/s41467-023-39983-4.

[18] Michael M Saint-Antoine and Abhyudai Singh. Network inference in systems biology: recent developments, challenges, and applications. *Current Opinion in Biotechnology*, 63:89–98, 2020. doi: 10.1016/j.copbio.2019.12.002.

[19] M. Sanchez-Castillo, D. Blanco, I. M. Tienda-Luna, M. C. Carrion, and Yufei Huang. A bayesian framework for the inference of gene regulatory networks from time and pseudo-time series data. *Bioinformatics*, 34(6):964–970, 2018. doi: 10.1093/bioinformatics/btx605.

[20] Thomas Schaffter, Daniel Marbach, and Dario Floreano. Genenetweaver: in silico benchmark generation and performance profiling of network inference methods. *Bioinformatics*, 27(16): 2263–2270, 2011. doi: 10.1093/bioinformatics/btr373.

[21] Ali Shojaie and Emily B Fox. Granger causality: a review and recent advances. *Annual Review of Statistics and Its Application*, 9(1):289–319, 2022. doi: 10.1146/annurev-statistics-040120-010930.

[22] Stan Development Team. Prior choice recommendations. https://github.com/stan-dev/stan/wiki/prior-choice-recommendations, 2023.

[23] Stan Development Team. *Stan Modeling Language Users Guide and Reference Manual*. Stan Development Team, 2024. Version 2.x.

[24] George Sugihara, Robert May, Hao Ye, Chih-Hao Hsieh, Ethan Deyle, Michael Fogarty, and Stephan Munch. Detecting causality in complex ecosystems. *Science*, 338(6106):496–500, 2012. doi: 10.1126/science.1227079.

[25] Polina Suter, Jack Kuipers, and Niko Beerenwinkel. Discovering gene regulatory networks of multiple phenotypic groups using dynamic bayesian networks. *Briefings in Bioinformatics*, 23 (4):bbac219, 2022. doi: 10.1093/bib/bbac219.

[26] Aki Vehtari, Andrew Gelman, Daniel Simpson, Bob Carpenter, and Paul-Christian Bürkner. Rank-normalization, folding, and localization: An improved r-hat for assessing convergence of mcmc. *Bayesian Analysis*, 16(2):667–718, 2021. doi: 10.1214/20-BA1221.

[27] Bin Zhang and Steve Horvath. A general framework for weighted gene co-expression network analysis. *Statistical Applications in Genetics and Molecular Biology*, 4(1):Article 17, 2005. doi: 10.2202/1544-6115.1128.